

PRICING FEATURES DO NOT PORT

Status: Documentation/investigation only. No code changed, no code removed from the legacy repository. This list identifies exactly what must NOT be copied, migrated, or re-implemented as-is in the rebuild. Every item cross-references `PRICING_LEGACY_ARCHITECTURE_MISTAKES_AND_PREVENTION.md`'s Mistake IDs where applicable.

1. Separate Wrap Lab calculator

- **File:** `routes/wrap/core.py`
- **Functions/classes:** `WrapPricingConfig` (L156-172), `_compute_pricing_snapshot()` (L665-746)
- **Pattern to avoid:** A category-specific pricing engine living outside the shared `PricingEngine`.
- **Why:** M-01, M-25. Produces a different price than the Foundation's `calculate_vehicle_graphics()` for the same job.
- **What replaces it:** Wrap Lab calls the shared `vehicle_wraps` calculator (Phase 4). The formula ITSELF ($(\text{design} + \text{production} + \text{install hours}) \times \text{labor_rate}$, single markup %) may be kept only as a **reference sanity-check**, not ported as the live mechanism.

2. Wrap-specific materials array

- **File:** `routes/wrap/core.py`
- **Classes/constants:** `WrapMaterialUpdate` (L142-153), `WRAP_MATERIAL_TYPES` (L175-179)
- **Pattern to avoid:** A per-ticket, per-category materials list disconnected from the shared Materials Library.
- **Why:** M-03.
- **What replaces it:** The shared `pricing_materials` collection (Phase 2), filtered by `compatible_categories` to `vehicle_wraps`.

3. Webstore disconnected `retail_price/base_cost`-only logic

- **File:** `routes/webstores.py`
- **Classes:** `Product` (L173-194, specifically `base_cost/retail_price` fields), `ProductCreate/ProductUpdate` (L197-225)
- **Pattern to avoid:** A manually-typed flat cost/price pair with zero calculator linkage.
- **Why:** M-02.
- **What replaces it:** Every Webstore product maps to a category calculator (or Custom); the calculator produces the internal `PricingResult`; an optional public override sits on top per the price-priority rule (Phase 8).

4. Giant category pricing dispatcher

- **File:** `server.py`
- **Function:** `calculate_pricing()` (L4356) and the surrounding ~3,700-line block of 9 `calculate_*` functions (L696-4356)
- **Pattern to avoid:** One monolithic if/elif dispatcher with category logic inlined in a single file.

- **Why:** M-05, M-06.
- **What replaces it:** `PricingEngine.calculate(category, input, tenant_config)` with small, independently testable per-category/per-family modules.

5. Raw float currency values

- **Pattern to avoid:** Any `float` type representing a dollar amount, and any ad hoc `round(x, 2)` call as a substitute for a documented rounding rule.
- **Where it appears:** Every field in all 9 `calculate_*` functions in `server.py`; every field in `routes/wrap/core.py::_compute_pricing_snapshot()` (11 separate `round(..., 2)` calls, L691-729); `routes/webstores.py::Product.base_cost/retail_price`.
- **Why:** M-04.
- **What replaces it:** `services/money.py::Money` — integer cents, one documented half-up rounding rule (Sprint 1, already built).

6. Legacy hours-based labor fallback

- **File:** `server.py`
- **Function:** `calculate_vehicle_graphics()`, the `labor_rates.get("production", {}).get("hourly_rate", ...)` branch (around L2800-2820), used only when the minute-based system (`get_labor_minutes_and_rate()`) returns zero.
- **Pattern to avoid:** A silent dual labor-costing system with no visible indicator of which path is active for a given tenant/category.
- **Why:** M-08.
- **Note:** Retirement is pending explicit Owner re-affirmation (still an Open Business Decision per `BANNERS_PORT_WIRING_SPEC.md` §3) — flagged here as "do not port as a permanent dual system," not as an instruction to delete it from the legacy repo today.

7. Empty `ai_prefill_overrides`

- **Field:** `ai_prefill_overrides` in the pricing configuration schema (exact model location not re-confirmed this session; carried forward from prior investigation, PRC-007)
- **Pattern to avoid:** A schema field with no reader/writer anywhere in the codebase.
- **Why:** M-10.
- **What replaces it:** Nothing, unless Phase 10 designs a real AI-prefill feature from scratch using the Historical Import's proven suggest/accept pattern.

8. Non-functional `category_setup_status`

- **Field:** `category_setup_status` (exact model location not re-confirmed this session; carried forward from prior investigation, PRC-008)
- **Pattern to avoid:** A UI status badge with no enforced meaning.
- **Why:** M-09.
- **What replaces it:** Either a real enforcement rule or removal — Phase 2 decision.

9. Duplicate Banner Setup Wizard — CONFIRMED redundant with the Pricing Setup Quiz

- **File:** `frontend/src/components/pricing/BannerSetupWizard.js` (501 lines)
- **Pattern to avoid:** A second, independent UI writing to `category_defaults.banners` — the exact same destination the Pricing Setup Quiz's own Banners section already writes to.
- **Why:** M-22. **Confirmed** this session — both files read directly; `BannerSetupWizard.js`'s own top-of-file comment states `// Saves to settings.category_defaults.banners`.
- **What replaces it:** Phase 9's unified setup quiz. The Wizard's step-by-step guided UX (Pricing Method → Materials & Rates → Minimum Charge → Labor → Add-ons → Templates → Review) may be preserved as a *UX pattern* inside the unified flow — but not as a second entry point.

10. Shop Rate Quiz — NOT confirmed redundant (correction to a prior assumption)

- **File:** `frontend/src/components/pricing/ShopRateQuiz.js` (534 lines)
- **Status:** This session confirmed `ShopRateQuiz.js` is a genuinely distinct, more detailed shop-labor-rate-derivation methodology (home/small/growing shop presets, billable-hours efficiency %, payroll burden %, profit buffer, rounding rule) — **not** a subset or duplicate of the Pricing Setup Quiz's simpler labor questions. **It is explicitly NOT on this do-not-port list.** See `PRICING_MODULE_EXTRACTION_INVENTORY.md` for its "Bring Over Formula/Business Rules Only" disposition.

11. Reports based on deprecated legacy job records

- **Files:** `routes/profit_analytics.py` (L167, L175 — `db.jobs.find()`, `db.job_items.find()`); `routes/jobs.py::sync_job_items_from_embedded_line_items()` (L100-101, self-documented as "legacy embedded"); `routes/invoices.py` (L33, L176 — imports and falls back to the same legacy sync function)
- **Pattern to avoid:** Any reporting or invoicing code path reading `jobs/job_items` instead of the live Order Item collection (`job_tickets`) or, post-rebuild, `PricingSnapshot` records directly.
- **Why:** M-18, M-19.
- **What replaces it:** Profitability/margin reporting and invoicing both read `PricingSnapshot` records exclusively (Phase 6).

12. Any unused/legacy pricing routes

- **Pattern to avoid:** Copying an endpoint into the rebuild simply because it exists, without confirming it is still called by any live UI.
- **Action for the rebuild team:** Before porting any route not explicitly covered elsewhere in this handoff package, confirm at least one live frontend call site exists. Routes with none should be flagged `Retire / Do Not Port` in the Extraction Inventory rather than carried forward by default.

13. Any separate price snapshot shapes

- **Confirmed 4 distinct shapes today** (see `PRICING_SHARED_SYSTEMS_AND_DATA_OWNERSHIP_MAP.md`'s "Pricing Snapshot" row for full detail):
 1. `job_tickets.pricing_snapshot` — `{pricing_mode, calculated_price, manual_price, active_price, calculation_breakdown, saved_at}` (`routes/job_tickets.py` L1809-1816)
 2. Wrap Lab's `_compute_pricing_snapshot()` output — `{pricing_method, total_billable_sqft, total_labor_cost, material_total, base_cost, markup_percent, markup_amount, suggested_price, per_sqft_price, manual_quoted_price, quoted_price, estimated_profit, estimated_margin_percent, computed_at}` (`routes/wrap/core.py` L731-746)
 3. `jobs.line_items[].cost_snapshot` (legacy, `routes/jobs.py` L118)
 4. Whatever Webstores does at checkout (unconfirmed — Needs More Investigation)
- **Pattern to avoid:** Porting any of these shapes as-is, or inventing a 5th.
- **What replaces it:** Exactly one `PricingSnapshot` shape (Sprint 1, `models/pricing_core.py`), used by every surface.

14. Any duplicated manual override flow

- **File:** `routes/job_tickets.py::save_ticket_pricing()` (L1791-1826) — `pricing_mode: "manual"`, `manual_price`, no reason/approver/permission
- **Also:** `routes/wrap/core.py::WrapPricingConfig.manual_quoted_price` (L172) — same gap
- **Pattern to avoid:** A bare number swap with no `PricingOverride` provenance and no `PRICING_OVERRIDE` permission check.
- **Why:** M-14, M-15.
- **What replaces it:** Sprint 1's `PricingOverride` schema + `Permission.PRICING_OVERRIDE`, wired into every commit endpoint in Phase 6.

15. Any duplicated materials, labor, markup, or overhead configuration

- **Confirmed duplicates:**
 - Materials: Wrap Lab's per-ticket array (item 2 above) vs. the shared Materials Library.
 - Apparel tiers: `routes/webstores.py::APPAREL_TIER_DEFAULTS` (L95-99) — a simple economy/standard/premium price-modifier table living inside Webstores, disconnected from whatever `ApparelPricingTable` the Foundation's `calculate_apparel()` actually uses.
 - Labor: the legacy hours-based fallback vs. the minute-based system (item 6 above) is a duplication-in-methodology, not duplication-in-location, but has the same net effect.
- **Pattern to avoid:** A second copy of any of these four config types living inside a category-specific or surface-specific module.
- **Why:** M-03, M-06, M-07, M-08.
- **What replaces it:** One `CategoryPricingConfig` per tenant per category (Phase 2), one `pricing_materials` collection, one overhead-basis rule, one labor-costing methodology — consumed identically everywhere.

PRICING SHARED SYSTEMS AND DATA

OWNERSHIP MAP

Status: Documentation/investigation only. No code changed, no migrations created.

Purpose: For every pricing-relevant entity, name exactly one owning system in the rebuild, who is allowed to read/write it, what must never be duplicated, and whether it follows the "live" or "frozen snapshot" rule (see

PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md §4E for the full live-vs-snapshot architecture statement).

Entity	Single Owner in Rebuild	Systems That Consume It	What Must Never Be Duplicated
Pricing Foundation Settings	One PricingFoundationSettings document per tenant (Phase 2)	Every calculator, every category config, Historical Import, AI Pricing Guidance (Phase 10)	A second Foundation-equivalent document for Wrap Lab or Webstore
Pricing Material	One pricing_materials collection per tenant (Phase 2), each material tagged with compatible_categories	All area-based calculators (Banners/Rigid Signs/Cut Vinyl/Digital Print/Vehicle Graphics), Wrap Lab (reads the same collection filtered to vehicle_wraps)	Wrap Lab's current per-ticket material routes/wrap/core.py::WrapMaterial any category-specific materials
Category Pricing Config	One CategoryPricingConfig document per tenant per category (Phase 2), including vehicle_wraps for both the Foundation calculator and Wrap Lab	The one calculator surface that prices that category, across every consuming screen (Quote Item, Order Item, Wrap Lab, Webstore product setup)	Wrap Lab's WrapPricingConfig routes/wrap/core.py L156-172 Webstore-specific pricing config
Pricing Template	One PricingTemplate collection per tenant (already exists, structure sound per routes/pricing.py L294-360)	Quote/Order Item creation ("apply a saved preset")	A second per-category template
Pricing Import (Historical Invoice Import)	One pricing_imports collection (already exists, routes/pricing_setup.py)	Feeds accepted values into PricingFoundationSettings/CategoryPricingConfig; writes a PricingChangeLog entry per acceptance	A second AI-suggestion acceptance (Phase 10's AI Pricing Guidance exact accept/reject pattern, not live)
Pricing Change Log	One pricing_change_logs collection per tenant (Sprint 1, models/pricing_core.py::PricingChangeLog, schema exists, not yet wired to any live data)	Every future write to PricingFoundationSettings, pricing_materials, or CategoryPricingConfig	A second, route-specific "setting" mechanism
Pricing Snapshot	One PricingSnapshot shape (Sprint 1, models/pricing_core.py), attached to whichever record committed it (Quote Item, Order Item, Wrap Project, Webstore Order Item)	Quotes, Orders/Order Items, Wrap Projects, Webstore Order Items, Profitability/Margin Reporting (Phase 6)	Confirmed today: at least 4 distinct "snapshot" shapes exist — (1) job_tickets.pricing_snapshot (per calculated_price/manual_price/calculation_breakdown, routes/wrap/core.py L1809-1816), (2) Wrap Lab's _compute_pricing_snapshot() (per suggested_price/quoted_price/markup_amount/estimated_price/estimated_margin_percent, routes/wrap/core.py L731-746), (3) Per-category generic warnings: jobs_line_items.cost_snapshot (confirmed pattern across all 9 routes/jobs.py L118), (4) what it does at checkout (unconfirmed, investigation). All 4 must collapse into one shape in the rebuild.
Pricing Warning	One PricingWarning shape (Sprint 1, typed type/severity/message)	Every calculator's PricingResult.warnings, carried into the PricingSnapshot.warnings at commit	

Pricing Override	One <code>PricingOverride</code> provenance shape (Sprint 1, <code>models/pricing_core.py</code>)	Attached to a <code>PricingSnapshot</code> whenever the committed price differs from the calculated <code>suggested_price_cents</code>	<code>job_tickets.pricing_snapshot</code> 's <code>manual_price/pricing_mode</code> : "n with zero reason, approver, or check (confirmed, <code>routes/job_t L1791-1826</code>); Wrap Lab's <code>manual_quoted_price</code> (<code>routes/L172</code>) with the same lack of pro
Vehicle table	One vehicle-type → base-sqft reference table, part of the Materials Library / <code>CategoryPricingConfig</code> for <code>vehicle_wraps</code> (Phase 2)	Foundation's <code>calculate_vehicle_graphics</code> (confirmed consumer, <code>server.py L2732-2736</code>); Wrap Lab (currently does NOT consume this — confirmed gap, Wrap	A second vehicle table scoped t
Vehicle coverage package	One coverage-tier → factor/benchmark table (spot/partial/half/full/custom, <code>server.py L2706-2728</code>), part of <code>CategoryPricingConfig</code> for <code>vehicle_wraps</code>	Foundation's <code>calculate_vehicle_graphics</code> (confirmed); Wrap Lab (does not consume this — Wrap Lab has no coverage-factor or benchmark-floor concept,	A second, Wrap-Lab-specific coverage/benchmark table
Vehicle install-time table	One vehicle-type-specific expected-install-hours reference, part of the minute-based labor system for	confirmed via <code>calculate_vehicle_graphics L150-172</code> labor section (<code>server.py L2799-2820</code>); Wrap Lab uses a <code>Webstore</code> type mapped to <code>install_hours</code> (Phase 8)	A second Wrap-Lab-specific in estimation mechanism
Apparel Pricing Table	One <code>ApparelPricingTable</code> structure not fully read this session — tagged Needs More Investigation in the Extraction Inventory placement, decoration method) feeding <code>calculate_apparel</code> and, per this task's scope, Webstore apparel products	Apparel calculator (Phase 9), <code>WrapPricingConfig.install_hours</code> , confirmed, L161)	A second Apparel price table in Webstore module (<code>APPAREL_TI routes/webstores.py L95-99</code> is this — confirmed a parallel, si apparel-tier concept already e Webstores, disconnected from
Customer Discount Profile	Does not exist today (confirmed absent — <code>models/customer.py::Customer</code> has no discount/wholesale/contract/tax-exempt fields). Phase 7 designs this as one profile per customer.	Quotes, Orders, Webstores (customer-specific pricing)	Foundation apparel calculator Phase 7 accidentally creating tw mechanisms (one for internal O Webstore checkout) instead of c profile
Quote	<code>quotes</code> collection, <code>QuoteLineItem</code> shape (<code>routes/quotes.py</code>)	Customer-facing proposal screens, <code>generate_invoice_from_order</code> -style conversions	A second Quote-equivalent emb Orders (Assumption to Verify v already exists)
Order	<code>orders</code> collection (<code>routes/orders.py</code>)	Order Items (<code>job_tickets</code>), invoicing, production	A second "order" concept
Order Item (currently "Job Ticket")	<code>job_tickets</code> collection — confirmed the real, current production/pricing unit (not <code>jobs/job_items</code> , which are legacy — <code>routes/jobs.py L100-101</code> explicitly calls <code>job_tickets</code> ("embedded"))	Work Order Summary, Inventory reservation (<code>material_requirements</code> keyed by <code>job_ticket_id</code>), Invoicing (<code>invoices.py</code> currently falls back to the legacy <code>jobs/job_items</code> path instead — confirmed mismatch)	The legacy <code>jobs/job_items</code> colle Phase 6 finishes the migration; Item" concept anywhere else
Work Order Summary	Derived/aggregated view generated from <code>job_tickets</code> (<code>routes/orders.py::generate_work_order</code>)	Production floor	A second, independently-maint summary
Inventory Item	<code>inventory_lots/inventory_items</code> (existing, well-built per <code>services/inventory_service.py</code> — no changes recommended , confirmed sound this session)	Material requirements, purchasing, pricing material cost lookups (Phase 2's <code>pricing_materials</code> should reference the same inventory	A second inventory-cost record inside <code>pricing_materials</code> with no the real inventory item
Vendor Cost	<code>models/inventory.py::VendorInput</code> /vendor price records (existing)	item where one exists, not duplicate cost data) More investigation item #4 is resolved — potentially <code>pricing_materials.cost_per_sqft</code>	A second, pricing-module-only record

Webstore Product	One Product per Webstore item, but its price comes from a category-calculator mapping (Phase 8), not a manually-typed retail_price/base_cost (confirmed current state: <code>routes/webstores.py::Product</code> L173-194 has zero calculator linkage today)	Storefront display, checkout, fundraiser profit allocation	A second flat-price-only product anywhere else in Webstores (Even if they currently reuse the same Product model, confirmed no separate Event Store product-pricing system exists, we should keep it that way)
Webstore Order Item	One line item per Webstore order, carrying its own PricingSnapshot (per the final rebuild rules)	Store profitability reporting, owner-share/platform-fee calculation	A duplicate snapshot shape distinct from Order Item's snapshot shape (both use the same PricingSnapshot model)
Wrap Project	The wrap-flavored job_tickets document (confirmed: Wrap Lab tickets live in the SAME job_tickets collection as general Order Items, just with additional wrap-specific fields — DesignBlock , WrapPricingConfig , materials , coverage, etc.)	Wrap Command Center UI, customer packet/PDF generation	A second Wrap-Project collection distinct from job_tickets (confirmed this doesn't currently happen — Wrap tickets and Order Item tickets share one collection, which is actually the correct pattern to follow)
Invoice	invoices collection (<code>routes/invoices.py</code>)	Payments, financial reporting	A second invoice-line-item shape distinct from the PricingSnapshot it should have (confirmed current anti-pattern: <code>jobs.line_items</code> vs <code>sync_job_items_from_embedded</code>)
Payment	Stripe/payment records (existing, out of pricing-rebuild scope except where it touches fees/payout)	Invoices, Webstore orders	N/A (out of scope)
Report Metric (profitability, margin)	Computed exclusively from PricingSnapshot records (Phase 6)	Owner-facing dashboards	Confirmed current anti-pattern: <code>routes/profit_analytics.py</code> L167 uses <code>db.jobs/db.job_items</code> (legacy) instead of <code>db.job_tickets</code> /its future PricingSnapshot replacement

Cross-cutting ownership rule (applies to every row above)

One collection, one shape, one set of consumers. Any time a second system (Wrap Lab, Webstores, a legacy `jobs/job_items` path) needs the same kind of data a shared entity already represents, the rebuild rule is to make that second system a *consumer* of the shared entity — never a second producer with its own shape. This is the single structural fix underneath nearly every "Confirmed" problem in this document and in `PRICING_MODULE_EXTRACTION_INVENTORY.md`.

PRICING REBUILD BUILD ORDER

Status: Documentation only. No implementation performed beyond what Sprint 1 (Phase 1) already completed and was previously reported. This order matches `PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md`'s phased plan; this document restates it in the exact 10-phase format required for this handoff package, cross-referenced against the fresh evidence gathered in this investigation pass.

Phase 1 — Shared Pricing Primitives ☒ COMPLETE (Sprint 1, 2026-02-15)

Field	Detail
Why this order	Every later phase needs a common money type, result/snapshot/warning shape, permission model, and test harness before any real formula code is written — building these alongside the first real category would mean rebuilding them again once inconsistencies surfaced.
Dependencies	None — this is the starting point.
Exact modules included	<code>services/money.py</code> (Money), <code>models/pricing_core.py</code> (PricingResult, PricingSnapshot, PricingWarning, PricingOverride, PricingChangeLog), <code>models/auth.py</code> additions (PRICING_VIEW/PRICING_EDIT_FOUNDATION/PRICING_OVERRIDE), <code>services/pricing_change_log.py</code> , <code>backend/tests/helpers/golden_formula.py</code> , <code>BANNERS_PORT_WIRING_SPEC.md</code>
What must not be built yet	Any calculator formula rewrite, any screen change, any config migration, any Wrap Lab/Webstore code
Required legacy data to extract	None — pure new infrastructure
Required tests	Unit tests only: Money math, permission role-mapping (with explicit no-regression assertions against pre-existing roles), change-log write helper, golden-formula harness self-test against a trivial fixture formula
Exit criteria	Met. 58/58 new tests passing; zero regression verified by running the full pre-existing <code>test_rbac.py/test_pricing*.py</code> suites before and after (identical results)

Phase 2 — Pricing Data Architecture

Field	Detail
Why this order	Every calculator ported in Phases 3-5 reads its config from here — building the calculator logic before its data shape is settled means rebuilding the calculator's config-reading code twice.
Dependencies	Phase 1 (<code>PricingChangeLog</code> must exist so material/config edits can log to it)
Exact modules included	<code>pricing_materials</code> collection (replacing the embedded array); <code>CategoryPricingConfig</code> per-tenant-per-category documents (replacing the nested dict-of-dicts, confirmed inconsistent shape per category); <code>PricingTemplate</code> (verify fit, likely unchanged); vehicle tables and coverage-package benchmarks as first-class reference data; Apparel price tables; a config-bundle-read function assembling <code>PricingFoundationSettings</code> + <code>CategoryPricingConfig</code> + filtered <code>pricing_materials</code> in one call (see the Owner-mandated live-recalculation requirement, Remediation Plan §4E and its Phase 2 subsection)
What must not be built yet	Any calculator formula rewrite, any screen change, wrap Lab/Webstore wiring
Required legacy data to extract	<code>pricing_configuration</code> (whole document), <code>category_defaults.*</code> (all 9 categories), embedded materials array (including <code>category="vehicle_type"</code> entries), <code>PricingTemplate</code> documents — see <code>PRICING_TO_REBUILD_MIGRATION_MAP.md</code> rows 1-8 for exact transformation detail
Required tests	Migration test (old embedded data → new collections, zero data loss, diffed against a real tenant's exported config); CRUD integration tests for materials/config; a live-recalculation timing test proving a config-bundle fetch happens once per session and zero additional database queries occur on subsequent input changes
Exit criteria	Every existing tenant's materials/category-config migrated with byte-for-byte equivalent values (restructured, not altered); old embedded fields no longer exist; the same <code>CategoryPricingConfig</code> bundle is provably reused by every surface that will eventually price that category

Phase 3 — Shared Live Calculator Experience and Area-Based Family

Field	Detail
Why this order	Banners is the simplest, most well-isolated category (per the investigation's own recommendation) — proving the shared live-calculation shell and area-based engine here, before touching Vehicle Graphics/Wrap Lab's much messier duplicate-engine problem, means Phase 4 unifies against an already-proven pattern instead of inventing it under pressure.
Dependencies	Phase 2 (config bundle must exist and be fast)
Exact modules included	Shared live calculator shell (debounced recompute on every input change, per <code>BANNERS_PORT_WIRING_SPEC.md §2h</code>); Banners port (first); Rigid Signs, Cut Vinyl, Digital Print ports (immediately after, same engine); shared cost-breakdown drawer UI; shared warnings/overrides UI; shared PricingSnapshot/Wrap Lab (Phase 4) deliberately deferred so its formula-comparison work happens with a proven pattern already in hand), Apparel/Services/Promotional/Custom (Phase 5), any Quote/Order/Reporting integration beyond what's needed to prove the snapshot-commit step works (full integration is Phase 6)
What must not be built yet	
Required legacy data to extract	Banners/Rigid Signs/Cut Vinyl/Digital Print formula logic from <code>server.py</code> (<code>calculate_banners</code> L2126-2683, <code>calculate_rigid_signs</code> L1698-2126, <code>calculate_cut_vinyl</code> L867-1214, <code>calculate_digital_print</code> L1214-1698) as business-rule reference, per <code>PRICING_MODULE_EXTRACTION_INVENTORY.md §B</code>
Required tests	A parity test proving the new engine matches the OLD dispatcher's live output for a representative input set per category (explicit acceptance criterion); golden-formula suites per category (exact-cents, warnings, rounding, minimum-charge, override, snapshot); a live-recalculation UI test proving every listed input (size, material, quantity, finishing, etc.) triggers a live price update with no page reload
Exit criteria	All 4 area-based categories produce byte-identical output to their legacy counterparts for the same inputs; live recalculation works in the calculator, a Quote Item editor, and an Order Item editor; a <code>PricingSnapshot</code> is created only on explicit commit and never on every keystroke

Phase 4 — Vehicle Graphics and Wrap Lab Unification

Field	Detail
Why this order	This is the single most dangerous duplicate system in the codebase (M-01/M-25, two engines with confirmed, materially different formulas for the same category) — it comes right after Phase 3 specifically so the unification has a proven area-based engine pattern to extend, rather than being the first thing built
Dependencies	Phase 3 (proven live-calculation shell + area-based engine pattern to extend with coverage/benchmark logic)
Exact modules included	Foundation versus Wrap Lab formula comparison (a documented side-by-side table, first build item, before any unification code); one canonical <code>vehicle_wraps</code> engine (extending the area-based engine with coverage %, benchmark floor, seam/difficulty multipliers, second installer); shared vehicle tables and shared materials (Wrap Lab reads the same <code>pricing_materials</code> collection); shared <code>PricingSnapshot</code> ; Wrap Lab UI wired to call the shared calculator instead of <code>_compute_pricing_snapshot()</code> ; retirement of <code>WrapPricingConfig/_compute_pricing_snapshot()</code> /the wrap-specific <code>materials</code> array
What must not be built yet	Apparel/Services/Promotional/Custom (Phase 5), full Quote/Order/Reporting integration (Phase 6)
Required legacy data to extract	Foundation's <code>calculate_vehicle_graphics()</code> (<code>server.py</code> L2683-3237) as the primary formula source; Wrap Lab's committed historical snapshots (see Migration Map's Wrap-specific rows) preserved as read-only history; Wrap Lab's <i>live</i> config structure (<code>WrapPricingConfig</code>) is explicitly not carried forward past this phase
Required tests	The formula-comparison table itself (documentation, reviewed before code); a parity test between old Wrap Lab output and new unified-engine output for representative inputs (expected to legitimately DIFFER in some cases, since the formulas were never actually equivalent — differences must be explained, not silently accepted); golden-formula suite for <code>vehicle_wraps</code> covering coverage tiers, benchmark floor, second installer, window perf
Exit criteria	Wrap Lab's pricing UI calls the shared engine exclusively; no <code>WrapPricingConfig/_compute_pricing_snapshot()</code> /wrap-specific materials code remains; a wrap job priced through Wrap Lab and the standalone calculator (same inputs) produce identical output

Phase 5 — Apparel, Services, Promotional, and Custom

Field	Detail
-------	--------

Why this order	These four categories don't share the area-based engine (Apparel is table-first, Services is time/unit/trip-based, Promotional is cost-plus, Custom is manual) — building them after the area-based family and Vehicle/Wrap unification means the shared primitives (Money, PricingResult, warnings, overrides, permissions, snapshot) are already proven across two structurally different formula styles before being applied to four more.
Dependencies	Phase 1 (primitives), Phase 2 (config architecture) — does NOT strictly depend on Phase 3/4's area-based engine internals, but comes after them in this plan because Banners/Vehicle-Wraps are the higher-risk, higher-evidence items to prove the pattern on first
Exact modules included	Apparel table-first module (blank cost, brand/style, quantity tiers, placement, decoration method, size breakdown, plus-size rules, add-ons, formula fallback); Services time/unit/trip module (hourly, flat, per-unit, travel, mileage, service-call minimum, equipment, subcontracting, permit fees, overtime); Promotional cost-plus module; Custom controlled manual module (still producing a real PricingResult/PricingSnapshot, going through the same permission gate — not an escape hatch)
What must not be built yet	Quote/Order/Reporting integration (Phase 6), Customer pricing (Phase 7), Webstore integration (Phase 8)
Required legacy data to extract	calculate_apparel() (server.py L3869-4356), calculate_services() (L3237-3739), calculate_promotional() (L696-867), calculate_custom() (L3739-3869) — all flagged "Needs More Investigation" for exact internal formula detail in PRICING_MODULE_EXTRACTION_INVENTORY.md §B; these must be read in full before this phase's build starts; not deferred further
Required tests	Phase 3 build starts; not deferred further; checking the pattern proven in Phase 3; a specific test that Custom still produces warnings/overrides/permission enforcement identically to every other category (guards against M-27/M-28-style "just fix it directly" shortcuts)
Exit criteria	All remaining 4 categories produce a standardized PricingResult/PricingSnapshot; no category-specific snapshot shape exists anywhere in the codebase

Phase 6 — Quotes, Orders, Work Orders, Inventory, Purchasing, and Reporting Integration

Field	Detail
Why this order	Only after every category produces a standardized, trustworthy PricingResult/PricingSnapshot does it make sense to rewire the systems that CONSUME pricing output — doing this earlier would mean rewiring Quotes/Orders/Reporting once per category as each one ships, instead of once total.
Dependencies	Phases 3-5 (all categories on the shared engine)
Exact modules included	Quote Item pricing (wired to PricingSnapshot, replacing the current independent QuoteLineItem shape); Order Item pricing (job_tickets gets the standardized snapshot, replacing its current {pricing_mode, calculated_price, manual_price, active_price, calculation_breakdown} shape and the missing-permission save_ticket_pricing endpoint); Work Order Summary estimates; material requirements (inventory) linkage; vendor-cost feedback loop (consolidating the Needs More Investigation item on whether PO scripts inform material cost); profitability/margin, rewriting required to read PricingSnapshot exclusively (retiring the confirmed jobs/job_items discount logic in routes/profit_analytics.py); invoice/deposit interaction (retiring sync_job_items_from_embedded_line_items's legacy fallback)
What must not be built yet	Customer-specific pricing (Phase 7), this phase wires the INFLUENCE these will plug into, not the PRICING_OVERRIDE; webstore integration (Phase 8)
Required legacy data to extract	The 3+ divergent Order Item / line-item representations (job_tickets, jobs/job_items, quotes.line_items, invoices.line_items) — consolidation plan per PRICING_TO_REBUILD_MIGRATION_MAP.md; routes/profit_analytics.py's current query shape as the "before" baseline for the reporting rewrite
Required tests	End-to-end test: a price calculated live, committed to a Quote Item, converted to an Order Item, invoiced, and appearing correctly in a profitability report — all from ONE PricingSnapshot, no sync step; a permission test that PRICING_OVERRIDE is still confirmed; the live commit endpoint (not just the test endpoint M-14)
Exit criteria	No sync_job_items_from_embedded_line_items-equivalent function remains; routes/profit_analytics.py reads PricingSnapshot-derived data exclusively; every Order Item/Quote Item/Invoice line item traces back to exactly one PricingSnapshot

Phase 7 — Customer Pricing

Field	Detail
-------	--------

Why this order	Customer-specific pricing (discounts, wholesale, contract rates) needs to plug into an already-standardized Quote/Order pricing flow (Phase 6) — building it earlier would mean building it against the fragmented pre-Phase-6 system and then rebuilding again.
Dependencies	Phase 6 (standardized Quote/Order pricing flow to plug into)
Exact modules included	CustomerDiscountProfile (new — confirmed nothing exists today to migrate); wholesale/reseller plan; contract and volume pricing roadmap; discount approval behavior (mirroring the PRICING_OVERRIDE approval pattern); reporting impact (discount amounts visible in profitability reports)
What must not be built yet	Webstore integration (Phase 8) — though this phase's design must anticipate Webstores as a future consumer of the same discount profile, per the Data Ownership Map's cross-cutting rule
Required legacy data to extract	None — confirmed nothing exists. This phase designs from scratch.
Required tests	A discount-profile customer produces a predictably different PricingResult than a standard customer for the same input, with the discount amount visible and auditable in reporting
Exit criteria	CustomerDiscountProfile is consumable by both internal Quotes/Orders and (in design, not yet wired) the future Webstore checkout path

Phase 8 — Webstore Pricing Integration

Field	Detail
Why this order	Webstores is the platform's most disconnected pricing surface (M-02, confirmed flat retail_price/base_cost with zero calculator linkage) — fixing it requires every other phase's infrastructure (shared engine, standardized snapshot, customer pricing) to already exist, or Webstores would just become a 5th place reinventing the same wheel.
Dependencies	Phases 2-7 (full shared engine, snapshot standard, and customer-pricing infrastructure)
Exact modules included	Product-category mapping (pricing_category field distinct from the current display-only ProductCategory enum, which confirmed does not map 1:1 onto the 9 calculator categories); product setup pricing (live calculator embedded in Webstore product creation/edit); public override rules (precedence chain: internal calculated price → optional public override → variant modifiers → discount, per M-26); fundraiser/event price rules (reusing the confirmed-sound compute_event_profit_allocation() math, now fed by real profit numbers); owner share/platform fee (preserving the confirmed-sound LockedSettings admin-lock pattern); Webstore order PricingSnapshot at checkout; store profitability reporting rewired onto real PricingSnapshot data
What must not be built yet	Shared platform fee (preserving the confirmed-sound LockedSettings admin-lock pattern); Webstore order PricingSnapshot at checkout; store profitability reporting rewired onto real PricingSnapshot data
Required legacy data to extract	Product/ProductVariant documents (re-categorized, not auto-re-priced — Owner-reviewed per Migration Map); APPAREL_TIER_DEFAULTS (reconciled into the ONE shared Apparel table, not kept as a second copy); LockedSettings /profit-allocation data (as-is)
Required tests	Every published product resolves to a non-null internal PricingResult ; a below-cost public override triggers the same below_cost warning any other category would; the price-priority chain test matrix (M-26); a checkout produces a PricingSnapshot per line item
Exit criteria	Zero Webstore products have a calculator-disconnected flat price; no separate Webstore pricing engine exists anywhere in the codebase

Phase 9 — Pricing Onboarding and Historical Import

Field	Detail
Why this order	Onboarding UX is how NEW tenants populate all the config built in every prior phase — it comes near the end because it needs the final CategoryPricingConfig /Materials Library/labor-rate shapes to be stable before designing the guided flow that fills them in; building it earlier would mean redesigning the quiz every time an earlier phase's schema changed
Dependencies	Phase 2 (final config schema), ideally Phases 3-5 complete (so the quiz's category sections write into fully-supported categories)

Exact modules included	Unified setup quiz (merging the Pricing Setup Quiz and Banner Setup Wizard's confirmed-duplicate write path into ONE flow — Banner Wizard's step-by-step UX pattern may be preserved as a pattern, Shop Rate Quiz's distinct shop-rate-burden methodology is preserved as a step, not discarded); Banner Wizard decision (retire as separate entry point, per M-22); Shop Rate Quiz decision (keep, integrated as one step, per this session's confirmed non-redundancy finding); save/resume (confirming whether any exists today — Needs More Investigation); quiz answer history (new, if confirmed absent per Migration Map); Historical Import audits (as-is, already sound); Historical Import benchmark rules (as-is)
What must not be built yet	AI Pricing Guidance (Phase 10)
Required legacy data to extract	<code>PricingSetupQuiz.js</code> (1,375 lines) and <code>BannerSetupWizard.js</code> (501 lines)'s derivation formulas (business-rule reference, per Extraction Inventory); <code>ShopRateQuiz.js</code> (534 lines)'s shop-rate-burden methodology (business-rule reference, confirmed distinct and worth preserving)
Required tests	A new tenant completing the unified quiz produces a fully-populated <code>CategoryPricingConfig</code> for every category they configure; confirm save/resume behavior explicitly, whatever the decision
Exit criteria	Exactly one UI entry point writes Foundation/category defaults; <code>BannerSetupWizard.js</code> as a standalone entry point is retired

Phase 10 — Future AI Pricing Guidance

Field	Detail
Why this order	AI suggestions are only trustworthy once there is a stable, standardized <code>PricingResult</code> / <code>PricingSnapshot</code> / <code>PricingChangeLog</code> history for the AI to read and compare against — building this first would mean the AI is trained/prompted against a moving, inconsistent target
Dependencies	Phases 1-9 (a full history of standardized snapshots and change logs to learn from)
Exact modules included	AI recommendations, AI material suggestions, AI labor suggestions, AI market comparison, AI unusual-price explanations, AI approval rules (reusing the Historical Import's proven suggest/accept pattern, per M-28's prevention rule against inventing a second AI-acceptance mechanism), AI credit tracking (token usage/cost logging, already flagged as a separate P1 future task in the platform's own backlog)
What must not be built yet	Nothing after this — it is the final phase
Required legacy data to extract	The Historical Import module's exact suggest/accept UI/API pattern (<code>routes/pricing_setup.py</code>) as the mandatory template
Required tests	AI suggestions never auto-apply without a human accept step (mirroring Historical Import); a below-cost AI suggestion still triggers the standard <code>PricingWarning</code>
Exit criteria	AI Pricing Guidance produces suggestions a human must explicitly accept, using the exact same acceptance mechanism Historical Import already proved out — no second, parallel AI-acceptance flow

Cross-phase note on live recalculation (applies to Phases 3-8)

Per the Owner's explicit correction (`PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md` §4E), every phase from 3 through 8 must implement continuous live price recalculation as a hard requirement — never an optional enhancement, never weakened, delayed, or replaced by the snapshot system. A `PricingSnapshot` is created only at explicit commit in every one of these phases; this is restated here so no future phase plan drifts from it.

PRICING LEGACY ARCHITECTURE MISTAKES AND PREVENTION

Status: Documentation/investigation only. No code changed.

Legend: **Confirmed** = read the actual code directly (this session or a prior session) and verified. **Assumption to Verify** = plausible, not yet read line-by-line. Every mistake includes evidence, damage caused, the rebuild's prevention rule, where that rule is enforced, and how it will be verified.

M-01: Duplicate Pricing Foundation and Wrap Lab engines

Field	Detail
Legacy Problem	Vehicle Graphics/Wraps has two completely independent pricing engines: the Pricing Foundation's <code>calculate_vehicle_graphics()</code> and Wrap Lab's <code>_compute_pricing_snapshot()</code> .
Evidence	Confirmed. <code>server.py::calculate_vehicle_graphics()</code> (L2683-3237): coverage-tier factors, waste-by-coverage, materials-library vehicle-sqft lookup, laminate, window perf, minute-based labor, package-benchmark floor.
Damage Caused	<code>routes/wrap/core.py::WrapPricingConfig/_compute_pricing_snapshot()</code> (L156-172, L665-746): flat design, hours+production hours+install hours) × labor_rate, no markup percent, no coverage %, no vehicle type, no waste, no materials library, no overhead, no benchmark floor. Two different unreconciled numbers with no shared audit trail. Owners cannot trust either number as the price for a vehicle wrap. Any future Foundation side rate change (labor rate, markup) never reaches Wrap Lab jobs, and vice versa.
Rebuild Prevention Rule	One canonical vehicle-wrap engine. Wrap Lab calls <code>PricingEngine.calculate("vehicle_wraps", input, tenant_config)</code> — the exact same function and config every other surface uses. Wrap Lab may not define its own formula, materials list, labor rate, markup rule, or snapshot shape (final rebuild rules, this task).
Enforced In	Phase 4 (Vehicle Graphics and Wrap Lab Unification)
Verification Test	A golden-formula parity test: identical vehicle/coverage/material inputs fed through the Wrap Lab UI and the standalone calculator UI must produce the exact same <code>PricingResult</code> to the cent.

M-02: Webstore flat retail pricing disconnected from internal cost and margin

Field	Detail
Legacy Problem	Webstore <code>Product.base_cost</code> / <code>retail_price</code> are manually-typed floats with no link to any calculator, material, labor rate, or markup rule.
Evidence	Confirmed. <code>routes/webstores.py::Product</code> (L173-194): <code>base_cost: Optional[float]</code> , <code>retail_price: Optional[float]</code> , no <code>pricing_category</code> , no <code>PricingResult</code> reference anywhere in the model.
Damage Caused	Owners can accidentally sell Webstore products below true production cost with no warning, because "cost" here is just whatever number they typed, not a real calculation. Margin/profitability reporting on Webstore sales is only as accurate as a manually-entered guess.
Rebuild Prevention Rule	Every Webstore product maps to a category calculator (or Custom); its internal price is computed live by the shared engine, exactly like any other category. A public retail override may exist on top, but never erases the internal calculated cost, margin, or a frozen <code>PricingSnapshot</code> at checkout (final rebuild rules, this task).
Enforced In	Phase 8 (Webstore Pricing Integration)
Verification Test	Every Webstore product must resolve to a non-null internal <code>PricingResult</code> before it can be published; a below-cost public override must trigger the same <code>below_cost PricingWarning</code> any other category would.

M-03: Duplicate materials libraries

Field	Detail
Legacy Problem	Wrap Lab keeps its own per-ticket <code>materials</code> array instead of reading the shared Pricing Foundation materials library.

Evidence	Confirmed. <code>routes/wrap/core.py::WrapMaterialUpdate</code> (L142-153), <code>WRAP_MATERIAL_TYPES</code> (L175-179) — a separate materials structure with its own <code>material_name/brand/cost_per_sqft</code> fields, scoped per-ticket rather than per-tenant; disconnected from <code>models/pricing.py::MaterialConfig</code> .
Damage Caused	A tenant updating a vinyl cost in Pricing Foundation has no effect on Wrap Lab jobs; material cost data has to be re-entered per wrap ticket instead of selected from a managed library, inviting typos and stale prices.
Rebuild Prevention Rule	One <code>pricing_materials</code> collection per tenant, filtered by <code>compatible_categories</code> . Wrap Lab reads this collection filtered to <code>vehicle_wraps</code> -compatible materials — it does not maintain its own list.
Enforced In	Phase 2 (collection created), Phase 4 (Wrap Lab wired to consume it)
Verification Test	After Phase 4, <code>routes/wrap/core.py</code> must contain zero material-cost fields that are not read from <code>pricing_materials</code> .

M-04: Float-based currency calculations

Field	Detail
Legacy Problem	All pricing math across every calculator and Wrap Lab uses raw Python <code>float</code> , with ad hoc <code>round(x, 2)</code> calls scattered throughout instead of one rounding rule.
Evidence	Confirmed. Every field in <code>server.py</code> 's 9 <code>calculate_*</code> functions and <code>routes/wrap/core.py::compute_pricing_snapshot()</code> (e.g., L691, L694, L703, L705, L706, L709-711, L718, L722, L726, L728-729 — eleven separate <code>round(..., 2)</code> calls in one function alone) is a plain float.
Damage Caused	Floating-point binary representation error accumulates silently across multi-step formulas (material → labor → overhead → markup → rush); rounding happens at a different, inconsistent point in every function, so two formulas that should agree on a boundary case can disagree by a cent.
Rebuild Prevention Rule	All money is an integer number of cents (<code>services/money.py::Money</code> , Sprint 1, already built and unit-tested), with one documented half-up rounding rule applied once per named cost bucket — never scattered <code>round()</code> calls.
Enforced In	Phase 1 (Sprint 1, <code>Money</code> utility — already complete), Phase 3-5 (every calculator ported onto <code>Money</code>)
Verification Test	<code>test_money_utility.py</code> (58 passing tests, Sprint 1) plus, per category, a golden-formula test asserting exact-cent output for a fixed input set.

M-05: Giant pricing dispatcher/switch logic

Field	Detail
Legacy Problem	One <code>calculate_pricing()</code> function in <code>server.py</code> if/elif-dispatches to 9 separate ~500-line functions, all living in a single ~4,800-line file.
Evidence	Confirmed. <code>server.py::calculate_pricing()</code> (L4356), category functions spanning L696-4356 — roughly 3,700 lines of pricing logic inside one monolithic file, alongside unrelated route handlers.
Damage Caused	Impossible to unit-test one category's formula in isolation without importing the entire server module; any shared-logic bug fix has to be manually copy-pasted into every affected function; the file's sheer size makes review and onboarding slow.
Rebuild Prevention Rule	One <code>PricingEngine.calculate(category, input, tenant_config) -> PricingResult</code> entry point, with category-specific formula code in small, independently testable modules — not one giant dispatcher file.
Enforced In	Phase 3-5
Verification Test	Each category's formula module must be importable and unit-testable without importing the full FastAPI app.

M-06: Separate calculator functions with repeated shared logic

Field	Detail
Legacy Problem	Waste application, overhead application, minimum-charge flooring, and rush-fee addition are each re-implemented inline inside every <code>calculate_*</code> function instead of being one shared helper.

Evidence	Confirmed for overhead (<code>calculate_overhead_cost()</code> at L336 is shared, but the <i>basis</i> passed to it differs per call site — see M-07); minimum-charge and rush-fee patterns confirmed present but not yet centralized (Extraction Inventory "Needs More Investigation" #2).
Damage Caused	A bug fix or business-rule change to "how rush fees work" has to be found and fixed in up to 9 places.
Rebuild Prevention Rule	Shared helpers on <code>Money (percentage(), apply_percentage_increase(),</code> etc., Sprint 1) plus category-family-level shared functions for waste/minimum-charge/rush, not per-category copies.
Enforced In	Phase 3 (area-based family), Phase 5 (remaining families)
Verification Test	Code review checklist: no two category modules may contain near-identical waste/overhead/rush logic blocks.

M-07: Inconsistent overhead rules

Field	Detail
Legacy Problem	<code>calculate_overhead_cost()</code> is shared, but different <code>calculate_*</code> functions may pass different cost bases into it (already flagged by Phase 0 Group 2 before this session; re-confirmed structurally present in <code>calculate_vehicle_graphics</code> where design/install pass-through costs are handled separately from the base labor+material overhead basis, L2799-2830).
Evidence	Confirmed function exists and is shared; per-category basis consistency needs a full line-by-line pass across all 9 functions (Extraction Inventory Item).
Damage Caused	Two categories with the same material+labor cost can produce different suggested prices because one silently includes design/install costs in the overhead basis and another doesn't.
Rebuild Prevention Rule	One canonical overhead basis (<code>material_cost + production_labor_cost</code> only; design/install are pass-through, added after markup) — already locked in Phase 0 Group 2 and reflected in Sprint 1's <code>PricingResult.overhead_basis_cents</code> field.
Enforced In	Phase 0 (locked), Phase 3-5 (every category ported must match this basis)
Verification Test	Golden-formula test per category asserting <code>overhead_basis_cents == material_cost_cents + production_labor_cost_cents</code> exactly.

M-08: Legacy hours-based labor fallback

Field	Detail
Legacy Problem	Two coexisting labor-costing systems: an older hourly-rate lookup and a newer minute-based system with an <code>include_labor</code> toggle, both still live in the same function.
Evidence	Confirmed. <code>server.py::calculate_vehicle_graphics()</code> L2799-2820: <code>labor_rates.get("production", {}).get("hourly_rate", ...)</code> (legacy) alongside <code>get_labor_minutes_and_rate(...)</code> (modern), with an explicit <code>if labor_minutes > 0:</code> branch choosing between them.
Damage Caused	Two tenants (or the same tenant on two different categories) can be silently using materially different labor-costing methodologies with no visible indicator of which one is active.
Rebuild Prevention Rule	Retire the hours-based fallback once the Owner explicitly re-affirms this decision (flagged as still open in <code>BANNERS_PORT_WIRING_SPEC.md</code> §3) — the minute-based system becomes the only labor-costing path.
Enforced In	Phase 3 (Owner decision required first)
Verification Test	After retirement, no <code>calculate_*</code> function may contain an hourly-rate-lookup branch.

M-09: Non-functional category setup status badges

Field	Detail
Legacy Problem	A <code>category_setup_status</code> UI badge exists but its enforcement was already flagged (prior session, PRC-008) as either not functioning or not acted upon anywhere.
Evidence	Assumption to Verify carried forward from prior investigation — not re-read this session; flagged in <code>PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md</code> Phase 2 scope (PRC-008).

Damage Caused	Owners may believe a category is "fully configured" (green badge) when it is actually running on stale defaults, or vice versa — false confidence or false alarm either way.
Rebuild Prevention Rule	Either the badge gets real enforcement (blocking calculation, or a real warning) or it is removed entirely — no silent, non-functional UI signal.
Enforced In	Phase 2
Verification Test	If kept: a test proving the badge's state changes calculator behavior. If removed: confirm no remaining UI reference.

M-10: Dead `ai_prefill_overrides` placeholder field

Field	Detail
Legacy Problem	A field named <code>ai_prefill_overrides</code> exists in the pricing configuration schema with no code path that reads or writes it.
Evidence	Assumption to Verify carried forward from prior investigation (PRC-007); not re-read this session.
Damage Caused	Dead schema surface area — confuses anyone reading the model into thinking AI prefill is an active feature.
Rebuild Prevention Rule	Remove entirely; if AI prefill of pricing defaults is wanted, it is designed fresh in Phase 10 using the Historical Import's proven suggest/accept pattern.
Enforced In	Phase 2
Verification Test	Field absent from the new <code>CategoryPricingConfig</code> schema.

M-11: Missing customer-specific pricing

Field	Detail
Legacy Problem	No per-customer pricing adjustment mechanism exists at all.
Evidence	Confirmed absent. <code>models/customer.py::Customer</code> read directly this session — no discount, wholesale, contract, or tax-exempt field of any kind.
Damage Caused	Shops that offer repeat-customer or trade discounts today can only do so by manually typing a different price on every quote, with no consistency, no audit trail, and no reporting visibility into how much revenue is discounted.
Rebuild Prevention Rule	Phase 7 designs one <code>CustomerDiscountProfile</code> entity, consumed identically by Quotes, Orders, and Webstores.
Enforced In	Phase 7
Verification Test	A customer with a discount profile produces a <code>PricingResult</code> reflecting that discount identically whether priced through an internal Quote or a Webstore checkout.

M-12: Missing wholesale/contract/volume pricing

Field	Detail
Legacy Problem	No wholesale tier, contract-rate, or volume-pricing mechanism exists.
Evidence	Confirmed absent — same <code>Customer</code> model read as M-11; quantity-tier discount logic inside calculators (Extraction Inventory item #2) is a different concept (quantity breaks on ONE order) from a standing wholesale/contract rate across ALL future orders.
Damage Caused	Reseller/wholesale relationships have no first-class support; shops manage this entirely outside the system today (manual price overrides, side agreements).
Rebuild Prevention Rule	Phase 7 roadmap item — designed alongside the Customer Discount Profile, not bolted onto individual calculators.

Enforced In	Phase 7
Verification Test	A contract-rate customer's price differs predictably and auditably from the standard calculated price, with the difference visible in reporting.

M-13: Missing margin and below-cost guardrails

Field	Detail
Legacy Problem	Generic string warnings exist, but there was no dedicated, typed below-cost/margin-warning mechanism with severity before Sprint 1.
Evidence	Confirmed — prior sessions verified the generic <code>warnings: [string]</code> pattern; Sprint 1 (this job, 2026-02-15) built the first typed replacement (<code>PricingWarning</code>), not yet wired into any live calculator.
Damage Caused	An employee could sell an item below cost with no system-level flag distinguishing "this is fine" from "this loses money," relying entirely on human attentiveness.
Rebuild Prevention Rule	Every <code>PricingResult</code> carries typed <code>below_cost/margin_warning</code> entries with severity; below-cost commits require <code>PRICING_OVERRIDE</code> permission and an override reason (Phase 0 Group 3/4, Sprint 1 schema).
Enforced In	Phase 1 (schema — done), Phase 3-5 (wired into every category)
Verification Test	<code>test_golden_formula_framework.py</code> 's below-cost/margin-warning test cases (Sprint 1, passing) as the template every category's golden-formula suite must replicate.

M-14: Weak or absent manual override permissions

Field	Detail
Legacy Problem	Any authenticated user can call the manual-price-save endpoint with no permission check specific to pricing overrides.
Evidence	Confirmed. <code>routes/job_tickets.py::save_ticket_pricing()</code> (L1791-1826) — no <code>require_permission</code> dependency visible gating <code>pricing_mode: "manual"</code> , <code>manual_price</code> .
Damage Caused	Any staff member, regardless of role, could set an arbitrary manual price with no system-level check, including selling below cost.
Rebuild Prevention Rule	<code>Permission.PRICING_OVERRIDE</code> (Sprint 1, <code>models/auth.py</code> — Owner/Admin self-approving, Staff must route through a pending-approval workflow).
Enforced In	Phase 1 (permission — done), Phase 6 (wired into the actual commit endpoints)
Verification Test	<code>test_pricing_permissions_sprint1.py</code> (Sprint 1, passing) verifies Staff lacks <code>PRICING_OVERRIDE</code> ; a Phase 6 integration test must verify the live commit endpoint actually enforces it.

M-15: Missing override provenance

Field	Detail
Legacy Problem	A manual override records only the number itself — no reason, no who, no approval state.
Evidence	Confirmed. <code>routes/job_tickets.py::save_ticket_pricing()</code> stored fields: <code>pricing_mode</code> , <code>calculated_price</code> , <code>manual_price</code> , <code>active_price</code> , <code>calculation_breakdown</code> , <code>saved_at</code> — no <code>reason</code> , <code>overridden_by_user_id</code> , or <code>approval_status</code> . Wrap Lab's <code>manual_quoted_price</code> (WrapPricingConfig L172) has the identical gap.
Damage Caused	No way to answer "why was this priced below the calculated suggestion?" months later, during a dispute or a margin-erosion investigation.
Rebuild Prevention Rule	<code>PricingOverride</code> (Sprint 1 schema): original price, final price, reason from a fixed taxonomy, <code>overridden_by_user_id</code> , <code>approval_status</code> , approver.
Enforced In	Phase 1 (schema — done), Phase 6 (wired into commit flow)
Verification Test	<code>test_golden_formula_framework.py::test_manual_override_below_cost</code> (Sprint 1, passing) as the template; Phase 6 integration test that a live override always produces a populated <code>PricingOverride</code> block.

M-16: Missing pricing configuration history

Field	Detail
Legacy Problem	No record of who changed a material cost, labor rate, or markup, or what the value was before.
Evidence	Confirmed absent prior to Sprint 1 — no such mechanism found anywhere in <code>routes/pricing.py</code> 's settings-update endpoints.
Damage Caused	If a price suddenly looks wrong, there is no way to see what config value changed, when, or by whom.
Rebuild Prevention Rule	<code>PricingChangeLog</code> (Sprint 1 schema, <code>services/pricing_change_log.py::log_pricing_change()</code>) — write-through helper every future config-save route must call.
Enforced In	Phase 1 (schema — done), Phase 2 (wired into settings/material/category-config save routes)
Verification Test	<code>test_pricing_change_log.py</code> (Sprint 1, passing); Phase 2 integration test that saving a material cost produces exactly one new <code>PricingChangeLog</code> entry.

M-17: Missing pricing audit/change log (route-level enforcement)

Field	Detail
Legacy Problem	Same root cause as M-16, called out separately because it also covers the <i>enforcement</i> gap — even once a log schema exists, nothing forces every write path to use it.
Evidence	Confirmed — same evidence as M-16; the schema is Sprint 1-only, no route calls it yet.
Damage Caused	Same as M-16, plus the risk that only <i>some</i> config-save routes get the audit call wired in, leaving silent gaps.
Rebuild Prevention Rule	Code-review rule: any new/modified route that writes to <code>PricingFoundationSettings</code> , <code>pricing_materials</code> , or <code>CategoryPricingConfig</code> MUST call <code>log_pricing_change()</code> in the same transaction/request.
Enforced In	Phase 2
Verification Test	Grep-based CI check (or code review checklist) confirming every settings-write route imports and calls <code>log_pricing_change</code> .

M-18: Pricing snapshots not feeding profitability reports

Field	Detail
Legacy Problem	Profitability/margin reporting reads a different collection than the one that actually carries live pricing snapshots.
Evidence	Confirmed. <code>routes/profit_analytics.py</code> L167 (<code>db.jobs.find(...)</code>), L175 (<code>db.job_items.find(...)</code>) — reads the legacy collections, not <code>db.job_tickets</code> (which is where <code>pricing_snapshot</code> actually lives, per <code>routes/job_tickets.py</code>)
Damage Caused	Margin/profit dashboards can be stale or simply wrong relative to what was actually priced and committed on live jobs.
Rebuild Prevention Rule	All profitability/margin reporting reads <code>PricingSnapshot</code> records exclusively.
Enforced In	Phase 6
Verification Test	A test order priced and committed through the live flow must appear correctly in a profitability report within the same test run, with no separate "sync" step required.

M-19: Reports reading legacy/deprecated jobs or job-item records

Field	Detail
Legacy Problem	At least three different "order item" representations coexist: <code>job_tickets</code> (current), <code>jobs/job_items</code> (explicitly labeled legacy in its own code), and embedded shapes inside <code>quotes/invoices</code> .

Evidence	Confirmed. <code>routes/jobs.py::sync_job_items_from_embedded_line_items()</code> (L100-101) docstring: "Ensure legacy embedded line_items are also represented in the job_items collection." <code>routes/invoices.py</code> L33 imports this exact function and uses it as a fallback (L176). <code>routes/profit_analytics.py</code> reads <code>jobs/job_items</code> directly (M-18), which is exactly the kind of place data silently drifts out of sync.
Damage Caused	Same underlying data (an order's line items) has to be kept in sync across 3+ representations by hand-written sync functions, which is exactly the kind of place data silently drifts out of sync.
Rebuild Prevention Rule	One Order Item representation (<code>job_tickets</code> , or its rebuilt successor), consumed directly by Invoices and Reporting — no sync functions, no legacy fallback path.
Enforced In	Phase 6
Verification Test	After Phase 6, <code>sync_job_items_from_embedded_line_items</code> -equivalent functions no longer exist in the codebase; Invoices and Reporting both read the same single collection.

M-20: Missing inventory/material-usage connection (to pricing)

Field	Detail
Legacy Problem	Whether a material's actual on-hand inventory or vendor cost ever flows back into what the calculator charges is unconfirmed.
Evidence	Assumption to Verify — <code>services/inventory_service.py</code> itself is well-built (confirmed, reservation/allocation/lot tracking), but whether <code>pricing_materials.cost_per_sqft</code> is ever informed by a real inventory/vendor record versus being independently manually maintained was not confirmed this session.
Damage Caused	If unconfirmed and actually absent, a shop could be charging based on a stale manually-entered material cost while its actual vendor cost has changed, silently eroding margin.
Rebuild Prevention Rule	Phase 2/6 explicitly investigates and, if missing, builds a defined (not necessarily automatic) link — e.g., a "review suggested material cost updates from recent PO receipts" flow, mirroring the Historical Import's suggest/accept pattern.
Enforced In	Phase 2 (investigate), Phase 6 (integration)
Verification Test	A PO receipt at a different cost than the current <code>pricing_materials</code> value produces a visible, actionable signal (not a silent no-op and not a silent auto-overwrite).

M-21: Missing purchasing/vendor-cost connection (to pricing)

Field	Detail
Legacy Problem	Same category as M-20, specifically the vendor/PO side.
Evidence	Assumption to Verify — <code>models/inventory.py::VendorInput/PurchaseOrderInput</code> exist but their feedback loop into <code>pricing_materials</code> was not read this session.
Damage Caused	Same as M-20.
Rebuild Prevention Rule	Same as M-20.
Enforced In	Phase 2/6
Verification Test	Same as M-20.

M-22: Duplicate onboarding/configuration flows — Banner Setup Wizard vs. Pricing Setup Quiz overlap

Field	Detail
Legacy Problem	Two entirely separate UIs both write to the exact same configuration destination for Banners.
Evidence	Confirmed this session — read both files directly. <code>frontend/src/components/pricing/BannerSetupWizard.js</code> top-of-file comment: <code>// Saves to settings.category_defaults.banners.</code> <code>frontend/src/components/pricing/PricingSetupQuiz.js</code> (L45-52, L165, L241-244, L337-338, L486-492) has its

	own Banners section deriving <code>sell_rate/min_sell</code> /labor defaults, feeding the same <code>category_defaults.banners</code> path.
Damage Caused	An owner could run the general Quiz, get one set of Banner defaults, then separately run the Banner Wizard and overwrite them with a different methodology's output — with no indication either flow knows the other exists. Confusing, and risks silently discarding carefully-derived numbers.
Rebuild Prevention Rule	Phase 9 builds ONE unified setup quiz. The Banner Wizard's step-by-step guided UX pattern may be preserved as a <i>pattern</i> inside the unified flow, but not as a second, independent entry point writing the same destination.
Enforced In	Phase 9
Verification Test	After Phase 9, there is exactly one UI entry point that can write <code>category_defaults.banners</code> -equivalent config.

M-23: Unclear cost/margin visibility rules

Field	Detail
Legacy Problem	Whether Staff-role users can see internal cost/margin figures (as opposed to only the final price) versus Owner/Admin was flagged in prior investigation as inconsistently enforced (PRC-037).
Evidence	Assumption to Verify — carried forward from prior session; not re-read this session.
Damage Caused	Staff might see internal cost/margin data they shouldn't (competitive-sensitivity/trust issue), or conversely be unable to see data they need to do their job.
Rebuild Prevention Rule	A dedicated cost-visibility permission, separate from <code>PRICING_VIEW</code> (which grants seeing the <i>price</i> , not necessarily the internal cost breakdown) — final decision and exact permission name is a Phase 6 design item, informed by Phase 0's existing direction.
Enforced In	Phase 6
Verification Test	A Staff-role test account's API response for a <code>PricingResult</code> must have cost/margin fields present or absent exactly per the locked rule, tested explicitly.

M-24: Missing standardized warning codes and severity

Field	Detail
Legacy Problem	Every calculator returns free-text warning strings with no machine-readable type or severity.
Evidence	Confirmed pattern (see M-13); Sprint 1 replaces it with <code>PricingWarning.type/severity</code> .
Damage Caused	UI cannot distinguish "just FYI" from "block this from being sold" without brittle string-matching on English text.
Rebuild Prevention Rule	Typed <code>PricingWarning</code> (already built, Sprint 1).
Enforced In	Phase 1 (done), Phase 3-5 (wired in)
Verification Test	Same as M-13.

M-25: Wrap formula differences between Foundation and Wrap Lab

Field	Detail
Legacy Problem	Distinct from M-01 (which is about the existence of two engines) — this specifically documents that the two engines' formulas produce materially different numbers for comparable inputs (no waste %, no coverage-tier concept, no overhead concept, no benchmark floor on the Wrap Lab side, versus a much more sophisticated Foundation formula).
Evidence	Confirmed via direct side-by-side reading this session (<code>server.py</code> L2683-2820 vs. <code>routes/wrap/core.py</code> L156-172, L665-746).
Damage Caused	Same damage as M-01, called out separately because the fix (Phase 4) explicitly requires a formula-comparison step BEFORE unification, not just a plumbing change — the numbers genuinely disagree today, not just the code paths.

Rebuild Prevention Rule	Phase 4's first build item is explicitly "Foundation versus Wrap Lab formula comparison" (already in the Build Order) before any unification code is written.
Enforced In	Phase 4
Verification Test	A documented side-by-side comparison table (same inputs, both formulas, both outputs) produced and reviewed before Phase 4's unification code begins.

M-26: Missing Webstore price priority rules

Field	Detail
Legacy Problem	When multiple price-affecting things could apply to a Webstore product (base retail price, variant <code>additional_cost</code> , fundraiser profit allocation, any future customer/discount rule), there is no documented precedence order.
Evidence	Assumption to Verify carried forward from prior investigation (PRC-033); not re-read this session, but the underlying model fields (<code>Product.retail_price</code> , <code>ProductVariant.additional_cost</code> , fundraiser allocation) are confirmed to coexist with no visible precedence logic read this session.
Damage Caused	Ambiguous which value "wins" when a public override exists alongside a calculated internal price and a variant modifier.
Rebuild Prevention Rule	Phase 8 documents an explicit precedence chain: internal calculated price → optional public override → variant modifiers → any future discount — evaluated in one place, not scattered.
Enforced In	Phase 8
Verification Test	A test matrix covering every combination of override/variant/discount presence, asserting the documented precedence order holds.

M-27: Building UI before locking pricing business rules

Field	Detail
Legacy Problem	Historically, calculator screens and wizards were built before the underlying business rules (rounding, overhead basis, warning taxonomy, override workflow) were formally decided and written down anywhere.
Evidence	Confirmed by the very existence of inconsistencies documented across M-04, M-07, M-08 — these could only arise if formula decisions were made ad hoc per screen rather than centrally decided first.
Damage Caused	Every new category or screen re-derives its own answer to "how do we round," "what's the overhead basis," etc., compounding inconsistency over time.
Rebuild Prevention Rule	Phase 0 (already completed, <code>PHASE_0_PRICING_DECISIONS_FORMULA_GOVERNANCE.md</code>) locks every formula/business-rule decision BEFORE any calculator UI is rebuilt — this is the entire reason Phase 0 was done first in this project.
Enforced In	Phase 0 (done), enforced procedurally for every subsequent phase
Verification Test	Any new category port must cite a Phase 0 decision ID for every formula choice it makes; a formula choice with no corresponding Phase 0 decision is a process violation to flag before merging.

M-28: Fixing the legacy repo instead of extracting knowledge for the rebuild

Field	Detail
Legacy Problem	The temptation, when finding a bug like M-14's missing permission check, is to patch it directly in the legacy <code>routes/job_tickets.py</code> — which produces a slightly-less-broken legacy system instead of progress toward the rebuild, and risks masking exactly the evidence (like M-01's formula divergence) the rebuild needs to see clearly.
Evidence	Not a code-level finding — a process risk this task itself was explicitly commissioned to prevent (per this task's own instructions: "Do not recommend rebuilding or 'fixing' the legacy pricing system in place").
Damage Caused	Time spent hardening code that is being replaced; legacy patches can also change the very evidence (formula outputs, snapshot shapes) future golden-formula parity tests need to compare against.
Rebuild Prevention Rule	The Legacy Repo Usage Rule (stated in full in <code>PRICING_LEGACY_REBUILD_HANDOFF_MASTER.md</code>): the legacy repository is a source of formulas, business rules, workflows, examples, configuration data, and verification cases — not the place the final architecture gets built.

Enforced In	Every phase, as a standing project rule
Verification Test	Process check, not a code test: no pull request/commit in the rebuild should be a same-shape bug fix applied directly to legacy calculator/Wrap Lab/Webstore pricing files, outside of the previously-approved, explicitly-scoped security patches already tracked separately in RUNNING_ISSUE_TRACKER.md .

PRICING TO REBUILD MIGRATION MAP

Status: Documentation/investigation only. No migrations created, no data moved. This is the legacy data extraction plan — what data exists, whether it should come forward into the rebuild, what transformation it needs, where it lands, and whether history must be preserved.

Legacy Source	Data Found	Bring to Rebuild?	Transformation
Pricing configuration (<code>pricing_configuration</code> collection)	Per-tenant document: labor rates, markup rules, overhead %, category defaults, embedded materials array	Yes	Split into <code>PricingFoundation</code> + <code>CategoryPricing</code> category (Phase 0 converted to int)
Category defaults (<code>category_defaults.*</code>)	Nested dict-of-dicts, one sub-document per category (banners, rigid_signs, cut_vinyl, digital_print, vehicle_wraps, apparel, services, promotional, custom)	Yes	Restructure into <code>CategoryPricing</code> document per category with a consistent schema for all 9 (today's schema inconsistent per PRC-004)
Materials (embedded array inside <code>pricing_configuration</code>)	Material name, <code>cost_per_sqft</code> , <code>category/compatible_categories</code> tags, vehicle base-sqft entries (<code>category="vehicle_type"</code>)	Yes	Extract into its own collection; add <code>last_verified_at</code> ; <code>compatible_categories</code>
Hardware/accessories (per-category add-on config, e.g., Banner Setup Wizard's "Add-on Defaults" step)	Add-on cost items	Yes	Needs More Investigation to confirm exact shape (Extraction item) before deciding transformation
Labor rates (<code>labor_rates.*</code> , both legacy hourly and modern minute-based)	Production/design/removal/install hourly rates; minute-based rate+include_labor toggle	Bring only the minute-based system (pending Owner re-affirmation of retiring the hourly fallback, M-08)	Consolidate onto minute-based system once approved
Markup rules (<code>markup_percent/markup_multiplier</code> per category)	Category-level markup value	Yes	Verify consistency with application order pass-through cost categories before (M-06)
Overhead rules (<code>overhead_percent</code> , shared <code>calculate_overhead_cost()</code>)	Tenant-level or category-level overhead %	Yes	Re-verify the cost for every category once passes in, correct Phase 0 Group 2 basis if it currently (M-07)
Pricing templates (<code>PricingTemplate</code> documents)	Saved reusable presets	Yes, as-is	None significant structure already prior investigation
Historical invoice imports (<code>pricing_imports</code> collection, uploaded past invoices/AI-extracted benchmarks)	Raw uploaded files, AI-extracted rows, human accept/reject decisions, resulting benchmark suggestions	Yes, as-is	None significant module's suggestion pattern is sound

Pricing snapshots — general Job Ticket (<code>job_tickets.pricing_snapshot</code>)	<code>{pricing_mode, calculated_price, manual_price, active_price, calculation_breakdown, saved_at}</code>	Yes, transform	Map onto the new <code>PricingSnapshot</code> structure. <code>active_price</code> → <code>final_price_cents</code> ; dollars → int cents; <code>calculation_breakdown</code> → <code>cost_breakdown</code> . Map quoted <code>final_price_cents</code> to the <code>total_cost_cents</code> should be <code>estimated_total_labor_cost</code> + <code>cost_breakdown</code> by the size, a minimum of <code>manual_quoted_price</code> or "other" reason. Synthesize <code>PricingOverride</code> the rebuild. Where a Quote/Order item can be matched to <code>job_tickets.pricing_snapshot</code> , link it to the new <code>PricingSnapshot</code> . Cannot (older, pre-job_tickets record) leave the legacy table untouched.
Pricing snapshots — Wrap Lab (<code>compute_pricing_snapshot()</code> output)	<code>{pricing_method, total_billable_sqft, total_labor_cost, material_total, base_cost, markup_percent, markup_amount, suggested_price, per_sqft_price, manual_quoted_price, quoted_price, estimated_profit, estimated_margin_percent, computed_at}</code>	Yes, transform	Map onto the new <code>PricingSnapshot</code> structure. <code>manual_quoted_price</code> → <code>manual_price</code> ; <code>quoted_price</code> → <code>final_price_cents</code> ; <code>estimated_profit</code> → <code>estimated_profit_cents</code> ; <code>estimated_margin_percent</code> → <code>estimated_margin_percent</code> ; <code>computed_at</code> → <code>computed_at</code> .
Pricing-related Quote/Order Item fields (<code>QuoteLineItem</code> , embedded order line items)	<code>quantity, unit_price, line_total/total, description</code>	Yes, transform where a corresponding snapshot exists; otherwise leave as historical record only	Where a Quote/Order item can be matched to <code>job_tickets.pricing_snapshot</code> , link it to the new <code>PricingSnapshot</code> . Cannot (older, pre-job_tickets record) leave the legacy table untouched.
Vehicle tables (<code>materials</code> array entries with <code>category="vehicle_type", base_sqft</code>)	Vehicle-type → base sqft	Yes	From untouched <code>materials</code> array, fabricate a <code>vehicle_type</code> structure inside <code>CategoryPricingConfig</code> .
Vehicle package benchmarks (referenced in <code>calculate_vehicle_graphics</code> docstring, exact table not read)	Coverage-tier benchmark floor prices	Yes, pending confirmation of exact structure	Needs More Investigation. <code>vehicle_package_benchmarks</code> b
Wrap pricing config (<code>WrapPricingConfig</code> current values on live wrap tickets)	<code>pricing_method, price_per_sqft, hours, labor_rate, fees, markup_percent, manual_quoted_price</code>	No — do not bring the config structure forward. Only the already-committed snapshot output (see "Pricing snapshots — Wrap Lab" row above) is preserved as history; the live config structure itself is replaced by the shared <code>CategoryPricingConfig</code> for vehicle wraps.	N/A (retired)
Wrap materials (per-ticket <code>materials</code> array)	<code>material_name, brand, cost_per_sqft, sqft_used, total_material_cost</code>	Partially. The historical, already-used-in-a-committed-snapshot values are preserved as part of that snapshot's frozen <code>cost_breakdown</code> (see snapshot row above). The live reusable material definitions should be reviewed by the Owner and manually promoted into the shared <code>PricingMaterials</code> collection where they represent a real, reusable material (not auto-migrate, during Phase 8, if the concept is worth keeping, it becomes part of the ONE shared Apparel table, not a Webstore-only copy).	<code>pricing_materials</code> (selectively, Owner-reviewed)
Apparel price tables (Foundation's <code>calculate_apparel()</code> internal tables, exact structure not read)	Blank cost, brand/style, quantity tiers, placement, decoration method	Yes, pending confirmation of exact structure	Needs More Investigation
Apparel tier defaults (Webstores' <code>APPAREL_TIER_DEFAULTS</code>)	Economy/standard/premium price modifiers	No — do not bring forward as a second table. Reconcile with the Foundation's real <code>ApparelPricingTable</code> , may be one-off ticket-specific.	Reconciliation required; merge into shared <code>ApparelPricingTable</code> .
Services rates (Foundation's <code>calculate_services()</code> internal config, exact structure not read)	Hourly/flat/per-unit rates, travel, service-call minimum	Yes, pending confirmation of exact structure	Needs More Investigation
Promotional vendor-cost settings (Foundation's <code>calculate_promotional()</code> internal config)	Vendor cost + markup rule	Yes, pending confirmation of exact structure	Needs More Investigation

Webstore products (Product documents)	name, description, category (display), base_cost, retail_price, variants	Yes, but re-categorize, do not re-price automatically. Each product must be manually mapped by the Owner to a real pricing_category (Phase 8) — the existing base_cost/retail_price become the STARTING public override values not blindly trusted as "the" cost; the Owner opted to run each product through the real calculator once, post-migration	WebstoreProduct real pricing_category mapping + optional override
Webstore product prices at time of past sale	Whatever retail_price was at checkout time (exact snapshot mechanism unconfirmed — Needs More Investigation #7 in Extraction Inventory)	Yes, preserve whatever historical record exists; do not fabricate a PricingSnapshot retroactively with fields that were never actually captured	Investigate first; structured snapshot today, past orders keep their legacy as a read-only historical value — only New after the rebuild
Webstore fees/owner-share data (LockedSettings, profit_split, payment_platform_fee)	Admin-locked cost/profit/fee fields	Yes, as-is	PricingSnapshot admin-locked-field protection pattern and should be precise exactly
Webstore orders (webstore_orders-equivalent collection, exact name not confirmed this session)	Order records with profit-allocation calculations	Yes, as-is for historical orders; new orders get real PricingSnapshot per line item going forward	None for historical orders wired to PricingSnapshot
Customer discount/tax-exempt fields	None found — confirmed to have no such fields	N/A — nothing to migrate	N/A
Inventory material records (inventory_lots, inventory_items)	On-hand quantity, reservations, lot tracking	Yes, as-is — this system is not part of the pricing rebuild's data migration. Only its link to pricing_materials.cost_per_sqft (if any. Needs More Investigation) is pricing-rebuild scope.	None to the inventory system itself
Vendor price records (VendorInput and similar)	Vendor cost per material/item	Yes, as-is for the vendor/purchasing system itself. Only the potential feedback link into pricing_materials.cost_per_sqft is (non-pricing-specific financial records) nothing as-is only the pricing-derived portion of reporting moves to reading PricingSnapshot	None to the vendor itself
Profitability-report source data (jobs, job_items, sales_entries, expense_entries, invoices as currently read by ProfitAnalytics.py)	Legacy job/job-item records used for margin/profit calculations today	No — do not migrate the legacy jobs/job_items/reporting source forward as the new report's data source. sales_entries/expense_entries/invoices (non-pricing-specific financial records) nothing as-is only the pricing-derived portion of reporting moves to reading PricingSnapshot	PricingSnapshot reporting (Phase pricing-derived sales_entries/expense_entries)
ProfitingAnalytics.py activity/audit logs	None found prior to Sprint 1	N/A	N/A for the rest
Pricing quiz answers and derived settings (PricingSetupQuiz.js/ShopRateQuiz.js/BannerSetupWizard.js session state)	In-browser session state only — confirmed no persistence of partial quiz progress found this session (Assumption to Verify: confirm whether ANY quiz answer history is stored server-side at all, versus purely applied-to-settings-then-discarded)	Likely nothing to migrate — Needs More Investigation to confirm whether any quiz-answer history table exists server-side before concluding there is truly nothing to bring forward	If found: a new PricingQuizResponse history table (Phase confirmed absence designs answer-storage fresh

Migration principles that apply across every row above

1. **Never rewrite a price that already happened.** A committed PricingSnapshot (or its legacy equivalent) for a job that already shipped is a historical fact, not a live calculation — migration transforms its *shape* (float dollars → cents, field renames) but never its *value*.
2. **Config migrates; unexplained history is not fabricated.** Where the legacy system never captured a "why" (e.g., a manual override with no reason field), the migration synthesizes a clearly-labeled placeholder (reason: "other", a note that provenance predates the rebuild) rather than inventing a false explanation.
3. **Curated over automatic where trust is uncertain.** Webstore product re-categorization and Wrap Lab material promotion are explicitly Owner-reviewed, not blind automatic migrations, because the legacy data's accuracy is

already in question (that's WHY it's being rebuilt).

4. **"Needs More Investigation" rows are not blocking Phase 2's start** — they are read-before-you-build items for whichever build-order phase actually touches that specific category (Apparel/Services/Promotional details block Phase 5, not Phase 2).

PRICING MODULE EXTRACTION INVENTORY

Status: Documentation/investigation only. No code changed. This inventory covers every pricing-related module, page, route, service, model, calculator, wizard, and major component found in the legacy repository during this pass, plus prior investigation sessions (`PRICING_SYSTEM_CALCULATORS_ONBOARDING_QUIZ_REBUILD_DOC.md`, `PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md`).

Legend for "Confidence": `Confirmed` = read the actual code this session or a prior session and verified the claim directly. `Assumption to Verify` = plausible based on naming/structure but not read line-by-line — exact file/route to inspect is listed. `Open Business Decision` = a choice for the Owner, not a code fact.

Dispositions used below: Bring Over Mostly As-Is | Bring Over Formula/Business Rules Only | Refactor Before Rebuild | Merge Into Shared System | Replace Completely | Use Only as Reference | Retire / Do Not Port | Needs More Investigation

A. CORE PRICING FOUNDATION

Item	Current Purpose	Files/Routes/Models	Share
Pricing calculation dispatcher	Routes a <code>(category, JobItemPricingData, quantity)</code> call to one of 9 <code>calculate_*</code> functions	<code>server.py::calculate_pricing()</code> (line 4356), 9 <code>calculate_*</code> functions (<code>calculate_banners</code> L2126, <code>calculate_rigid_signs</code> L1698, <code>calculate_cut_vinyl</code> L867, <code>calculate_digital_print</code> L1214, <code>calculate_vehicle_graphics</code> L2683, <code>calculate_apparel</code> L3869, <code>calculate_services</code> L3237, <code>calculate_promotional</code> L696, <code>calculate_custom</code> L3739)	mo Job Pric coll
<code>/api/pricing/calculate</code> endpoint	Live calculation entry point, no persistence	<code>routes/pricing.py</code> L162-186 (<code>calculate_price</code>)	calo
Pricing Foundation settings CRUD	Get/update tenant's <code>PricingFoundationSettings</code> -equivalent defaults	<code>routes/pricing.py</code> L188-273 (<code>get_my_pricing_defaults</code> , <code>get_my_pricing_settings</code> , <code>update_pricing_defaults</code> , <code>update_pricing_settings</code>)	pric
Materials Library (embedded)	Per-tenant list of materials with cost/sqft, category tags	<code>models/pricing.py::MaterialConfig</code> ; embedded array inside <code>pricing_configuration</code> ; <code>routes/pricing.py::get_materials</code> L274-293; helper <code>find_material()</code> /	Eve calo veh
Labor rates	Production/design/removal/install hourly rates, plus a newer per-category minute-based system	<code>models/pricing.py</code> (<code>LaborRateRule</code> ; <code>get_my_pricing_defaults</code> similar); <code>server.py::get_labor_minutes_and_rate()</code> (referenced L2810)	Eve
Markup rules	Category-level markup multiplier/percent applied to cost basis	Embedded in each <code>calculate_*</code> function + <code>cfg.get("markup_percent"/"markup_multiplier")</code> per category config	All

Overhead rules	Overhead % applied to a cost basis	<code>server.py::calculate_overhead_cost()</code> (L336)	All (As calco + p already ince
Minimum charges	Category minimum billable price/area floor	Per-category <code>cfg.get("install_minimum"...)</code> , <code>min_sell</code> fields	All
Rush pricing	Rush surcharge applied to suggested price	Per-category rush fields (<code>rush_fee</code> , similar)	All
Quantity discounts	Price break at quantity tiers	Referenced in Remediation Plan investigation (<code>PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md</code>)	Cal
Hardware/accessory pricing	Add-on cost items (grommets, stakes, mounting hardware)	Referenced via <code>BannerSetupWizard.js</code> "Add-on Defaults" step (L283-383) and Rigid Signs "hardware" config	Rig
Category pricing defaults	Per-category config sub-document (<code>category_defaults.<category></code>)	<code>pricing_configuration.category_defaults.*</code> , written to by <code>PricingSetupQuiz.js</code> , <code>BannerSetupWizard.js</code> , and category-specific setup screens (<code>CategoryPricingMethodSetup.js</code>)	All get
<code>PricingTemplate</code>	Saved reusable pricing preset (e.g., "Standard 4x8 banner")	<code>routes/pricing.py</code> L294-360 (list/create/update/delete/favorite); model in <code>models/pricing.py</code>	Qu to V
Pricing Setup Quiz	11-section guided onboarding quiz converting real-world prices into Foundation defaults	<code>frontend/src/components/pricing/PricingSetupQuiz.js</code> (1,375 lines)	Wr des
Banner Setup Wizard	7-step category-specific wizard for Banners pricing setup	<code>frontend/src/components/pricing/BannerSetupWizard.js</code> (501 lines), launched from <code>CategoryPricingMethodSetup.js</code>	Wr exa Qu
Shop Rate Quiz	Standalone shop/labor-rate-derivation calculator (home/small/growing shop presets, billable %, payroll burden %, profit buffer, rounding rule)	<code>frontend/src/components/pricing/ShopRateQuiz.js</code> (534 lines), used in <code>pages/PricingFoundation.js</code>	Fee sett sess

Historical Invoice Import	AI-assisted extraction of benchmark prices from uploaded past invoices/PDFs, with column-mapping, outlier detection, and a human-review acceptance step	routes/pricing_setup.py (818 lines) — imports, mapping, analyze, review endpoints; uses ai_analyze_category_metrics/ai_extract_pdf_rows	Wr pric acc pat flag Gui
Pricing snapshots (general Job Ticket)	Frozen price record attached to a Job Ticket (the platform's current generic "Order Item")	routes/job_tickets.py L1791-1826 (save_ticket_pricing) writes pricing_snapshot = {pricing_mode, calculated_price, manual_price, active_price, calculation_breakdown, saved_at} onto job_tickets	rou /ge t.ge (L2
Manual price overrides (general)	User can type a manual price instead of using the calculated one	routes/job_tickets.py::save_ticket_pricing, pricing_mode: "calculator" \ "manual"	San
Pricing warnings/guardrails	Generic warnings: [string] array on calculator output	models/pricing.py::PricingCalculation.warnings (Assumption to Verify exact field name — inferred from PricingCalculation model referenced across the dispatcher)	Eve
Pricing permissions	RBAC gating on who can view/edit pricing config or override	Historically: none dedicated (confirmed absent before this sprint)	—
Pricing audit/change history	Log of who changed a material cost, labor rate, markup, etc. and when	Historically: none found	—

B. CALCULATOR CATEGORIES

Item	Current Purpose	Files/Routes/Models	Shared Dependencies	Current Problems
Banners	Area-based calculator: hems, grommets, pole pockets, sewing, finishing	server.py::calculate_banners() L2126-2683	Area-based engine (shared with Rigid Signs/Cut Vinyl/Digital Print per Remediation Plan)	All-float math; giant function; see Foundation-level issue above
Rigid Signs	Area-based: substrates, thickness, shape, hardware, drilling	server.py::calculate_rigid_signs() L1698-2126	Same area-based family	Same as Banners
Cut Vinyl	Area-based: colors, weeding, transfer tape, masking	server.py::calculate_cut_vinyl() L867-1214	Same area-based family	Same as Banners
Digital Print	Area-based: ink coverage, laminate, contour cutting, mounting	server.py::calculate_digital_print() L1214-1698	Same area-based family	Same as Banners

Vehicle Graphics / Vehicle Wraps (Foundation engine)	Area-based extension: vehicle type, coverage %, package benchmark floor, waste-by-coverage, laminate, window perf, seam/difficulty multipliers, second installer	<code>server.py::calculate_vehicle_graphics()</code> L2683-3237	Materials library (vehicle base-sqft lookup L2732-2736), minute-based labor system	Genuinely sophisticated, mostly sound formula, fully disconnected from Wrap Lab's separate, much simpler engine . Wrap Lab section below
Apparel	Table-first: blank cost, brand/style, quantity tiers, placement, decoration method, size breakdown, plus-size, add-ons, formula fallback	<code>server.py::calculate_apparel()</code> L3869-4356	Apparel price tables (referenced, structure not read line-by-line this session)	Not verified line-by-line this session beyond function existing
Services	Time/unit/trip-based: hourly, flat, per-unit, travel, mileage, service-call minimum, equipment, subcontracting, permit fees, overtime	<code>server.py::calculate_services()</code> L3237-3739	None area-based; standalone module	Not verified line-by-line this session beyond function existing
Promotional	Vendor-cost/cost-plus-markup pricing	<code>server.py::calculate_promotional()</code> L696-867	None	Not verified line-by-line this session beyond function existing
Custom	Manual-entry fallback for uncategorizable items	<code>server.py::calculate_custom()</code> L3739-3869	None	Risk of becoming an unhatched that bypasses snapshot/warning/performance discipline if not deliberately wired the same as every other category

C. CONNECTED OPERATIONAL AREAS

Item	Current Purpose	Files/Routes/Models	Shared Dependencies	Comments
Quotes	Customer-facing price proposal, own <code>line_items/subtotal</code> shape	<code>routes/quotes.py</code> (343 lines) — <code>QuoteLineItem</code> , subtotal calc L57-186	Independent of <code>job_tickets.pricing_snapshot</code> — quotes appear to carry their own embedded line-item price, not a reference to a <code>PricingSnapshot</code> id	Also appears in job tickets
Orders / Order Items (current, called "job tickets")	The real production/pricing unit today	<code>routes/orders.py</code> , <code>routes/job_tickets.py</code> , <code>db.job_tickets</code> collection	<code>pricing_snapshot.active_price</code> , <code>db.material_requirements</code> (inventory), <code>db.orders</code> (FROM a job tickets snapshot, vs. always being manually entered)	Tracked in "Orders" items or different
Work Order Summary	Aggregated production ticket for a job	<code>routes/orders.py::generate_work_order()</code> L424-484	<code>db.job_tickets</code>	Not explicitly
Production time estimates	Estimated labor time per item, feeds Work Order Summary and (should) profitability reporting	Embedded on <code>job_tickets</code> items, referenced by <code>get_order_production_summary()</code> L523-567	<code>job_tickets</code>	Not explicitly

Inventory material usage	Reserve/pull materials against a job's material requirements	<code>services/inventory_service.py</code> (<code>reserve_requirement</code> , <code>reserve_order</code> , <code>allocatable_quantity</code>) — reads <code>material_requirements</code> collection keyed by <code>job_ticket_id/order_id</code>	<code>job_tickets</code> , <code>inventory_lots</code>	G sy lo m
Purchasing and vendor costs	Vendor price records feeding material cost	<code>models/inventory.py::VendorInput</code> , <code>PurchaseOrderInput</code> (per handoff's existing model references)	<code>pricing_materials</code> (future), <code>inventory_lots</code>	N
Invoices and deposits	Billing document generated from a Quote or Order	<code>routes/invoices.py</code> (488 lines) — imports <code>sync_job_items_from_embedded_line_items</code> from <code>routes/jobs.py</code>	<code>jobs.line_items</code> (fallback), <code>quotes</code>	C in -e
Profitability / margin reporting	Owner-facing margin/profit dashboards	<code>routes/profit_analytics.py</code> (587 lines) — reads <code>db.jobs.find()</code> (L167) and <code>db.job_items.find()</code> (L175), not	<code>jobs</code> , <code>job_items</code> , <code>customers</code> , <code>sales_entries</code> , <code>expense_entries</code> , <code>invoices</code>	C E E m e le
Tax interaction	Sales tax calculation on quotes/invoices/orders	<code>db.job_tickets</code> ; this session (no dedicated tax module found in the files inspected)	—	C pr it
Customer-specific pricing, discounts, wholesale, contract pricing	Per-customer price adjustments	<code>models/customer.py::Customer</code> class inspected directly — no discount, wholesale, contract, or tax-exempt fields exist on the Customer model	—	

D. WRAP LAB / WRAP COMMAND CENTER

Item	Current Purpose	Files/Routes/Models	Shared Dependencies
Wrap project pricing engine	Independent price calculation for a wrap job: <code>material_labor_markup</code> , <code>per_sqft</code> , or <code>manual</code> method	<code>routes/wrap/core.py::WrapPricingConfig</code> (L156-172), <code>_compute_pricing_snapshot()</code> (L665-746)	None — this is the core pricing engine; it does not call <code>server.py::calculate_pricing</code> or share any Foundation dependencies
Wrap-specific materials array	Per-ticket materials list (<code>material_name</code> , <code>brand</code> , <code>cost_per_sqft</code> , <code>sqft_used</code>)	<code>routes/wrap/core.py::WrapMaterialUpdate</code> (L142-153), <code>WRAP_MATERIAL_TYPES</code> (L175-179), stored inline on the <code>job_tickets</code> doc's <code>materials</code> array	None — a second, independent materials list, disconnected from the Foundation's Phase 2 <code>pricing_materials</code> entirely
Vehicle tables	Vehicle type → base sqft lookup	Foundation side: embedded in the shared materials library, <code>category="vehicle_type"</code> (L2732-2736 of <code>server.py</code>); Wrap Lab side: none found — Wrap Lab does not appear to look up vehicle base sqft at all, relying entirely on manual coverage/sqft entry	Foundation's materials library

Coverage packages / package benchmark floor	"Full wrap starts at \$X" benchmark price used as a floor against cost-plus pricing	Foundation side: referenced in <code>calculate_vehicle_graphics</code> docstring step 7 ("Suggested price = max(package benchmark scaled by complexity, cost-plus)"), exact benchmark table not read this session; Wrap Lab side: no benchmark concept found — Wrap Lab has no floor/ceiling sanity check at all	Foundation calculator
Install-time tables	Vehicle-type-specific expected install hours	Referenced in Foundation's <code>calculate_vehicle_graphics</code> labor section (L2799-2820, minute-based system); Wrap Lab side uses a flat manually-entered <code>install_hours</code> field with no table lookup (<code>WrapPricingConfig.install_hours</code> , L161)	Foundation minute-b
Wrap quote approval	Customer approval step for a wrap price/design	<code>routes/wrap/core.py</code> — <code>DesignBlock.proof_status</code> (L196), contract/signature workflow (per handoff scope item D)	<code>job_tickets</code> (wrap-fla
Wrap-specific snapshots	<code>_compute_pricing_snapshot()</code> output stored on the ticket	<code>routes/wrap/core.py</code> L1254-1289 (<code>PUT .../pricing</code> recomputes AND persists together — no separate live-preview-vs-commit step, unlike the general Job Ticket's <code>/calculate-pricing</code> + <code>/save-pricing</code> two-step pattern)	—
Wrap documents/packets consuming pricing	Customer packet PDF, work order PDF that display the wrap price	<code>routes/wrap/pdfs.py</code> (not read this session)	Wrap snapshot
Wrap inspection, measurements, design workflow, install workflow, aftercare, issue tracking	Non-pricing wrap-specific workflow	<code>routes/wrap/core.py</code> , <code>routes/wrap/portal.py</code> , <code>frontend/src/components/wrap/*</code> , <code>frontend/src/pages/WrapCommandCenterPage.js</code>	None pricing-related

E. WEBSTORES

Item	Current Purpose	Files/Routes/Models	Shared Dependencies	Current Problem
Webstore product pricing	Flat <code>base_cost</code> / <code>retail_price</code> entered manually per product	<code>routes/webstores.py::Product</code> (L173-194), <code>ProductCreate</code> / <code>ProductUpdate</code> (L197-225)	None — no link to <code>pricing_configuration</code> , no calculator category, no <code>PricingResult</code>	Confirmed: <code>base_cost</code> / <code>retail_price</code> are manually-typed zero connection. Pricing Foundation calculator, material rate, or markup is whatever <code>retail</code>

				base_cost happens to be entered by hand
Product-category mapping	ProductCategory enum used for storefront display grouping	routes/webstores.py::ProductCategory (L80-86): apparel, signs, decals, promotional, events, other	Product model	Confirmed: the NOT map 1:1 of Pricing Foundation calculator categories "signs" and "decals" collapse what signs and Banners/Rigid Signs/Vinyl/Digital Products are in a bucket, and it has vehicle, wraps, custom product, and is a manually-typed price not derived from calculator. Same as above provenance, not internal cost/metadata snapshot
Product variants	Size/color/tier options with a flat additional_cost	routes/webstores.py::ProductVariant (L162-170)	Product	
Public retail prices	Customer-facing price shown on the storefront	Product.retail_price	—	
Fundraiser pricing	Event/fundraiser store profit-allocation rules	routes/webstores.py L321-330, L368-381, compute_event_profit_allocation() (L1034-1073)	Product/retail_price, total_profit	The profit-allocation (percentage/fix modes, running tracking) is source independent of pricing-engine operates on what total profit is how that profit calculated
Event-store pricing	Same Webstore product system, used for one-off event stores	Same Product/ProductCategory model	Same	
Owner share	Store owner's share of fundraiser profit	LockedSettings (L106-116, referenced fields), profit_split (L118)	Product, fundraiser profit allocation	Tenant-admin- explicitly documented code as "MUST" overwritten by answers... or a non-admin flow (L109-111) — protection pattern well-designed
Platform fee	Emergent/platform's cut of a Webstore transaction	webstores.py L577 (payment_platform_fee, referenced on the order/checkout side)	Stripe checkout	Not read line-by-session beyond existence
Discounts/promotions	Storefront-level discount codes	Not located by name in the files inspected this session	—	Unknown

Webstore order pricing snapshots	Frozen price record for a Webstore order line	Not confirmed to exist as a structured object this session — order creation appears to just copy <code>Product.retail_price</code> at checkout time (Assumption to Verify: read the actual checkout/order-creation function in <code>routes/webstores.py</code> , which was not read this session)	<code>Product</code>	If no structured exists, this is a "pricing at com mechanism (or at all)
Webstore profitability reporting	Store-level profit/margin dashboards	Likely reads <code>total_profit</code> / <code>Product.base_cost</code> fields directly	Same disconnected cost data as general Webstore pricing	Inherits the same — any margin only as good as manually-typed
Stripe/payment relationship	Payout, fees, and reporting tied to Webstore checkout	Referenced (<code>payment_platform_fee</code>); the actual Stripe integration itself is out of scope for the pricing rebuild per this task's own instructions ("only where it affects price, fees, payout, or reporting")	Webstore checkout	Not investigated fee field

Items explicitly flagged "Needs More Investigation" (consolidated, for the next session)

- Whether `routes/pricing.py`'s `get_my_pricing_defaults` vs `get_my_pricing_settings` are truly two different documents or two views of one — read both handlers side-by-side.
- Quantity-discount, hardware/accessory, minimum-charge, and rush-fee exact field shapes per category — read each `calculate_*` function in full (`server.py` L696-4356).
- Apparel, Services, and Promotional calculators' internal logic — only their existence and line ranges were confirmed this session, not their formulas.
- Whether a PO receipt in Purchasing ever updates a material's `cost_per_sqft` (the vendor-cost-to-material-cost feedback loop) — read `routes/inventory.py`'s PO-receive endpoint.
- Tax calculation — not located; search `routes/quotes.py`, `routes/invoices.py`, `routes/orders.py`.
- Whether Wrap Lab has any vehicle-table/benchmark lookup at all, or is 100% manual entry — read the rest of `routes/wrap/core.py` (only L140-750 and L1174-1330 were read; file is 2,126 lines).
- Webstore checkout/order-creation function and whether any Webstore order pricing snapshot exists — `routes/webstores.py` is 3,776 lines; only L1-260 and L1030-1080 were read this session.
- Discount codes/promotions on Webstores — not located by name.
- Whether a Quote Item can ever be created directly from a `job_tickets.pricing_snapshot`, or is always manually re-entered — read `routes/quotes.py` creation flow against `routes/orders.py::generate_quote_from_order`.
- Exact `PricingCalculation.warnings` field shape in `models/pricing.py` (L1302) — re-confirm during Phase 3.

PRICING LEGACY REBUILD HANDOFF MASTER

Role: Pricing, Calculator, Wrap Lab, and Webstore Rebuild Handoff Architect deliverable.

Status: Documentation, investigation, and extraction only. No calculators rebuilt, no Pricing Foundation refactored, no live pricing behavior modified, no migrations created, no screens changed, no pricing code moved. This document is the top-level entry point into the handoff package; the other 7 documents it references contain the full supporting detail.

Companion documents:

- `PRICING_MODULE_EXTRACTION_INVENTORY.md` — every pricing-related module/route/model, current purpose, problems, rebuild disposition
- `PRICING_TO_REBUILD_MIGRATION_MAP.md` — legacy data extraction plan
- `PRICING_LEGACY_ARCHITECTURE_MISTAKES_AND_PREVENTION.md` — 28 documented mistakes with evidence and prevention rules
- `PRICING_REBUILD_BUILD_ORDER.md` — the 10-phase build order (Phase 1 already complete)
- `PRICING_SHARED_SYSTEMS_AND_DATA_OWNERSHIP_MAP.md` — one owner per entity, live-vs-snapshot rule per entity
- `PRICING_FEATURES_DO_NOT_PORT.md` — exact files/patterns that must not be copied forward
- `PRICING_REBUILD_VERIFICATION_AND_PARITY_TEST_PLAN.md` — what must match legacy, what's intentionally different, required tests

Prior-session foundation this package builds on (still valid, not superseded):

- `PHASE_0_PRICING_DECISIONS_FORMULA_GOVERNANCE.md` — locked business-rule decisions
- `PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md` — 59-issue inventory, phased plan, \$4E live-calc/snapshot architecture statement
- `BANNERS_PORT_WIRING_SPEC.md` — Phase 3's first-port wiring plan
- Sprint 1 (2026-02-15) — `Money`, `PricingResult`/`PricingSnapshot`/`PricingWarning`/`PricingOverride`/`PricingChangeLog` schemas, `PRICING_VIEW`/`PRICING_EDIT_FOUNDATION`/`PRICING_OVERRIDE` permissions — all already built and unit-tested (58 passing tests), zero live behavior changed

1. Executive summary

The legacy repository's pricing system is one shared calculation dispatcher (`server.py`, ~3,700 lines across 9 `calculate_*` functions) plus at least 3 independent parallel systems that were each built to solve a pricing problem on their own instead of consuming the shared dispatcher: Wrap Lab (`routes/wrap/core.py`, a fully separate engine, materials list, and snapshot shape — **confirmed** this session), Webstores (`routes/webstores.py`, manually-typed flat `base_cost`/`retail_price` with zero calculator linkage — **confirmed** this session), and a legacy `jobs/job_items` reporting path that profitability reporting still reads from instead of the modern `job_tickets` collection (**confirmed** this session, `routes/profit_analytics.py` L167-175). On top of this, at least 4 divergent "pricing snapshot" shapes exist across the codebase, all-float currency math is universal, and manual price overrides have zero provenance or permission gating anywhere (**confirmed**, `routes/job_tickets.py::save_ticket_pricing`).

None of this is fixed in place. This package extracts the business rules, data, and workflow concepts worth keeping, and

hands off a build plan for one shared Pricing Foundation, one shared live-calculation engine, and category-specific formula configuration — consumed identically by every calculator, Wrap Lab, and Webstores, with a frozen `PricingSnapshot` created only at explicit commit.

2. Calculator family analysis

2a. Shared Area-Based Calculator Family

Members: Banners, Rigid Signs, Cut Vinyl, Digital Print, Vehicle Graphics/Vehicle Wraps.

Shared logic every member of this family uses (confirmed present across `calculate_banners`/`calculate_rigid_signs`/`calculate_cut_vinyl`/`calculate_digital_print`/`calculate_vehicle_graphics` in `server.py`):

- Dimensions → billable area
- Minimum billable area (floor on area before cost calculation)
- Waste % applied to area before material cost
- Material cost (area × cost-per-sqft, from the shared Materials Library)
- Production labor (minute-based, with a legacy hourly fallback still present per M-08)
- Markup (category-level %, applied to a cost basis)
- Minimum charge (floor on final price)
- Quantity discounts (Needs More Investigation for exact mechanism, Extraction Inventory item #2)
- Rush fees
- Manual override behavior (today: no provenance, gets replaced by `PricingOverride`)
- Live recalculation (the calculator's live-preview UX, generalized from the pattern already proven in `routes/job_tickets.py`'s `/calculate-pricing` endpoint)
- Warning generation (today: generic strings, gets replaced by typed `PricingWarning`)
- Snapshot generation (today: `job_tickets.pricing_snapshot`, gets replaced by standardized `PricingSnapshot`)

Category-specific extensions:

- **Banners:** hems, grommets, pole pockets, sewing, finishing
- **Rigid Signs:** substrates, thickness, shape, hardware, drilling
- **Cut Vinyl:** colors, weeding, transfer tape, masking
- **Digital Print:** ink coverage, laminate, contour cutting, mounting
- **Vehicle Graphics/Wraps:** vehicle type, coverage % (spot/partial/half/full/custom, **confirmed**, `server.py` L2706-2728), package benchmark floor, vehicle install-time tables, seam complexity, prep, removal, window perf (**confirmed**, L2773-2797), second installer

2b. Vehicle Graphics / Wrap Lab Rule

Confirmed this session: Wrap Lab's current pricing engine (`WrapPricingConfig/_compute_pricing_snapshot()`, `routes/wrap/core.py` L156-172, L665-746) is a completely independent, materially simpler formula than the Foundation's `calculate_vehicle_graphics()` (`server.py` L2683-3237) — no coverage %, no vehicle-type differentiation, no waste %, no materials-library lookup, no overhead, no benchmark floor, and it computes-and-persists its snapshot in a single step with no live/commit separation.

Wrap Lab MAY add (explicitly in scope, per this task's own final rebuild rules — confirmed these are genuinely pricing-unrelated workflow features already present and correctly scoped):

- Vehicle inspection, vehicle measurements
- Design workflow (`DesignBlock`, confirmed present, `routes/wrap/core.py` L182-200)
- Proof workflow
- Contract/signature workflow
- Installation workflow
- Customer packet (`routes/wrap/pdfs.py`, not read this session)
- Aftercare, issue tracking

Wrap Lab MAY NOT create its own:

- Formula engine — retire `_compute_pricing_snapshot()`
- Materials Library — retire the per-ticket `materials` array (`WrapMaterialUpdate`)
- Labor rates — retire `WrapPricingConfig.labor_rate`/hours fields as a config source; consume the shared minute-based system
- Markup rules — retire `WrapPricingConfig.markup_percent`
- Overhead rules — Wrap Lab has none today; it must gain the shared one, not invent its own
- Pricing methods — retire the `per_sqft/material_labor_markup/manual` method selector as an independent concept; the shared engine's method selection (if any) applies uniformly
- Pricing snapshots — retire the 4-field-divergent shape; use the standard `PricingSnapshot`
- Manual override rules — retire `manual_quoted_price`'s no-provenance override; use `PricingOverride`

2c. Apparel Calculator Family

Separate table-first module using: blank cost, brand/style, quantity tiers, placement, decoration method, size breakdown, plus-size rules, add-ons, formula fallback, live price updates, standardized snapshot output. Legacy source:

`server.py::calculate_apparel()` (L3869-4356) — internal formula structure confirmed to exist, not yet read line-by-line (Needs More Investigation). A parallel, simpler `APPAREL_TIER_DEFAULTS` concept already exists inside Webstores (`routes/webstores.py` L95-99) — this is a confirmed duplicate-in-waiting that Phase 8 must reconcile into ONE shared Apparel table, not preserve as two.

2d. Services Calculator Family

Separate time/unit/trip-based module using: hourly rates, flat rates, per-unit rates, travel, mileage, service-call minimum, equipment, subcontracting, permit/pass-through fees, overtime/after-hours rules, standardized snapshot output. Legacy source: `server.py::calculate_services()` (L3237-3739) — confirmed to exist, internal formula not read line-by-line this session (Needs More Investigation).

2e. Promotional and Custom

- **Promotional:** vendor-cost/cost-plus pricing. Legacy source: `server.py::calculate_promotional()` (L696-867), confirmed to exist, formula not read line-by-line (Needs More Investigation).
- **Custom:** controlled manual-pricing fallback. Legacy source: `server.py::calculate_custom()` (L3739-3869), confirmed to exist. **Both must create standardized snapshots. Custom must not become an excuse for a**

second disconnected pricing system — it still goes through the same `PricingResult`/`PricingSnapshot`/`PricingWarning`/permission gate as every calculated category; the only difference is the price itself is typed in rather than derived from a formula.

2f. Webstore Pricing — final relationship

Pricing Foundation

|

v

Shared Pricing Engine

|

v

Category Calculator Family (Banners/Rigid Signs/Cut Vinyl/Digital Print/Vehicle Wraps/Apparel/Services/Promotional/Custom)

|

v

Live internal product calculation (runs continuously as the store admin edits product setup, per the shared live-calc rule)

|

v

Optional Webstore public retail override (per the price-priority chain, M-26 – never erases internal cost/margin/provenance/owner-share/platform-fee/snapshot)

|

v

Webstore order-line Pricing Snapshot at checkout

Confirmed today: none of this relationship exists. `Product.retail_price/base_cost` (`routes/webstores.py` L173-194) are manually-typed floats with zero connection to the Pricing Foundation, any calculator, or any category. `ProductCategory`'s enum (`apparel, signs, decals, promotional, events, other`) does not map 1:1 onto the 9 real calculator categories. Phase 8 builds this relationship from scratch.

3. Cross-references

- Full module-by-module inventory with dispositions: `PRICING_MODULE_EXTRACTION_INVENTORY.md`
 - Data ownership (who owns what, live-vs-snapshot per entity): `PRICING_SHARED_SYSTEMS_AND_DATA_OWNERSHIP_MAP.md`
 - All 28 architecture mistakes with evidence and prevention rules: `PRICING_LEGACY_ARCHITECTURE_MISTAKES_AND_PREVENTION.md`
 - Exact do-not-port list: `PRICING_FEATURES_DO_NOT_PORT.md`
 - 10-phase build order: `PRICING_REBUILD_BUILD_ORDER.md`
 - Legacy data extraction table: `PRICING_TO_REBUILD_MIGRATION_MAP.md`
 - Verification/parity plan: `PRICING_REBUILD_VERIFICATION_AND_PARITY_TEST_PLAN.md`
-

4. Top 20 pricing-related concepts, formulas, workflows, and data structures worth preserving

1. The area-based formula family's core shape (dimensions → billable area → waste → material cost → labor → overhead → markup → minimum charge → rush) — sound business logic, confirmed across 4 categories in `server.py`.
2. The Vehicle Graphics coverage-tier factor system (spot/partial/half/full/custom, with linear interpolation for custom %) — a genuinely well-designed piece of business logic (`server.py` L2706-2728).
3. The package-benchmark-floor concept for Vehicle Wraps (`suggested price = max(benchmark, cost-plus)`) — a real-world sanity guardrail worth preserving as a formula pattern.
4. Window perf scope-based sell-rate logic (rear/side/full weighted average, `server.py` L2788-2797) — specific, correct business detail.
5. The minute-based labor costing system (the modern half of the dual system in M-08) — this is the correct direction, not the legacy hourly fallback.
6. `calculate_overhead_cost()`'s existence as ALREADY a shared function (`server.py` L336) — proof the codebase already knows how to share logic; the rebuild extends this instinct, not invents it.
7. The two-step live-preview-then-explicit-commit workflow already proven in `routes/job_tickets.py` (`/calculate-pricing` then `/save-pricing`) — this is the exact pattern Sprint 1's live-calc/snapshot correction generalizes to every category and surface.
8. The Historical Invoice Import's AI-suggest/human-accept pattern (`routes/pricing_setup.py`) — architecturally sound, becomes the template for Phase 10's AI Pricing Guidance.
9. `services/inventory_service.py`'s reservation/allocation/lot-tracking logic — a positive counter-example, confirmed well-built, needs no rebuild.
10. The `LockedSettings` admin-locked-field pattern in Webstores (fields "MUST NOT be overwritten by questionnaire answers... or any other non-admin flow," `routes/webstores.py` L106-118) — a good protection pattern worth explicitly preserving.
11. `compute_event_profit_allocation()`'s percentage/fixed-per-item allocation math (`routes/webstores.py` L1034-1073) — confirmed sound, independent of the pricing-engine problem.
12. `PricingTemplate`'s CRUD structure (`routes/pricing.py` L294-360) — already sound, reusable presets worth keeping.
13. The Shop Rate Quiz's shop-rate-burden-derivation methodology (home/small/growing presets, billable-hours %, payroll burden %, profit buffer, rounding rule — `ShopRateQuiz.js`) — confirmed this session to be a genuinely distinct, valuable business-rule reference, NOT redundant with the general Quiz.
14. The Pricing Setup Quiz's benchmark-price-to-Foundation-defaults derivation formulas (11 sections, `PricingSetupQuiz.js`) — the underlying math converting "what do you charge for a 4x8 banner" into `sell_rate/min_sell` is a useful reference even as the UI gets unified.
15. `compatible_categories` tagging on materials (even though enforcement is currently inconsistent, PRC-050) — the right concept, needs enforcement not redesign.
16. The vehicle base-sqft-by-type lookup embedded in the Materials Library (`server.py` L2732-2736) — the right data shape, just needs promoting to a first-class table.

17. Phase 0's locked decisions themselves (`PHASE_0_PRICING_DECISIONS_FORMULA_GOVERNANCE.md`) — money/rounding rules, overhead basis, warning taxonomy, override taxonomy, permission table — all still valid, foundational to every phase in the Build Order.
 18. Sprint 1's already-built and unit-tested primitives (`Money`, `PricingResult`, `PricingSnapshot`, `PricingWarning`, `PricingOverride`, `PricingChangeLog`, pricing permissions) — not "legacy," but explicitly the CURRENT correct foundation this whole handoff package builds toward.
 19. The Custom category's role as a controlled, permission-gated manual-entry fallback (concept, not implementation) — every business needs an escape hatch; the rebuild rule is that it stays disciplined.
 20. The fundamental idea, already partially present in the legacy system's better corners (Job Ticket live/commit, Historical Import suggest/accept), that a human explicitly confirms before something becomes permanent — this philosophy, more than any specific formula, is what the whole rebuild generalizes.
-

5. Top 20 pricing-related code patterns, routes, models, and systems that must NOT be ported

1. `routes/wrap/core.py::WrapPricingConfig/_compute_pricing_snapshot()` — the independent Wrap Lab pricing engine (M-01).
2. `routes/wrap/core.py::WrapMaterialUpdate/WRAP_MATERIAL_TYPES` — the per-ticket materials array (M-03).
3. `routes/webstores.py::Product.base_cost/retail_price` as calculator-disconnected manually-typed floats (M-02).
4. `routes/webstores.py::ProductCategory` as a stand-in for real pricing-category mapping — it is a display category, not a pricing category.
5. `server.py::calculate_pricing()`'s giant if/elif dispatcher structure (M-05) — the formula content inside each branch is a reference; the dispatcher shape itself is not.
6. Any raw `float` currency field, anywhere (M-04) — replaced universally by `Money`.
7. The legacy hours-based labor hourly-rate lookup branch inside `calculate_vehicle_graphics()` (and likely other categories) — pending final Owner retirement decision (M-08).
8. `ai_prefill_overrides` — dead placeholder field (M-10).
9. `category_setup_status` in its current non-functional form (M-09) — either gets real enforcement or is removed, never carried forward unchanged.
10. `frontend/src/components/pricing/BannerSetupWizard.js` as an independent entry point — confirmed duplicate write destination with the Pricing Setup Quiz (M-22).
11. `routes/profit_analytics.py`'s `db.jobs/db.job_items` read path for margin/profit reporting (M-18/M-19).
12. `routes/jobs.py::sync_job_items_from_embedded_line_items()` and its "legacy embedded" fallback usage inside `routes/invoices.py` (M-19).
13. `routes/job_tickets.py::save_ticket_pricing()`'s no-provenance, no-permission manual-override mechanism (M-14/M-15) — the ENDPOINT'S current implementation, not the "manual override" concept itself.
14. Any of the 4 confirmed divergent `PricingSnapshot`-equivalent shapes (`job_tickets.pricing_snapshot`, Wrap Lab's `_compute_pricing_snapshot()` output, `jobs.line_items[].cost_snapshot`, and whatever Webstores does at checkout) — none of the four shapes survives; exactly one new shape replaces all of them.

15. `routes/webstores.py::APPAREL_TIER_DEFAULTS` as a second, Webstore-only Apparel pricing table, separate from the Foundation's real Apparel calculator tables.
 16. Any per-category copy of waste/overhead/minimum-charge/rush logic that duplicates what should be one shared helper (M-06).
 17. Any inconsistent overhead-basis calculation that doesn't match the Phase 0 Group 2 canonical basis (M-07).
 18. Wrap Lab's compute-and-persist-together pricing PUT endpoint pattern (`routes/wrap/core.py` L1254-1289) — no live/commit separation, unlike the general Job Ticket flow.
 19. Any second AI-suggestion-acceptance mechanism that doesn't reuse the Historical Import's proven pattern (a future-facing "do not invent a second one" rule for Phase 10, per M-28's underlying lesson).
 20. Any direct legacy-repository "fix" applied to calculator/Wrap Lab/Webstore pricing files outside the previously-approved, separately-tracked security patches in `RUNNING_ISSUE_TRACKER.md` — per the Legacy Repo Usage Rule below, this repository is a reference source, not the build target.
-

6. The exact first rebuild phase that should begin after this handoff package is complete

Phase 2 — Pricing Data Architecture, as already scoped in `PRICING_REBUILD_BUILD_ORDER.md` and previously confirmed with the Owner (this session's own prior turn: the "Revised Phase 2 Plan — Live Recalculation Support" documented in `PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md`). Phase 1 (shared primitives) is already complete and tested (Sprint 1, 2026-02-15). This handoff package does not change that sequencing — it adds the deeper, code-verified evidence base (Wrap Lab's confirmed divergent engine, Webstores' confirmed disconnected pricing, the confirmed legacy-reporting-path mismatch, the confirmed Banner Wizard/Quiz overlap, the confirmed absence of customer-specific pricing) that Phase 2 and every phase after it should build against. **This package does not itself authorize starting Phase 2 — that remains the Owner's explicit go-ahead to give separately.**

7. The Legacy Repo Usage Rule

The legacy repository is a source of pricing formulas, business rules, workflows, examples, configuration data, and verification cases. It is not the place where the final Pricing Foundation, calculators, Wrap Lab pricing, or Webstore pricing architecture should be rebuilt.

SINGUY AI — PHASE 0 PRICING DECISIONS, FORMULA GOVERNANCE & BUILD RULES

Mode: Documentation only. No code written, no calculators modified, no migrations created, no live pricing behavior changed, no implementation work begun. **Source material:**

/app/memory/PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md, /app/memory/PRICING_SPEC.md, /app/memory/pricing_spec.md, /app/memory/pricing_quiz_spec.md,

/app/memory/PRICING_SYSTEM_CALCULATORS_ONBOARDING_QUIZ_REBUILD_DOC.md **Date:** 2026-02-15

Labeling convention used throughout this document:

- **Confirmed** — directly verified in the source documents/code this session or a prior one.
- **Recommended Decision** — this document's proposed answer, grounded in the evidence above.
- **Owner Approval Required** — a recommended decision that still needs an explicit yes/no/revise from the Shop Owner/Product Owner before it is locked.
- **Open Business Decision** — no recommended default exists; a genuine choice with no clearly superior option from the evidence alone.
- **Assumption to Verify** — plausible but not confirmed by a direct code read; requires a follow-up verification task before final approval.

PURPOSE

Phase 0 exists to lock the business rules, formula rules, override rules, data rules, and ownership decisions that every pricing calculator will depend on.

Nothing in Pricing Foundation, Banners, Wrap Lab, Webstores, Quotes, Orders, or Reports may be rebuilt until the decisions in this document are approved.

This document does not make hidden assumptions. Every unresolved rule is explicitly labeled per the convention above — nothing is quietly treated as settled.

1. PHASE 0 EXECUTIVE SUMMARY

Why pricing rebuild work cannot begin yet: The investigation (

PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md) identified 59 confirmed issues, a meaningful number of which are not code defects at all but **unresolved business rules** — the overhead basis, the rounding policy, the override-approval workflow, the pricing precedence order, and the canonical vehicle-wrap formula. Building any calculator (even the simplest, Banners) against an unresolved rule means re-deriving that formula a second time the moment the rule is finally decided. Two of the source specification documents themselves (PRICING_SPEC.md and pricing_spec.md) **directly disagree** with each other on whether design and install labor sit inside the overhead basis (see §3) — concrete proof that this ambiguity is real, not hypothetical, and must be resolved before any formula is coded.

Which pricing issues are blocked by unresolved business rules: PRC-005 (overhead basis), PRC-003/PRC-017 (money/rounding), PRC-014/PRC-015/PRC-016 (override guardrails), PRC-009/PRC-034 (customer pricing precedence), PRC-001/PRC-032 (Vehicle Wrap canonical formula), PRC-018/PRC-019 (shipping/overtime), PRC-024/PRC-053 (Banner Wizard vs. Quiz), PRC-033 (Webstore override rules), PRC-012/PRC-027 (change-history

governance). All are cataloged with their Decision IDs in §2.

Which decisions affect every calculator: The overhead formula (§3), currency/rounding rules (§4), below-cost/margin guardrails (§5), override permissions (§6), the precedence hierarchy (§8), and the global formula governance rules (§15) apply identically to all 9 pricing categories — Banners, Rigid Signs, Cut Vinyl, Digital Print, Vehicle Graphics/Wraps, Apparel, Services, Promotional, Custom.

Which decisions affect only Wrap Lab, Webstores, or specialized categories: The Vehicle Wrap canonical formula decision (§10) affects only Vehicle Graphics/Wraps and Wrap Lab. The Webstore override rules (§9) affect only Webstore-sold products. The Banner Wizard vs. Quiz decision (§11) affects only Banners' onboarding path. Shipping/installation/service-call ownership (§12) affects categories that use those cost components (primarily area-based categories, Services, Vehicle Wraps).

What must be approved before Sprint 1 can begin: Per the Recommended First Implementation Sprint already documented in the Remediation Plan, Sprint 1 builds the Money utility, standardized schemas, permissions, change-log infrastructure, and the golden-formula test framework — none of which can be built correctly without §3 (overhead), §4 (currency/rounding), §5/§6 (override/permission rules), §8 (precedence hierarchy), and §14 (change-history governance) being locked first. The full list of Sprint-1 gating decisions is restated in §18.

What Phase 0 does not include: Phase 0 does not write a PricingEngine, does not touch server.py, does not migrate any tenant's pricing_configuration document, does not change what a single existing quote or webstore order currently costs a customer, and does not resolve every open question in the Remediation Plan — only the subset that blocks Sprint 1 and the first calculator port (Banners, Phase 3). Lower-priority open items (PRC-051 hardware auto-suggestion, PRC-057 material comparison mode, PRC-055 promo codes, etc.) remain correctly deferred to their assigned later phases and are not re-litigated here.

Phase 0 creates rules and governance, not customer-facing features. No shop owner, staff member, or customer will see any visible change as a result of this document being approved.

2. DECISION APPROVAL REGISTER

Every decision defaults to **Proposed** unless the source documentation proves it was already locked (none were — every pricing business rule found in the investigation was either inconsistent across categories or entirely undocumented).

Decision ID	Decision Name	Status	Owner Approval Required	Affected Modules	Blocks V Phases
P0-PRC-001	Canonical overhead cost basis (material + production labor only)	Proposed	Yes	All 9 categories	Phase 1, 3, Phase 5
P0-PRC-002	Whether design labor is included in the overhead basis	Proposed	Yes	All categories with design charges	Phase 3, 4, Phase 5
P0-PRC-003	Whether installation labor is included in the overhead basis	Proposed	Yes	All categories with install charges, Vehicle Wraps	Phase 3, 4, Phase 5
P0-PRC-004	Whether subcontractor cost is included in the overhead basis	Proposed	Yes	Services, Promotional	Phase 5
P0-PRC-005	Whether setup fees are included in the overhead basis	Proposed	Yes	Promotional, Apparel, Services	Phase 3, 5
P0-PRC-006	Whether shipping/delivery is included in the overhead basis	Proposed	Yes	All categories (once shipping exists, PRC-018)	Phase 3, 5

P0-PRC-007	Whether Services/Apparel/Promotional/Custom apply the overhead formula at all	Proposed	Yes	Services, Apparel, Promotional, Custom	Phase 5
P0-PRC-008	Store all pricing money as integer cents	Proposed	Yes	Every category, Invoices, Reports	Phase 1 (blocks everything downstream)
P0-PRC-009	Rounding method and rounding timing (per-bucket vs. final-only)	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-010	Whole-dollar pricing toggle rule	Proposed	Yes	Category/tenant settings	Phase 3
P0-PRC-011	Rounding rule for quantity discounts, rush fees, and tax	Proposed	Yes	Every category, Invoices	Phase 3
P0-PRC-012	Rounding rule for manual overrides and imported historical prices	Proposed	Yes	Manual override flow, Historical Import	Phase 3, 10
P0-PRC-013	Below-target-margin behavior (warn-only at launch)	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-014	Below-cost behavior (warning + required override reason)	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-015	Staff below-cost override requires Owner/Admin approval	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-016	Required override-reason taxonomy, including loss-leader/relationship-price	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-017	Warning visibility across Quote, Invoice, Work Order Summary, Reports, Customer Portal	Proposed	Yes	Quotes, Orders, Invoices, Reports, Customer Portal	Phase 1, 6, Phase 8
P0-PRC-018	Warning/override interaction with customer discount, wholesale, and Webstore override	Proposed	Yes	Customers, Webstores	Phase 7, 8
P0-PRC-019	PRICING_VIEW / PRICING_EDIT_FOUNDATION / PRICING_OVERRIDE default role mapping	Proposed	Yes	RBAC (models/auth.py)	Phase 1
P0-PRC-020	Cost/margin internal data visibility scope (who sees raw cost/margin)	Proposed	Yes	All calculator UIs, Customer Portal	Phase 1, 6
P0-PRC-021	Formal below-cost override approval workflow (staff request → owner approval → snapshot)	Proposed	Yes	Every category	Phase 1, 3
P0-PRC-022	Webstore public pricing edit permission	Proposed	Yes	Webstores	Phase 8
P0-PRC-023	Owner-share/platform-fee visibility permission	Proposed	Yes	Webstores, Reports	Phase 8, 9
P0-PRC-024	Customer discount percent MVP scope and precedence position	Proposed	Yes	Customers, all categories	Phase 7
P0-PRC-025	Customer discount cannot silently push price below cost	Proposed	Yes	Customers	Phase 7
P0-PRC-026	Customer-specific minimum-margin exceptions — deferred, not MVP	Proposed	Yes	Customers (future)	Phase 7 (future expansion)
P0-PRC-027	Customer tax-exempt separation remains selling-price-independent	Proposed (reaffirm existing behavior)	Yes	Customers	Phase 7
P0-PRC-028	Canonical 14-layer pricing precedence hierarchy	Proposed	Yes	Every category, Customers, Webstores	Phase 1, 3, Phase 8

P0-PRC-029	Webstore public price override principle (never erases internal snapshot)	Proposed	Yes	Webstores	Phase 8
P0-PRC-030	Which Webstore products remain manually priced vs. calculator-backed	Proposed	Yes	Webstores	Phase 8
P0-PRC-031	Vehicle Wrap canonical formula direction	Proposed (direction only — final formula deferred pending code diff)	Yes	Vehicle Graphics/Wraps, Wrap Lab	Phase 4
P0-PRC-032	Banner Setup Wizard / Pricing Setup Quiz consolidation direction	Proposed	Yes	Banners onboarding	Phase 10
P0-PRC-033	Shipping/delivery pricing ownership (Pricing Foundation vs. Fulfillment)	Proposed	Yes	All categories, Fulfillment	Phase 3, 5, Phase
P0-PRC-034	Installation labor ownership split (pricing input vs. fulfillment execution)	Proposed	Yes	All categories with install, Fulfillment	Phase 3, 6
P0-PRC-035	Service-call/trip-charge ownership	Proposed	Yes	Services, Fulfillment	Phase 5, 6
P0-PRC-036	Overtime labor rate policy	Proposed	Yes	Labor rate configuration, all categories	Phase 3
P0-PRC-037	Weekend/after-hours/emergency labor policy	Proposed	Yes	Labor rate configuration, all categories	Phase 3, 5
P0-PRC-038	Service-call minimum, travel-time billing, and second-installer policy	Proposed	Yes	Services, Vehicle Wraps	Phase 4, 5
P0-PRC-039	PricingChangeLog vs. Activity Log write-routing rules	Proposed	Yes	Every pricing write path	Phase 1, 2
P0-PRC-040	Historical Pricing Snapshots are permanently frozen and never recalculate	Proposed (reaffirm existing correct behavior)	Yes	Pricing Snapshots, Reports	Phase 1, 9
P0-PRC-041	Adoption of the full Phase 0 Formula Governance Rulebook (§15) as binding for every future calculator	Proposed	Yes	Every category, all future calculators	Phase 3, 4, Phase

3. CANONICAL OVERHEAD FORMULA

Current behavior by category (Confirmed, with one direct source conflict)

Two internal specification documents describe the overhead formula differently, and this discrepancy is itself the strongest piece of evidence for why this decision must be locked in Phase 0 rather than assumed:

- `pricing_spec.md`'s "**Master Formula**" (its stated general engine, lines 21-40) explicitly defines:

```
labor_cost    = (production_hours × production_rate) + (design_hours × design_rate) + (install_hours × install_rate)
overhead_cost = (material_cost + labor_cost) × (overhead_percent / 100)
```

This formula includes **design and install labor inside the overhead basis**.

- `PRICING_SPEC.md`'s **own rebuild rule (§16.7)** states the opposite conclusion after documenting the real per-category code: "*overhead is calculated on `(material_cost + direct_labor_cost)` only. Setup fees, design fees, and install fees are pass-through charges — never subject to overhead.*"
- **The traced per-category calculation flows in `PRICING_SPEC.md`** (Banners §4.2 step 6-7, Cut Vinyl §6.2 step

9-10) show `overhead_cost = calculate_overhead_cost(material + labor, ...)` computed *before* `design_cost` and `install_cost` are added — and both design/install costs are added to `suggested_price` *after* the cost-plus/markup step, not folded into the marked-up cost-plus basis. This is consistent with §16.7's rule (design/install as pass-through), not with the Master Formula's version.

- **Confirmed, explicit in code comments:** In Promotional, the setup fee is added "FLAT (not marked up, not in overhead basis)" (`PRICING_SPEC.md` §11.2 step 4). Cut Vinyl similarly excludes its file-cleanup fee from the pre-overhead cost stack.
- **Confirmed per `PRICING_SPEC.md` §16.7's own direct claim:** Banners and Vehicle Wraps include "the setup fee" in the overhead basis, while Promotional and Cut Vinyl exclude it — **this specific claim is flagged as an Assumption to Verify** against the traced Banners/Cut Vinyl formulas above, which both show design/install/finishing costs added *after* the overhead+markup step in both categories, not before. The exact mechanism behind §16.7's claim (which specific cost bucket differs) needs a direct side-by-side code diff before this document's recommendation is finalized as code — flagged, not guessed.
- **Confirmed:** Apparel's documented formula (`pricing_spec.md` §Apparel) shows no explicit overhead line at all — only `total_cost × 2.15 markup`. Services' documented formula similarly shows no overhead line — `labor + travel + trip + equipment + subcontract + permit`, then rush, with no overhead step. This suggests these two categories may not apply the shared overhead calculation the same way area-based categories do — **Assumption to Verify**, addressed in §3's Services/Apparel/Promotional/Custom question below.

Why inconsistent overhead creates inaccurate margins

If Category A includes design/install labor (and setup fees) in its overhead basis and Category B does not, then a "15% overhead" charge is not the same dollar amount relative to the same job in both categories — meaning any report that compares `profit_margin_percent` across categories, or any tenant-wide "average margin" figure, is comparing numbers computed with two different definitions of cost. This directly corrupts the Profit & Margin Analytics repair planned in Phase 9 (PRC-021/PRC-022) — that phase cannot produce trustworthy comparative reporting until every category computes overhead identically.

Possible options

Option	Basis Includes	Basis Excludes
(a) Material + production labor only	Material cost, direct production labor	Design, install, subcontract, setup fees, shipping
(b) Material + all labor (production + design + install)	Material cost, production + design + install labor	Subcontract, setup fees, shipping
(c) Material + labor + setup fees (current Banners/Vehicle Wraps behavior per §16.7's claim)	Material, labor, setup fees	Subcontract, shipping
(d) Category-specific basis (status quo)	Varies per category	Varies per category

Recommended formula

Recommended Decision (P0-PRC-001) — Owner Approval Required:

```
Overhead = (direct material cost + direct production labor cost + direct
installation labor cost) × configured overhead percentage
```

This is the default provided for this document and is adopted as the baseline recommendation. It sits between option (a) and (b) above — it includes installation labor (which the user-provided default explicitly names) but leaves design labor

as a separate, explicitly flagged sub-decision below, since `PRICING_SPEC.md`'s own §16.7 rule argues install labor should be excluded (aligned with option (a)), creating a direct tension between the given default and the strongest documented evidence. **This tension is not silently resolved — see P0-PRC-003 below.**

Cost buckets included: Direct material cost, direct production labor cost. Installation labor cost is included in the recommended default formula above but is a distinct, separately-flagged approval decision (P0-PRC-003) given the conflicting evidence.

Cost buckets excluded: Tax, customer discounts, pass-through fees (permits, subcontract pass-through), rush premiums, deposits, and profit markup are never included in the overhead basis, per this document's instruction and consistent with every source document reviewed — no evidence anywhere suggests otherwise.

Whether design labor is included — P0-PRC-002, Owner Approval Required: Not decided by default. Two options:

- (a) Exclude design labor from the overhead basis, add it as a pass-through cost after the cost-plus/markup step — matches `PRICING_SPEC.md` §16.7's explicit rebuild rule and the traced Banners/Cut Vinyl formula order.
- (b) Include design labor in the overhead basis — matches `pricing_spec.md`'s Master Formula.
- **Recommended Decision:** (a), exclude — because it is the conclusion the investigation's own most detailed, code-traced document (`PRICING_SPEC.md`) reached after reading the real per-category formulas, whereas the conflicting Master Formula in `pricing_spec.md` appears to be a simplified/idealized restatement not fully cross-checked against the actual category-by-category flows. Owner must approve or override this reasoning.

Whether installation labor is included — P0-PRC-003, Owner Approval Required: This is the most directly conflicting item in the entire overhead question:

- The document-provided default formula explicitly includes installation labor.
- `PRICING_SPEC.md` §16.7's own rebuild rule explicitly excludes it ("install fees are pass-through charges — never subject to overhead").
- The traced Banners/Cut Vinyl formulas add `install_cost` *after* the overhead+markup step, consistent with exclusion.
- **Recommended Decision:** Exclude installation labor from the overhead basis, treating it as a pass-through cost added after cost-plus/markup — for internal consistency with the design-labor recommendation above and with the strongest documented evidence (§16.7 + the traced formulas). **This recommendation deliberately diverges from the literal default formula text supplied for this document; the Owner must explicitly approve either the literal default (include install labor) or this evidence-based alternative (exclude it) before Phase 1 locks the `PricingEngine`'s overhead implementation.**

Whether subcontractor cost is included — P0-PRC-004, Owner Approval Required: Confirmed excluded for Services (§Services formula: subcontract cost is a separate additive bucket, optionally marked up on its own, never folded into the material+labor overhead basis) and Promotional (vendor unit cost is the "material" analog, not a separate subcontract concept). **Recommended Decision:** Exclude subcontractor/outsourced cost from the overhead basis everywhere — it already carries its own optional markup mechanism and should not be double-taxed by overhead too.

Whether setup fees are included — P0-PRC-005, Owner Approval Required: Confirmed excluded for Promotional and Cut Vinyl (explicit code comment/formula order). **Confirmed claimed as included** for Banners and Vehicle Wraps per `PRICING_SPEC.md` §16.7, though the exact mechanism is an Assumption to Verify per the analysis above.

Recommended Decision: Exclude setup fees from the overhead basis everywhere, matching §16.7's own stated rebuild rule and the majority of traced category behavior — setup fees become a uniformly pass-through charge across all 9

categories.

Whether shipping/delivery is included — P0-PRC-006, Owner Approval Required: Shipping/delivery pricing does not exist as a cost component in any category today (PRC-018 — confirmed absent, only a display-only notes field exists). This decision is therefore prospective: once §12 (P0-PRC-033) decides shipping is a Pricing Foundation cost component, should it sit inside the overhead basis? **Recommended Decision:** No — treat shipping/delivery as a pass-through cost added after cost-plus/markup, consistent with the design/install/setup-fee pattern above, for the same reason: it is a cost that varies job-to-job in a way that doesn't scale with production effort, and folding it into overhead would distort the overhead percentage's meaning.

How overhead behaves for Services, Apparel, Promotional, and Custom — P0-PRC-007, Owner Approval Required:

- **Apparel:** Documented formula shows no explicit overhead line (pricing_spec.md §Apparel) — only $\text{total_cost} \times 2.15$ markup, where total_cost = blank cost + decoration cost + add-ons + labor + design. **Assumption to Verify:** whether the shared `calculate_overhead_cost()` function is silently called somewhere in Apparel's actual `server.py` implementation despite not appearing in this simplified spec. **Recommended Decision, pending verification:** If not currently applied, do not add it retroactively without an explicit product decision — Apparel's markup multiplier (2.15×) may already implicitly account for overhead the way its table-first pricing does for margin generally; adding a second overhead layer on top could double-count.
- **Services:** Documented formula shows no overhead line either ($\text{labor} + \text{travel} + \text{trip} + \text{equipment} + \text{subcontract} + \text{permit}$, then rush). **Recommended Decision:** Do not add overhead as a separate line — Services pricing is fundamentally rate-based (the labor rate itself is assumed to already be overhead-inclusive, consistent with how professional service hourly rates typically work), and permit/subcontract are already explicitly pass-through per §Services confirmed behavior.
- **Promotional: Confirmed** to include an overhead step (PRICING_SPEC.md §11.2 step 5, excluding the setup fee). **Recommended Decision:** Keep overhead applied here, using the same canonical basis (material + production labor) as the area-based categories, since Promotional's cost-plus structure is materially similar to them.
- **Custom:** No formula exists (pure manual entry) — overhead is not applicable by design. No decision needed; Custom remains the deliberate escape hatch.

How overhead is stored in snapshots: Every `PricingSnapshot` (per the Remediation Plan's schema, §4C) stores `overhead_cost` as its own explicit cost bucket, plus an `overhead_basis` explainability object recording the formula, the exact basis amount, and which cost components were included — reusing the existing, already-well-designed `overhead_basis` field confirmed in PRICING_SPEC.md §3.8's response shape, updated to reflect whichever basis this decision locks.

How overhead is displayed in breakdowns: The shared `<CalculationBreakdown>` component (Remediation Plan PRC-043) shows `overhead_cost` as its own line, with the `overhead_basis` explainability data available on demand (e.g., "15% of \$42.00 material + labor = \$6.30"), consistent with the already-correct pattern the current system uses.

Migration impact: Every existing tenant's live `pricing_configuration` continues to compute overhead exactly as it does today until Phase 3 ports a category onto the new canonical formula. No historical `pricing_snapshot` is retroactively recalculated (see §14/P0-PRC-040) — old snapshots keep whatever overhead basis was live when they were created.

Tests required: A golden-formula test per category proving the new canonical overhead basis produces the documented expected cents value for a fixed set of inputs; a cross-category consistency test proving the same \$ of material + labor

cost produces the same \$ of overhead regardless of category, once P0-PRC-001 through P0-PRC-007 are locked.

4. CURRENCY STORAGE, PRECISION, AND ROUNDING RULES

Confirmed current state: Every money field across `models/pricing.py` and every category's calculation output is a raw Python `float` (PRC-003). No rounding rule is documented anywhere for any category — every "Rounding Rules" field in the source investigation's category-by-category breakdown (\$5F per category) reads "Not explicitly specified" (Assumption to Verify, confirmed absent by omission, not guessed).

Official pricing currency rules

P0-PRC-008 — Store all money as integer cents — Owner Approval Required. Every stored and computed monetary value (material cost, labor cost, overhead, markup result, final selling price, snapshot values, invoice line items) is an integer representing cents. No raw float currency value survives anywhere in the rebuilt pricing domain. This directly resolves PRC-003.

Where decimal quantities are allowed: Non-money quantities — square footage, linear feet, hours, minutes, piece counts, percentages — remain decimal (float or fixed-point decimal) values, since they are physical measurements or rates, not currency. Only the final dollar-and-cents *money* values are cents-integers.

How percentages are stored: As a decimal fraction or basis-point integer at the configuration layer (e.g., `overhead_percentage: 15` meaning 15%, applied as $\times 0.15$), never as a pre-multiplied dollar amount. **Recommended Decision:** store as a plain percentage number (matching current behavior, e.g. `15` for 15%) for continuity with existing tenant configuration and the Quiz's existing question format — no migration of the percentage fields themselves is required, only of how they're *applied* to cents-based money.

How markup multipliers are stored: As a decimal multiplier (e.g., `2.35` for a $2.35\times$ markup), consistent with current behavior — multipliers are not currency and do not need cents-integer storage, only careful multiplication order (multiply the integer-cents cost by the float multiplier, then round once — see rounding order below).

When rounding occurs / whether intermediate steps round or only final totals round:

Recommended Decision (P0-PRC-009):

```
Store money in integer cents.
Keep calculation precision internally until each cost bucket is finalized.
Round each customer-visible cost bucket to cents.
Round the final selling price to the nearest whole dollar only when a category
or tenant setting explicitly enables whole-dollar pricing.
Default rounding method: half-up.
Never repeatedly round intermediate values in a way that compounds errors.
```

Concretely: a formula like `waste_area × material_cost_per_sqft` is computed at full available precision (e.g., using integer-cents-per-unit inputs multiplied by a precise decimal area, or a fixed-point `Decimal` type internally) and only rounded to the nearest cent once, at the moment that value becomes a **named cost bucket** in the `PricingResult` (`material_cost`, `labor_cost`, `overhead_cost`, etc.) — not at every micro-step inside the formula (e.g., not rounding the waste-adjusted area itself, not rounding an intermediate per-sqft rate before it's multiplied by area). This is the specific practice the recommended rule prohibits ("never repeatedly round intermediate values in a way that compounds errors").

Rounding method — half-up: Standard "round half away from zero" (e.g., $\$1.005 \rightarrow \1.01), the simplest and most predictable method for both staff and customers, and consistent with typical accounting/POS software conventions.

Owner Approval Required — an alternative (banker's rounding / round-half-to-even) exists but has no evidence of being expected anywhere in the source documentation; half-up is the default absent a stated preference.

Rules for fractional cents created by percentage calculations: A percentage calculation (overhead %, markup, discount %) applied to an integer-cents value will frequently produce a fractional cent (e.g., $847 \text{ cents} \times 1.15 = 974.05 \text{ cents}$). This fractional result is carried at full precision through the remaining formula steps and only rounded to the nearest integer cent when it becomes a finalized, displayed cost bucket or the final selling price — never rounded mid-formula.

Rules for quantity discounts: The quantity-discount percentage is applied to the pre-discount `sell_base` (per confirmed current behavior, `PRICING_SPEC.md` §3.7 — "applied to `sell_base` before adding design/install/setup fees"); the discounted result rounds to the nearest cent once, as its own named value, before design/install/setup fees (which are separate cost buckets) are added on top.

Rules for rush fees: The rush percentage is applied to the pre-rush total (after quantity discount, before tax), and the rush-inclusive total rounds to the nearest cent once. Rush is never applied to an already-rounded-and-then-re-derived intermediate value.

Rules for tax calculations: Tax is computed on the final, fully-resolved selling price (after every pricing-domain step above) and is explicitly outside the pricing-domain's own rounding scope — tax rounding follows whatever standard the Invoice/Order domain already uses (confirmed elsewhere as `tenant.default_tax_rate` applied at invoice time, not recalculated by the pricing engine itself). This document does not redefine tax rounding, only confirms the pricing engine hands tax a single, already-rounded pre-tax total.

Rules for displayed prices versus stored values: The stored value is always the integer-cents ground truth. Display formatting (e.g., "\$847.00") is a pure presentation-layer conversion done at render time and never written back to the database — this prevents a rounding artifact introduced for display from ever contaminating the stored snapshot.

Rules for imported historical prices (Historical Invoice Import, PRC-036): Prices extracted from an uploaded invoice (PDF/CSV/XLSX) are parsed into integer cents at ingestion time, using half-up rounding if the source document contains sub-cent or ambiguous values (e.g., an OCR-extracted price with unclear decimal placement is flagged as a warning, per the existing "confidence" labeling system already correctly used by this feature — not silently rounded and accepted).

Rules for manual overrides: A manually-entered override price is parsed directly into integer cents (e.g., a staff member types "\$847.50" → stored as `84750` cents) with no additional rounding applied — the human-entered value is authoritative and exact, never re-derived or adjusted by the rounding rules above (those rules govern *calculated* values, not direct entry).

Worked examples

Banners: A 3×6 ft banner (18 sqft, waste-adjusted to 19.44 sqft at 8% waste) with a material cost of \$1.10/sqft → $1944 \times 110 = 213,840$ cent-sqft-units → $213840 / 100 = 2138.4$ cents = round-half-up → **\$21.38** material cost bucket. This rounds once, here, as the finalized `material_cost` bucket — not at the waste-adjustment step, not at the per-sqft lookup step.

Apparel: A 24-piece HTV order at 1 color: $24 \times \$0.50 \times 100 = 1200$ cents decoration cost + $\$10.00$ setup fee (1000 cents) = **2200** cents decoration bucket — already a whole-cent value, no rounding needed, illustrating that most apparel per-piece math naturally lands on clean cents, but the rule still applies identically if a future decoration formula introduces a fractional rate.

Vehicle Wraps: A Sprinter Van full wrap at 350 base sqft × 100% coverage × 1.15 waste = 402.5 sqft, material cost

\$3.50/sqft (premium cast per `pricing_spec.md`'s wrap material table adjusted for this example) → $402.5 \times 350 \text{ cents} = 140,875 \text{ cents} = \$1,408.75$ material cost bucket, rounded once here; this then feeds into the larger cost-plus vs. package-benchmark comparison (\$10) without any further intermediate rounding before the final `max()` comparison is resolved and *that* result rounds once more as the finalized `selling_price`.

5. MINIMUM MARGIN, BELOW-COST, AND MANUAL OVERRIDE GUARDRAILS

Confirmed current state: No margin-specific or below-cost-specific warning exists anywhere — only a generic `warnings: [string]` array used for missing-material/fallback notices (PRC-015). The system does not appear to block a manual override from going below cost today, but this has never been made an explicit, documented rule (PRC-016).

Difference between below cost and below target margin:

- **Below cost** means the selling price is less than `total_cost` (material + labor + overhead) — the shop would lose money on this specific job.
- **Below target margin** means the selling price is above cost but the resulting `profit_margin_percent` is lower than the tenant's configured `target_profit_margin_percent` (default 40% per confirmed Foundation defaults) — the shop still profits, just less than its own stated goal.

These are meaningfully different severities and must produce different warning behavior, not be collapsed into one generic flag.

Warning severity levels

Severity	Condition	Meaning
info	Fallback/default used (e.g., minute-based config missing, hours-fallback fired)	Informational, no financial risk
warning	Below target margin, above cost	Shop profits less than its own goal on this job
critical	Below cost	Shop loses money on this job unless explicitly approved

Recommended launch rules

P0-PRC-013 — Below target margin: warning only — Owner Approval Required. No block, no approval step. A `margin_warning` (severity `warning`) is attached to the `PricingResult/PricingSnapshot` and visible per §P0-PRC-017/020's visibility rules.

P0-PRC-014 — Below cost: warning plus required override reason — Owner Approval Required. A `below_cost` warning (severity `critical`) is attached, and the system requires a selected reason before the price can be finalized (see taxonomy below).

P0-PRC-015 — Staff below-cost override requires Owner/Admin approval — Owner Approval Required. Staff may *request* a below-cost price with a reason, but cannot finalize/commit it to a Quote/Order Item/Snapshot without an Owner or Admin explicitly approving it (see the full workflow in §6). Owner/Admin themselves may finalize a below-cost override directly, without a second approver, consistent with their elevated trust level elsewhere in the RBAC system.

Whether staff can enter below-cost overrides — Confirmed answer under this rule: yes, staff can *enter* one, but cannot *finalize* it alone. This distinguishes "typing a number" from "committing a price," matching the

approval-workflow pattern already correctly used elsewhere in the platform (e.g., Purchasing's Admin-creates/Owner-approves segregation of duties, cited as the platform's own reference-quality pattern in the Remediation Plan's Bucket 4 analysis).

Required override reason options

P0-PRC-016 — Required override-reason taxonomy — Owner Approval Required. Recommended starting taxonomy (extensible, not exhaustive):

- `loss_leader` — deliberate strategic underpricing to win/retain a customer relationship
- `relationship_pricing` — long-term customer, one-time accommodation
- `competitive_match` — matching a known competitor quote
- `damaged_goods_or_rework` — correcting a shop error, no full charge appropriate
- `material_already_owned` — customer supplied materials, cost basis doesn't apply normally
- `owner_discretion` — no specific reason, Owner/Admin judgment call
- `other` (custom, free-text) — **Recommended Decision:** Custom reasons are allowed via an `other` option with a required free-text explanation, so the taxonomy doesn't force an artificial fit and the audit trail still captures real context.

Whether a loss-leader or relationship-price reason should be allowed: Yes — explicitly included above (`loss_leader`, `relationship_pricing`) per this document's own guidance that below-cost blocking should not prevent "a legitimate loss-leader/relationship-pricing decision a shop owner might intentionally make" (consistent with the Remediation Plan's Phase 0 Decision #3 rationale).

Warning visibility and provenance

P0-PRC-017 — Warning visibility across surfaces — Owner Approval Required.

Surface	Sees the warning?	Detail shown
Calculator UI (Staff/Owner/Admin)	Yes	Full warning, severity, code, explanation, recommended action
Work Order Summary	Yes (internal)	Presence of an override flag only, not full cost/margin detail — production staff need to know "this was manually adjusted," not necessarily why
Quote (internal view)	Yes	Full detail, same as calculator
Quote (customer-facing)	No	Customers never see margin warnings, cost data, or override reasons
Invoice (customer-facing)	No	Same as Quote
Invoice (internal view)	Yes	Full detail
Reports (Profit & Margin Analytics)	Yes	Aggregated and per-item, including override frequency and reason breakdown
Customer Portal	No	Consistent with the platform's existing correct pattern of never exposing internal cost/margin to customers

Whether a warning becomes part of the Pricing Snapshot — Confirmed Recommended Decision: Yes. Every warning present at commit time is stored inside `PricingSnapshot.warnings[]` (typed `PricingWarning` objects per the Remediation Plan's schema), so the warning history travels with the frozen price forever, even if the live Foundation config later changes.

How warnings behave when customer discounts, wholesale pricing, or Webstore retail overrides are involved (P0-PRC-018, Owner Approval Required): A customer discount, wholesale tier, or Webstore public-price override that pushes an otherwise-adequate price below cost or below target margin triggers the **exact same warning/approval rules** as a direct manual override — there is no separate, weaker guardrail path for discount-driven below-cost pricing. Concretely: if applying a customer's 15% discount would bring a job below cost, the system raises the same **below_cost** critical warning and requires the same reason + approval workflow before the discounted price can be committed. This closes what would otherwise be an obvious loophole (staff/customers "discounting around" the override guardrail instead of overriding directly).

6. MANUAL OVERRIDE PERMISSIONS AND APPROVAL WORKFLOW

Confirmed current state: No `Permission.PRICING_*` enum entries exist anywhere in `models/auth.py`. Foundation editing uses a hardcoded role-string check (`owner/admin/platform_admin`). Calculator *usage* and manual override entry have no confirmed permission gate at all (PRC-013, PRC-014).

P0-PRC-019 — New pricing permissions and default role mapping — Owner Approval Required

Permission	Purpose
<code>PRICING_VIEW</code>	View calculator results, view own quotes/orders' prices
<code>PRICING_EDIT_FOUNDATION</code>	Edit Pricing Foundation settings, materials, labor rates, markup, category config, templates
<code>PRICING_OVERRIDE</code>	Enter and finalize a manual price override without requiring a second approver

Recommended default role behavior:

Role	<code>PRICING_VIEW</code>	<code>PRICING_EDIT_FOUNDATION</code>	<code>PRICING_OVERRIDE</code>	Cost/Margin Visibility
Owner	Yes	Yes	Yes (self-approving)	Full
Admin	Yes	Yes	Yes (self-approving)	Full
Staff	Yes	No	No — may <i>request</i> an override (goes to approval queue), cannot finalize below-cost alone	Configurable (Owner decides per §P0-PRC-019, recommended default is hidden)
Employee	Yes (own assigned jobs only)	No	No	Hidden
Webstore Manager	Yes (Webstore-scoped)	No	Limited — may set the Webstore public-price override field (§9) but not the internal Foundation calculation	Hidden
Webstore Owner	Yes (own store)	No (Foundation is tenant-level, not store-level)	Limited — same as Webstore Manager	Hidden, except the aggregate revenue cost/margin
Platform Admin	Yes (support/ops access)	Yes (support/ops override capability)	Yes	Full (support content only)
Customer	No	No	No	None

Who can view internal cost breakdowns / margin (P0-PRC-020, Owner Approval Required): Recommended default — Owner and Admin only, at launch. Staff visibility into raw cost/margin is left as an explicit tenant-level toggle (some shop owners may trust senior Staff with this; others won't) rather than a hardcoded platform-wide rule, since the investigation found this exact ambiguity already unresolved in the live product ("not confirmed as a hard permission

gate," §5E/§6 of the rebuild investigation) and no evidence points to one universal answer being correct for every shop.

Who can edit materials / labor rates / markup / create templates: PRICING_EDIT_FOUNDATION holders only (Owner, Admin, Platform Admin) — no distinction between these four sub-capabilities at launch; a more granular split (e.g., "can edit materials but not labor rates") is not justified by any documented need and would add complexity without a confirmed use case.

Who can apply a manual selling-price override: Any PRICING_OVERRIDE-or-higher role may apply an override that stays at or above cost. Staff without PRICING_OVERRIDE may still *request* one (see workflow below) but cannot finalize it themselves even above cost, unless the tenant's role configuration grants Staff PRICING_OVERRIDE directly (a per-tenant configuration choice, not a hardcoded platform rule) — **Owner Approval Required** on whether Staff gets PRICING_OVERRIDE by default or must request it every time; recommended default is **request-only for Staff** at launch, revisited if shops report this is too restrictive for routine, above-cost, minor price adjustments.

Who can approve a below-cost override: Owner or Admin only, platform-wide, non-configurable — this is the one guardrail the investigation's findings suggest should not be loosened per-tenant, since it's the platform's core financial-risk control (PRC-014, PRC-016).

Who can edit Webstore public pricing: Webstore Owner/Manager may set the public-facing override field (§9); they cannot edit the underlying Foundation-calculated price or cost data, which remains PRICING_EDIT_FOUNDATION-gated.

Who can view owner share/platform fee information: Owner and Platform Admin only — Webstore Manager/Owner roles see their own store's gross figures but not the platform's fee-split internals, consistent with how this is already handled elsewhere per the Business Finance investigation's Webstore Analytics findings.

Who can view historical pricing changes (PricingChangeLog): Owner and Admin — Staff does not see the change log by default, though this could be revisited if a shop wants transparency; Platform Admin sees it in a support-ops context only.

P0-PRC-021 — Formal below-cost override approval workflow

1. Staff enters a requested price and selects a reason from the required-reason taxonomy (§5), or **other** with free text.
2. The PricingEngine calculates the margin impact of the requested price against the calculated cost.
3. If the requested price is below target margin but at or above cost: a **margin_warning** is logged and attached to the in-progress calculation; **no approval is required**; Staff may proceed.
4. If the requested price is below cost: a **below_cost** critical warning is raised, and the line item is placed into a **pending-approval** state — it cannot be committed to a Quote/Order Item/Snapshot yet.
5. Owner or Admin reviews the pending override (requested price, calculated cost, margin impact, reason) and either **approves** or **rejects** it.
6. If approved, the final result is committed to the PricingSnapshot with full override provenance: **by_user_id** (the requesting Staff member), **approved_by_user_id** (the approving Owner/Admin), **original_calculated_price**, **final_price**, **override_reason**, **requested_at**, **approved_at**.
7. If rejected, the line item reverts to the system-calculated price (or Staff may revise the requested price and resubmit).
8. Both the PricingChangeLog (config-level changes only — not applicable here directly) and the relevant **Activity Log** (order/quote-level activity) receive an event: **"pricing_override_requested"**, **"pricing_override_approved"**, or **"pricing_override_rejected"**, each carrying the same user/timestamp/reason detail as the snapshot's own provenance block, so the event is discoverable both from the Snapshot itself and from the Order/Quote's general

activity history.

This workflow applies uniformly across every category — no category-specific variation is recommended, since the underlying risk (pricing below cost) is identical regardless of which calculator produced the number.

7. CUSTOMER-SPECIFIC PRICING MVP SCOPE

Confirmed current state: Full grep of `models/customer.py` and pricing routes found zero fields for per-customer discounts, price lists, wholesale tiers, or volume agreements (PRC-009, PRC-010, PRC-011). `Customer.is_tax_exempt` exists but only affects the tax calculation at invoice time, never the underlying selling price (PRC-027, confirmed correct and unaffected by this section).

Recommended MVP (P0-PRC-024, Owner Approval Required)

- **One optional customer discount percent**, set on the customer record.
- Applies to eligible sell-price components **after quantity discounts but before rush fees and tax** — i.e., positioned at: `quantity-discounted subtotal → apply customer discount % → rush fee → tax`.
- **Does not apply to** pass-through fees, permits, taxes, or deposits — these remain untouched by any customer discount, consistent with how they're already treated as non-marked-up, non-discountable pass-through amounts elsewhere in the formulas (§Services' permit fee, §Promotional's setup fee).
- **Applies to** material, labor, installation, and services sell-price components — i.e., the actual calculated/marked-up selling price, not the cost inputs themselves.
- **Cannot silently push a price below cost** — if a customer's discount would bring the final price below the calculated cost, the same below-cost warning + approval workflow from §5/§6 (P0-PRC-014/015/P0-PRC-021) applies before the discounted price can be committed (P0-PRC-025, Owner Approval Required).
- **Does not include** wholesale price lists, contracts, or volume agreements at this stage — those are explicitly Phase 7 future-expansion items, not MVP scope.

Whether discounts apply before or after quantity discounts: After — the customer discount is applied to the already-quantity-discounted subtotal, not stacked in the other order. **Recommended Decision**, Owner Approval Required, since no existing behavior determines this (the feature doesn't exist today) and either order is mathematically defensible; "after quantity discount" was chosen because quantity discounts are a property of the *order size*, while the customer discount is a property of the *relationship*, and it's more intuitive for a customer discount to be the "last" percentage-based adjustment before rush/tax.

Whether discounts apply before or after rush fees: Before — rush fees represent an actual elevated cost/urgency to the shop and should not be discounted away by a customer relationship discount; the discount is baked into the "normal" price, and rush is then added on top of that already-discounted amount.

Whether discounts apply to labor, material, installation, services, and pass-through fees: Applies to labor, material, installation, and services sell-price components (i.e., anything that is part of the calculated selling price); does **not** apply to pass-through fees (permits, subcontract pass-through), taxes, or deposits, per the MVP scope above.

Whether discounts can stack (with quantity discount, rush, etc.): Yes, by design — a customer discount stacks with quantity discounts (applied first) and interacts with rush fees (applied after the customer discount) per the precedence order above. There is no "customer discount OR quantity discount, whichever is better" logic; both apply in sequence.

Whether a customer discount can reduce a price below cost: Not silently — see P0-PRC-025 above; the standard

override guardrail applies.

How customer-specific pricing appears in snapshots: The `PricingSnapshot` records the discount percent applied, the pre-discount and post-discount subtotal, and (if relevant) any resulting override-approval provenance, so a report can always answer "how much of this price was the customer discount."

How it appears in reports: Profit & Margin Analytics (Phase 9) gains a discount-attribution breakdown — total revenue impact of customer discounts, by customer, by category, over time.

Whether staff can edit it: Setting a customer's discount percent on their record requires `PRICING_EDIT_FOUNDATION` or a dedicated customer-management permission (not itself part of `PRICING_OVERRIDE`, since it's a standing customer attribute, not a per-transaction override) — Staff cannot set a customer's standing discount without this permission, though any Staff member pricing a job for a discount-eligible customer automatically benefits from it being applied.

Required audit entries: Setting/changing a customer's discount percent writes a `PricingChangeLog` entry (who/when/old value/new value), consistent with every other Foundation-adjacent configuration change (§14).

Explicitly deferred to Phase 7 future expansion (not MVP)

- **P0-PRC-026 — Customer-specific minimum-margin exceptions** — deferred. No evidence this is needed for MVP; a shop wanting to intentionally run thinner margins for one customer can currently use the standard below-cost override workflow (§5/§6) on a per-job basis without a standing "this customer's minimum margin is different" rule.
- Wholesale tiers, reseller pricing, customer price lists, contract pricing, volume agreements, employee discounts, promo-code logic — all explicitly out of scope for the first build, per PRC-010/PRC-011/PRC-055.

P0-PRC-027 — Customer tax-exempt separation reaffirmed: `Customer.is_tax_exempt` continues to affect only the tax calculation at invoice time and has zero interaction with the selling-price calculation performed by the pricing engine — this existing behavior is confirmed correct and is not changed by the customer-discount MVP above; the two mechanisms (discount and tax-exemption) are independent and both apply, in their respective domains, without interfering with each other.

8. CANONICAL PRICING PRECEDENCE HIERARCHY

P0-PRC-028 — Owner Approval Required. This hierarchy locks the existing 13-step chain already documented in the source investigation (`PRICING_SYSTEM_CALCULATORS_ONBOARDING_QUIZ_REBUILD_DOC.md` §5H), extended with the two layers that don't exist today (Customer-Specific Pricing, Webstore-Specific Override) given an explicit, permanent slot instead of floating outside the chain as disconnected systems.

#	Layer	Can Override	Cannot Override	Cost, Price, or Both	Live Config or Frozen Snapshot	Can Trigger Warnings
1	System default	Nothing (lowest layer)	—	Both	Live config	No
2	Tenant Pricing Foundation default	System default	—	Both	Live config	No
3	Category configuration	Tenant default, for that category	System default's general shape	Both	Live config	No
4	Material/product configuration	Category's generic material assumption	—	Cost primarily (feeds price)	Live config	Yes — staleness (PRC-020)

5	Pricing template	Pre-fills calculator inputs	Nothing directly — a template only pre-fills fields the user can still edit	Both (pre-fill only)	Live config	No
6	Vehicle benchmark / apparel pricing table	The formula-derived price, when the table/benchmark is higher	Cannot be overridden by a <i>lower</i> formula result — it's a floor, not a ceiling	Price only	Live config (reference data)	No
7	Customer-specific pricing (discount %)	The post-quantity-discount subtotal	Pass-through fees, tax, deposits (never touches these)	Price only	Live config, applied at calc time	Yes — if it won't push below cost (\$5/\$7)
8	Webstore-specific public retail override	The customer-facing display price only	The internal Foundation calculation, cost, margin, and snapshot — never erased (§9)	Price only (display)	Live config, applied at calc/display time	Yes — if override is below cost (\$5/\$7)
9	Quote/Order Item inputs (dimensions, options, quantity)	Category/material defaults for this specific job		Both	Becomes part of the frozen snapshot at commit	Yes (missing-material fallback)
10	Manual override	The entire computed result of layers 1-9	Nothing above it except an approval-required block	Price only	Frozen at commit, with provenance	Yes — below-cost/manual override
11	Approval-required override gate	Whether a below-cost manual override (layer 10) can finalize	—	Neither directly — it's a gate, not a value	N/A (workflow state)	Yes
12	Quantity discount	The pre-discount sell_base	—	Price only	Applied at calc time, frozen in snapshot	No
13	Rush fee	The post-discount, post-customer-discount total	—	Price only	Applied at calc time, frozen in snapshot	No
14	Tax	The final pre-tax total	—	Price only (customer-facing total)	Computed at invoice time, not by the pricing engine itself	No
15	Deposit	Nothing pricing-related — it's a payment-schedule concept, not a price adjustment	—	Neither (payment terms only)	Invoice/Order domain, not pricing domain	No

Resolving the Webstore/internal-calculation conflict (P0-PRC-028, restated as a binding rule): A Webstore public retail override (layer 8) changes only what a customer sees on the storefront. It must never erase, replace, or hide the internal Foundation-calculated price, cost, margin, owner share, platform fee, or the underlying `PricingSnapshot` — both values (the internal calculation and the public override) are stored side-by-side, and every report/reconciliation reads the internal calculation for true profitability while the storefront reads the override for display. This directly resolves the conflict named in the task brief and is elaborated fully in §9.

9. WEBSTORE PUBLIC PRICE OVERRIDE RULES

Confirmed current state: `WebstoreProduct.retail_price` is a manually-typed number with zero connection to Materials,

Labor, or Markup config (PRC-002). No category mapping exists. No cost/margin data is computed for Webstore products at all today.

Recommended principle (P0-PRC-029, Owner Approval Required)

The Webstore may override the customer-facing retail price, but must still preserve the internal Pricing Foundation calculation and frozen snapshot for margin reporting and production.

Product category mapping: Every Webstore product must be mapped to a Pricing Foundation category (Banners, Apparel, etc.) at creation time. This mapping is what allows the product to have a real, calculated internal cost — without it, the product cannot participate in profitability reporting at all.

Base calculated internal price: Computed by the shared **PricingEngine** from the product's category mapping + configured attributes (dimensions, material, quantity assumptions, etc.), exactly as any other Order Item would be priced.

Public visible selling price: Defaults to the base calculated internal price. May be overridden per §"Store-specific retail override" below.

Store-specific retail override vs. product-specific override: Two override levels are supported —

- **Store-specific:** a blanket adjustment (e.g., "all products in this fundraiser store are marked up an additional 20% to fund the fundraiser") applied at the storefront level.
- **Product-specific:** an individual product's own override price, taking precedence over the store-level adjustment if both are set. Both are purely display/selling-price adjustments; neither touches the internal calculated cost or snapshot.

Owner share / platform fee: Computed from the **internal calculated price** (or the actual transaction amount charged to the customer, per whichever existing revenue-share model is already documented in the Webstores/Business Finance domain) — not from a number that could be manipulated independently of real cost data. This preserves accurate owner-share/platform-fee accounting even when a public override is in effect.

Discount/promo behavior: Any future promo-code or storefront-wide discount (Phase 7 future scope, PRC-055) applies to the public-facing override price, following the same "never touches the internal calculation" principle.

Fundraiser and event-store behavior: Fundraiser/event stores commonly need an artificially elevated public price (to fund a cause) — this is exactly the store-specific override use case above; the internal calculated cost/margin remains accurate underneath, so the shop can still see its true production cost separately from the fundraiser markup.

Whether Webstore products can remain manually priced: Yes, for products that genuinely don't fit any Pricing Foundation category (e.g., a true one-off consignment item) — these map to the **Custom** category (the platform's existing, deliberate manual-entry escape hatch), not to a second parallel pricing system. This satisfies "Webstore products that must remain manually priced" without reintroducing a disconnected pricing mechanism (PRC-002's core violation).

Which products should be calculator-backed versus manually priced (P0-PRC-030, Owner Approval Required):

Recommended Decision: Any product that corresponds to a real production category (signs, banners, apparel, vehicle graphics, etc.) should be calculator-backed by default; only genuinely uncategorizable items (drop-shipped third-party goods, consignment items, gift cards) should use the Custom-category manual path. This is a per-product decision made by the Webstore Owner/Manager at product-creation time, not a platform-wide restriction.

Snapshot rules when an order is placed: A Webstore checkout creates a **PricingSnapshot** for each order line exactly like any other Order Item — capturing the internal calculated price, any override applied, and full cost/margin data — even though the customer only ever saw the public override price.

Reporting rules: Webstore order profitability reports read the internal calculated cost/margin from the snapshot, never the public override price, for true profitability numbers; the override price is shown separately as "what the customer paid" for revenue reconciliation.

Cost/margin visibility rules: Customers never see cost/margin (consistent with §5/§6's visibility rules). Webstore Manager/Owner see their store's revenue and (if permitted, §6) their own margin; full platform-fee/owner-share internals remain Owner/Platform-Admin-only per P0-PRC-023.

Webstore Owner vs. Webstore Manager permissions: Both may set the public-facing override price (store- or product-level); neither may edit the underlying Pricing Foundation category configuration, materials, or labor rates — that remains `PRICING_EDIT_FOUNDATION`-gated at the tenant level (§6).

10. VEHICLE WRAP CANONICAL FORMULA DECISION

This section resolves the confirmed duplicate-engine issue between the Pricing Foundation's Vehicle Graphics/Wraps category and Wrap Lab / Wrap Command Center (PRC-001, PRC-032).

Current Foundation formula summary (Confirmed, per `pricing_spec.md` §Category 5 and `PRICING_SPEC.md` §5)

```
base_sqft      = vehicle_type_base_sqft table (Sedan 150 → Semi 800)
coverage_pct   = spot 15% / partial 40% / half 55% / full 100% / custom user-
input
wrap_sqft      = base_sqft × coverage_pct [or manual override]
waste_pct      = 10-15% depending on coverage
material_cost  = waste_sqft × wrap_material_cost_per_sqft
laminate_cost  = waste_sqft × laminate_cost_per_sqft [if required]
window_perf    = flat_sqft allowance × sell rate [if included]
design_hours    = coverage-based table (0.75-3.0 hr) × complexity multiplier
install_hours  = vehicle+coverage lookup table (e.g., Cargo Van full = 18hr) ×
difficulty × seam multipliers
install_cost   = max(install_hours × $75/hr, $125) + second-installer add-on
production_hours = wrap_sqft × 0.12hr/sqft (min 1.0hr)
overhead       = 15% (basis per §3's pending decision)
markup         = 2.4×
cost_plus_price = total_cost × 2.4
package_price  = benchmark_table[vehicle][coverage] (Sedan spot $150 → Semi
full $6,000)
selling_price  = max(cost_plus_price, package_price, $150.00) +
window_perf_sell + rush(+30%)
```

Current Wrap Lab / Wrap Command Center formula summary (Confirmed structurally distinct, not line-by-line diffed)

`routes/wrap/core.py::_compute_pricing_snapshot` uses its own `WrapPricingConfig` model with its own materials array and its own 3-value `pricing_method` enum (`per_sqft` | `material_labor_markup` | `manual`) — confirmed independent of the Foundation's method system, confirmed to exist and be actively used by the customer-facing wrap quote-approval flow (the only category with a working customer-portal approve/decline mechanism, per the New Order Workflow

investigation).

Known differences

- Different, narrower pricing-method enum (3 values vs. the Foundation's multi-arm compare-methods system).
- A separate materials array, meaning the same physical wrap material could have two different cost/sell-rate entries depending on which system priced it.
- No confirmed package-benchmark floor comparison in the Wrap Lab implementation (not verified either way).

Known unknowns requiring side-by-side code comparison (Confirmed gap, not guessed)

- Exact vehicle-type/base-sqft table used by Wrap Lab — same table, a different one, or user-entered dimensions only.
- Exact install-hour calculation — same lookup table or a different formula.
- Whether Wrap Lab applies overhead/markup at all, and if so, what percentage/multiplier.
- Whether Wrap Lab's `per_sqft` method has any relationship to the Foundation's sell-rate-per-sqft comparison arm.
- The magnitude of price disagreement between the two systems for the same real job — **not yet quantified** (this is PRC-032, and per the Remediation Plan, must happen before this decision is finalized).

Options

Option	Description
(a) Foundation formula as canonical	Wrap Lab adopts the Foundation's existing compare-methods engine wholesale
(b) Wrap Lab formula as canonical	Foundation's Vehicle Graphics category adopts Wrap Lab's formula instead
(c) Merged canonical formula	A new formula combining the Foundation's more sophisticated compare-methods/benchmark structure with any genuinely valuable Wrap Lab-specific inputs not present in the Foundation

Recommended decision

P0-PRC-031 — Owner Approval Required, direction only; final formula selection deferred pending code diff.

Recommend **(c), a merged canonical formula**, favoring the Foundation's structure (compare-methods + package-benchmark floor + vehicle/install-hour lookup tables — all more sophisticated and better-evidenced than anything confirmed about Wrap Lab's implementation) while explicitly preserving any Wrap Lab-specific input fields that capture real wrap-industry nuance not currently in the Foundation (e.g., if Wrap Lab captures a distinct "vinyl overlap/seam count" input the Foundation lacks, that input should be added to the merged formula rather than discarded).

This decision cannot be fully finalized until the required side-by-side code comparison below is completed — approving "merge, favoring Foundation's structure" as a *direction* now does not remove the need for that verification task before Phase 4 implementation begins.

Required formula components (for the merged canonical formula)

- Vehicle base-sqft table (Foundation's, unless Wrap Lab's is shown to be more current/accurate)
- Coverage-percentage table (spot/partial/half/full/custom)
- Waste-percentage-by-coverage table
- Material + laminate cost-per-sqft, drawn from **one** shared materials source (not two arrays)

- Window-perf sqft allowances and sell rates
- Design-hours-by-coverage table × complexity multiplier
- Install-hours-by-vehicle-by-coverage lookup table × difficulty × seam multipliers
- Second-installer add-on rule
- Package-benchmark floor table (vehicle × coverage → minimum acceptable price)
- Overhead/markup, per §3's locked decision
- Rush premium (+30%, category-specific override of the global rush default)

Required vehicle tables, coverage rules, waste rules, material/laminate rules, install-time lookup, second-installer behavior, package benchmark floor behavior, design/prep/removal behavior

All of the above are already fully specified in the Foundation's documented formula (this section's "Current Foundation formula summary") and should be adopted into the merged formula as the default source of truth, **unless** the side-by-side code comparison reveals Wrap Lab uses materially different, more currently-accurate numbers for any of them (e.g., if Wrap Lab's install-hour table has been tuned more recently based on real completed jobs) — in which case the more evidence-backed table wins, not automatically the Foundation's.

Required snapshot behavior

Identical `PricingSnapshot` structure and commit behavior as every other category, with no Wrap-Lab-specific snapshot shape surviving the merge — this is the direct fix for PRC-001's core violation.

Required parity test plan

Per PRC-039 (already scoped in the Remediation Plan): run identical vehicle+coverage+material inputs through both the pre-merge Foundation formula and the pre-merge Wrap Lab formula, document the delta for at least 5 representative scenarios (spot/partial/half/full/custom coverage, at least 2 different vehicle sizes), and use that documented delta to validate the merged formula produces defensible results before cutover. The merged formula's output must then be re-tested against both legacy systems' outputs to confirm it sits within a reasonable, explainable range of each (not necessarily identical to either, since it may correct real defects in both).

Retirement plan for the duplicate Wrap Lab pricing engine

`_compute_pricing_snapshot` and `WrapPricingConfig` are deleted (not merely deprecated-in-place) once the merged formula is live and the customer-facing wrap quote-approval flow is confirmed working end-to-end against the new shared engine, per the shadow-mode/feature-flag rollout plan already specified in the Remediation Plan's Phase 4 detail (rollback/protection plan).

11. BANNER SETUP WIZARD VS. PRICING SETUP QUIZ DECISION

Confirmed current state: `BannerSetupWizard.js` (7 steps) writes directly to `category_defaults.banners`. The 11-section Pricing Setup Quiz's Section 2 (Banners) also writes to `category_defaults.banners`. Whether these two flows write to the *same* field subset or have drifted apart was **not verified line-by-line** in either investigation session (confirmed open question, not guessed). `ShopRateQuiz.js` presents a similar, separate unconfirmed-overlap question focused on labor/shop rates specifically.

Are these separate necessary experiences, overlapping experiences, or duplicate screens?

Assumption to Verify. The evidence available (both write to `category_defaults.banners`; both are described as "guided

setup" for the same category) strongly suggests overlapping-to-duplicate, but this cannot be stated as Confirmed without a direct field-by-field code diff of `BannerSetupWizard.js` against the Quiz's Section 2 fields.

What still needs verification

A side-by-side diff of every field `BannerSetupWizard.js` writes versus every field the Quiz's Banner section (§4B of the rebuild investigation) writes, to determine: (a) identical field sets — clear duplication, retire one; (b) overlapping-but-not-identical field sets — one is a subset, consider merging the extra fields into the unified Quiz before retiring the wizard; (c) genuinely different field sets serving different purposes — keep both, but rename/reposition them so users understand they're not redundant. The same diff exercise applies to `ShopRateQuiz.js` against the main Quiz's Section 1 (Shop Basics) and Section 11 (Labor & Design Time).

Recommended consolidation approach

P0-PRC-032 — Owner Approval Required. Default recommendation: **use one unified Pricing Setup Quiz with optional category-specific guided sections, rather than two competing configuration flows that may write to the same Banner settings.** Concretely: if the verification above confirms overlap/duplication (the likely outcome given the evidence), `BannerSetupWizard.js`'s distinct value — a more focused, category-specific guided flow — should be preserved as an **optional guided sub-flow accessible from within the unified Quiz's Banner section** (e.g., a "walk me through this step by step" expansion inside Section 2), rather than existing as a second, separately-launched, separately-navigable screen. The same treatment applies to `ShopRateQuiz.js` relative to the main Quiz's shop-rate sections.

User experience impact

A shop owner currently encountering both a "Run Pricing Setup Quiz" button and a separate Banner-specific wizard (if that wizard is independently discoverable/launchable) has no way to know which one is "the real one" or whether running both is necessary, redundant, or conflicting. Consolidating into one entry point with an optional detailed sub-flow removes this ambiguity entirely.

Data impact

If the field-diff confirms full overlap, no data model change is needed — both flows already write to the same `category_defaults.banners` target; only the UI entry points consolidate. If the diff reveals `BannerSetupWizard.js` captures fields the Quiz's Section 2 does not, those fields must be added to the unified Quiz's Banner section before the wizard is retired, to avoid a silent capability loss.

Migration/retirement impact

Per the Remediation Plan's Phase 10 rollback plan: keep the legacy wizard's code present but unlinked from navigation for one release cycle before deleting outright, in case the consolidation decision needs revisiting after real shop-owner usage data is observed.

Decision criteria

Approve retirement/consolidation only after: (1) the field-diff verification task is complete, (2) any Wizard-only fields have been added to the unified Quiz, (3) the unified Quiz's Banner section has been confirmed to produce equivalent or better suggested defaults than the standalone wizard for a representative test shop's real answers.

Required code investigation before final approval

A direct read of `BannerSetupWizard.js` and `ShopRateQuiz.js` alongside the Quiz's corresponding sections, cataloging every field each writes — this is a discrete, bounded verification task that should be scheduled early in Phase 10, not left as a standing ambiguity.

12. SHIPPING, DELIVERY, INSTALLATION, AND SERVICE CALL PRICING OWNERSHIP

Confirmed current state: Only Installation has a real cost/labor formula today (complexity-tiered hours × rate, present in every area-based category and Vehicle Wraps). Shipping/delivery is a display-only free-text notes field with zero cost calculation anywhere (PRC-018, confirmed absent). Services' travel/trip/equipment/subcontract/permit fields are the most fully-modeled example of this general category of cost (§Services formula).

Recommended principle (P0-PRC-033, Owner Approval Required)

Pricing Foundation owns the price calculation inputs and snapshot values. Operations/Fulfillment owns scheduling, completion, actual labor, mileage, carrier tracking, and production execution.

Cost Component	Pricing Input?	Fulfillment Record?	Both?	Who Enters It	Categories	Markup-able?	Tax
Delivery labor	Yes	Yes	Both	Staff at quote time (estimate); Fulfillment records actual	All categories (optional)	Yes	Per rule yes
Mileage	Yes	Yes	Both	Staff (estimate); Fulfillment (actual via odometer/GPS if tracked)	All categories (optional)	Yes (small markup typical)	Usu (pa
Travel time	Yes	Yes	Both	Same as mileage	Services primarily, others optionally	Yes (billed as labor)	Yes
Vehicle usage (shop truck)	Yes (as a flat/per-mile rate)	No (not separately tracked as fulfillment)	Pricing only	Staff at quote time	All categories (optional)	Yes	Usu
Fuel surcharge	Yes	No	Pricing only	Configured rate, applied automatically	All categories (optional)	No (pass-through)	Dep ten
Shipping carrier cost	Yes	Yes	Both	Staff enters carrier quote at pricing time; Fulfillment tracks actual shipment/tracking number	All categories (optional)	Optional small markup	Dep ten
Packaging	Yes	No	Pricing only (a cost bucket)	Configured per-category default or manual entry	All categories (optional)	Yes	Yes
Installation labor	Yes (already exists)	Yes (already exists)	Both — confirmed already correctly split today	Staff (estimate via complexity tiers); Production/Fulfillment (actual time logged)	Most categories, Vehicle Wraps	Yes	Yes
Equipment rental	Yes	Yes	Both	Staff (estimate); Fulfillment (actual booking/return)	Services primarily	Optional	Yes
Service call / trip charge	Yes (already exists for Services)	Yes	Both	Staff	Services	No (typically a flat fee)	Yes

Subcontractor installation	Yes (already exists)	Yes	Both	Staff (estimate/quote from sub); Fulfillment (actual sub invoice/completion)	Services, any category using outsourced install	Optional (already toggleable)	Dep
Permit/municipal fees	Yes (already exists as pass-through)	Yes (tracking permit status/approval)	Both	Staff (enters the fee amount)	Vehicle Wraps (sign permits), Services	No (confirmed pass-through, never marked up)	Dep juri usu

General rule applied consistently across the table above: Every cost component that affects what the customer is charged is a **pricing input** (it must produce a cost/price figure that lands in the **PricingSnapshot**). Any component whose *actual, real-world execution* needs to be tracked over time (was the truck driven, did the shipment arrive, was the permit approved, did the installer actually show up) is *also* a **fulfillment record**, owned by Operations/Production, and the two are linked by the shared Order Item / Work Order Summary — but the fulfillment record never recalculates the frozen pricing snapshot; it only tracks what actually happened against what was estimated (a common and expected variance report, not a live price change).

13. OVERTIME, SERVICE RATE, AND AFTER-HOURS PRICING RULES

Confirmed current state: No overtime rate concept exists anywhere in **models/pricing.py** (PRC-019, confirmed absent by full grep). Rush surcharge (17.5-30% depending on category) is the only mechanism currently reflecting "this needs to happen faster/harder than normal."

Recommended launch rules (simple, but not fake-simple)

P0-PRC-036 — Overtime labor policy, Owner Approval Required. Add one optional overtime hourly-rate field per labor role (production, install, design), configured as either a flat replacement rate or a multiplier on the base rate (**Recommended Decision:** a multiplier, e.g., **1.5×** the base rate, mirroring standard overtime-pay conventions and requiring less duplicate rate-entry than a second flat rate per role). Overtime is **manually selected** by Staff when scheduling/pricing a job known to require overtime labor — **not automatically inferred** from job size or rush status, since the system has no visibility into actual shop-floor scheduling/capacity at pricing time (confirmed no such capacity-awareness exists anywhere in the platform).

P0-PRC-037 — Weekend/after-hours/emergency labor policy, Owner Approval Required. Recommended: a second, separate optional multiplier (distinct from overtime, since a weekend job isn't necessarily an overtime-hours job) — e.g., **weekend_multiplier: 1.25×**, **emergency_after_hours_multiplier: 2.0×** — each independently toggleable per job, each stacking multiplicatively with the base rate (not with each other by default; **Recommended Decision:** if a job is both overtime AND after-hours, apply only the higher of the two multipliers, not both compounded, to avoid an unreasonably inflated rate that could trigger unnecessary below-cost/margin confusion in the opposite direction — an "above normal margin" case that, while not a financial risk, could still surprise a customer with an unexpectedly high price).

Whether overtime affects cost, selling price, or both: Both — overtime/after-hours labor is a genuinely higher real cost to the shop (the employee is paid more), so it must flow through as an elevated **labor_cost** bucket, which then naturally produces a higher selling price through the normal markup/overhead math — it is not a separate surcharge

bolted onto an otherwise-normal price the way Rush is.

Whether overtime can be automatic or manually selected: Manually selected only, per P0-PRC-036 above — no automatic inference.

Whether customers see overtime as a line item: Recommended Decision, Owner Approval Required: Yes, shown as a distinct labor-cost line (e.g., "After-Hours Labor Premium") on the customer-facing Quote/Invoice, for transparency — unlike internal cost/margin data, this is a real cost driver the customer caused by requesting after-hours work, and shops typically want customers to see and understand why the price is higher in this specific case (distinct from the Rush fee, which is already customer-visible today).

Service-call minimums (P0-PRC-038, Owner Approval Required): Already partially modeled for Services (`services_minimum_applies/services_minimum_override`) — recommend generalizing this exact pattern (a configurable minimum charge that applies when `services_trip_charge_applies` or an equivalent "this required a dedicated visit" flag is set) to any category that can trigger a standalone service visit (e.g., a wrap-removal-only job, an install-only job with no new production).

Travel-time billing: Already modeled for Services (`services_travel_required/services_travel_miles` × mileage rate) — recommend this becomes the shared, reusable travel-cost component referenced in §12's table, available to any category, not Services-exclusive.

Second installer rules: Already modeled for Vehicle Wraps (`second_installer_required` × \$35/hr) — recommend generalizing to any category with an install component, using the same rate-plus-hours pattern.

Subcontractor markup: Already modeled for Services (`services_subcontract_markup_applies`, default Yes) — recommend this remains category-configurable (some shops may want zero markup on subs they merely coordinate, others want their normal markup) rather than a hardcoded platform rule.

Permit/pass-through rules: Already confirmed correct and consistent — permit/external fees are pass-through, never marked up, anywhere they appear. No change recommended.

How all of this appears in snapshots and reports: Overtime/after-hours/travel/second-installer/subcontract costs each get their own named bucket in the `PricingSnapshot` (extending the existing breakdown-array pattern), so reporting can answer "how much of our labor cost last quarter was overtime premium" as a distinct, queryable figure rather than being buried inside an undifferentiated `labor_cost` total.

14. PRICING CONFIGURATION CHANGE HISTORY AND SNAPSHOT GOVERNANCE

Confirmed current state: No `PricingChangeLog` exists; Pricing Foundation edits are not confirmed to write to any activity/audit log at all (PRC-012). No pricing config version history exists — only the current live value of any field is ever stored (PRC-027, confirmed absent per source spec §16.9). `JobTicket.pricing_snapshot` correctly freezes calculation output at commit time (confirmed correct, existing behavior).

The five distinct concepts and how they differ

Concept	What It Is	Mutable?	Where It Lives
Live Pricing Foundation configuration	The current, editable rates/materials/markup/category rules a tenant is using right now	Yes, freely editable by <code>PRICING_EDIT_FOUNDATION</code> holders	<code>PricingFoundationSettings</code> , <code>PricingMaterial</code> , <code>CategoryPricingConfig</code>

Historical pricing configuration	What the live configuration <i>used to be</i> at some past point in time	No — append-only history	Reconstructed from PricingChangeLog entries, or a full versioned snapshot of the config document (§P0-PRC-039/040)
Frozen Pricing Snapshot	The exact calculation result committed to one specific Quote Item / Order Item / Wrap Project / Webstore Order Item	Never — immutable once created	PricingSnapshot , attached to its specific commercial record
Manual override provenance	The record of who/when/why a specific snapshot's price was manually adjusted	Never, once recorded	Embedded inside the relevant PricingSnapshot.override block
Pricing Change Log	The append-only audit trail of every edit to <i>live configuration</i> (not to any specific snapshot)	Never (append-only)	PricingChangeLog collection
Activity Log	The tenant-wide, cross-domain feed of "who did what" (orders, quotes, tasks, pricing, etc.)	Never (append-only)	Existing platform-wide Activity Log system

Which actions write to **PricingChangeLog**, **Activity Log**, or both

P0-PRC-039 — Owner Approval Required.

Action	PricingChangeLog	Activity Log	Both?
Editing a material's cost/sell rate	Yes	No (too granular/frequent for the general feed)	ChangeLog only
Editing a labor rate, markup, overhead %, or category rule	Yes	No	ChangeLog only
Creating/editing a Pricing Template	Yes	No	ChangeLog only
Accepting a Historical Import benchmark suggestion	Yes	Yes (a meaningful, infrequent, owner-initiated event worth surfacing in the general feed)	Both
Completing the Pricing Setup Quiz and applying suggestions	Yes (one entry per changed field, or one batch entry referencing the quiz session)	Yes (one summary entry: "Pricing Setup Quiz completed, N defaults updated")	Both
A Staff-requested below-cost override	No (this isn't a config change)	Yes (on the relevant Order/Quote's activity feed)	Activity Log only, plus provenance inside the Snapshot itself
An Owner/Admin approving or rejecting a below-cost override	No	Yes	Activity Log only, plus provenance inside the Snapshot itself
Setting/changing a customer's discount percent	Yes	No	ChangeLog only
Setting a Webstore public-price override	Yes (it's a config-like change to that product's pricing)	No, unless the shop wants storefront price changes surfaced (Owner Approval Required as an optional tenant setting)	ChangeLog, optionally both

Whether old material costs can be edited

Yes — the *live* material cost can always be edited going forward (that's the point of live configuration); what cannot happen is any retroactive change to a **PricingSnapshot** that already used the old cost — editing today's material cost has zero effect on any quote/order/snapshot created before the edit.

Whether historical snapshots ever recalculate

P0-PRC-040 — No, never — Owner Approval Required (reaffirming existing correct behavior). A PricingSnapshot is permanently frozen at the moment it's created. No future change to live configuration, materials, labor rates, or formulas ever retroactively alters an existing snapshot's stored values. This is already correctly implemented today for JobTicket.pricing_snapshot and must remain true for every future pricing surface (Wrap Lab, Webstores) once unified.

How configuration versions are identified

Recommended Decision: Each PricingSnapshot stores a config_version_used reference (a timestamp or an incrementing version identifier for that tenant's Foundation configuration at the moment of calculation) — this, combined with the PricingChangeLog's append-only history, allows any report to reconstruct exactly what configuration state produced any given historical price, without needing to store a full duplicate copy of the entire configuration inside every single snapshot (which would be wasteful) — only the cost/price results are duplicated into the snapshot; the configuration that produced them is reconstructable from the versioned log.

How a report can explain why an old Order Item used a different cost/rate than a current quote

By comparing the old Order Item's PricingSnapshot (which shows the exact material cost, labor rate, markup, etc. used at that time) against the current live configuration, or against the PricingChangeLog's history of changes between the two dates — the report can produce a plain-English explanation such as "this quote used a \$1.10/sqft banner material rate; the current rate is \$1.25/sqft, changed on [date] by [user]."

How imported historical invoice benchmarks are tracked

Via the existing PricingImport model (confirmed already correctly separate and status-tracked: mapping_required → ready_for_analysis → analyzed → reviewed) — each accepted suggestion additionally writes a PricingChangeLog entry per the table above, closing PRC-054's confirmed gap.

How quiz-derived defaults are tracked

Per the Remediation Plan's PRC-025 fix: raw Quiz answers are persisted in their own collection, independent of the derived config values they produced, so a Foundation field's current value can always be traced back to "this was set by your Quiz answer on [date]" even if it was later manually edited — with the manual edit itself separately visible in the PricingChangeLog.

How manual edits after quiz completion are tracked

Exactly like any other Foundation edit — a PricingChangeLog entry, with the difference between the Quiz-derived value and the current live value visible by comparing the Quiz-answer-history record (immutable) against the ChangeLog's most recent entry for that field (which shows the manual edit that superseded it).

15. PHASE 0 FORMULA GOVERNANCE RULES

P0-PRC-041 — Owner Approval Required. The following rulebook is binding for every calculator built from Phase 3 onward. Rules are classified as **Universal** (every category must follow, no exceptions), **Category-Specific** (applies only to the categories named), **Optional Feature** (applies only if the tenant/category enables the feature), or **Future Feature** (not built yet, rule reserved for when it is).

Rule Area	Classification	Rule
-----------	----------------	------

Minimum billable area	Category-Specific	Each area-based category (Banners, Rigid Signs, Cut Vinyl, Digital Print, Vehicle Graphics) defines its own minimum billable area floor (confirmed range 0.5-4.0 sqft); the pricing engine always applies <code>billable_area = max(actual_area, category_minimum)</code> before any cost calculation.
Waste calculation order	Universal (for categories with a waste concept)	Apparel/Services/Promotional/Cut Vinyl have no area concept and this rule does not apply to them. Waste percentage is applied to <code>billable_area</code> immediately after the minimum billable area is confirmed, always <i>before</i> material cost is calculated — never applied after material cost is calculated twice.
Material cost calculation	Universal (for categories with a material concept)	<code>material_cost = waste_adjusted_area × material_cost_per_sqft</code> (or the unit-cost for apparel blanks/promotional unit costs) — always computed from the <i>waste-adjusted</i> area, never the raw entered quantity.
Labor calculation	Universal	Minute-based only (PRC-006's hours-fallback is retired) — every category computes labor in minutes internally, even if the UI displays hours to the user.
Design labor	Universal, pending P0-PRC-002	Computed as <code>design_hours × design_rate</code> ; whether it sits inside or outside the bucket is governed by §3's locked decision, but the calculation of the design-cost bucket is consistent everywhere it applies.
Installation labor	Universal, pending P0-PRC-003	Computed as <code>max(install_minimum_charge, install_hours × install_rate)</code> (+ shipping add-on if applicable, +overtime/after-hours multiplier if applicable per §13); inclusion governed by §3.
Overhead	Universal	Computed exactly once per calculation, using the canonical basis locked in §4, always on a full <code>overhead_basis</code> explainability object.
Markup	Universal	A single configured multiplier per category, applied to the fully-assembled cost; never applied twice, never applied to a sub-component in isolation and then to the whole.
Sell-rate comparison	Category-Specific	Applies only to the "Compare Methods" family (Banners, Rigid Signs, Cut Vinyl, Vehicle Graphics): the formula always computes both the cost-plus arm and the sell-rate-per-unit arm and takes the higher, per the existing correct <code>max(cost_plus, sell_rate_per_unit)</code> .
Package benchmark floors	Category-Specific	Applies only to Vehicle Graphics/Wraps (and any future category that adopts the benchmark floor): the benchmark price is a third comparison arm, always computed as <code>max(cost_plus, sell_rate_per_unit, benchmark_floor)</code> , never subtracted or blended.
Quantity discounts	Universal (where configured)	Applied to <code>sell_base</code> before design/install/setup/shipping fees are added, per the configured behavior — this order is preserved, not changed.
Customer discounts	Universal (once built, Phase 7)	Applied after quantity discounts, before rush fees, per §7/§8's locked precedence; never applied to pass-through fees, tax, or deposits.
Rush fees	Universal	Applied after quantity discount and customer discount, before tax — a straight surcharge on the already-discounted total, never applied to pass-through fees.
Manual overrides	Universal	Replaces the entire computed result of every step before it (per §8's precedence); never applied to the below-cost/below-margin guardrails in §5/§6 regardless of category.
Taxes	Universal	Computed by the Invoice/Order domain on the final pricing-engine output, never manually or second-guessed by the pricing engine itself.
Deposits	Universal	A payment-schedule concept, not a pricing concept — the pricing engine produces the price; how that price is split into deposit + balance is an Order/Invoice-domain decision, out of scope for the <code>PricingEngine</code> itself.
Rounding	Universal	Per §4's locked rule: internal precision maintained until each named cost bucket is computed, then rounded once to the nearest cent (half-up), with whole-dollar rounding as a category/tenant setting only.
Snapshot creation	Universal	Every committed price (Quote Item, Order Item, Wrap Project, Webstore Order) contains exactly one <code>PricingSnapshot</code> , containing the full cost breakdown, warnings, and provenance (if any) — never a partial snapshot, never a snapshot missing the full cost breakdown.
Warning creation	Universal	Every calculation evaluates the below-cost/below-margin/staleness/fallback conditions and attaches any applicable typed <code>PricingWarning</code> objects — warnings are never dropped even if the calculation ultimately succeeds.
Permission enforcement	Universal	Every pricing-domain write (Foundation edit, override, customer discount, Vehicle Graphics edit) checks the relevant <code>Permission.PRICING_*</code> grant from §6 before proceeding — no path is ever permission-exempt.
Audit logging	Universal	Every pricing-domain write produces the appropriate <code>PricingChangeLog</code> and/or entry per §14's routing table — no silent configuration change is permitted.
Reporting data output	Universal	Every <code>PricingSnapshot</code> exposes its cost/margin/override/discount data in a structured format, to be consumed without special-casing per category — the standardized <code>PricingResultSet</code> schema (Remediation Plan §4A) is the single contract every report reads against.
Shipping/delivery	Optional Feature	Once built (§12), available per category as an optional cost component, categories that enable it simply omit the bucket, they are not forced to carry a zero-value placeholder.

Overtime/after-hours	Optional Feature	Per §13, manually toggled per job; never automatically inferred.
Wholesale/contract/volume pricing	Future Feature	Reserved for Phase 7 expansion beyond the customer-discount MVP; no callout built assuming these exist yet.
AI pricing suggestions	Future Feature	Reserved for Phase 11; no calculator should call out to or depend on an AI suggestion.

16. DECISION DEPENDENCIES AND BUILD GATES

Build Gate	Required Phase 0 Decisions	Why It Cannot Begin Without Them
Shared pricing infrastructure (Sprint 1 / Phase 1)	P0-PRC-008, P0-PRC-009, P0-PRC-013, P0-PRC-014, P0-PRC-015, P0-PRC-016, P0-PRC-019, P0-PRC-039, P0-PRC-040	The Money utility needs the currency/rounding rule locked; the PricingWarning /override-provenance schema needs the guardrail and approval-workflow rules locked; the permission enforcement helper needs the role mapping locked; the PricingChangeLog write-through helper needs the Money utility locked.
Materials migration (Phase 2)	P0-PRC-005, P0-PRC-006	When the setup rules are locked, building a factory of the base overhead basis is resolved, what then needs building is a Material collection. Material records are elsewhere; migrating the Materials collection before this is settled risks a second migration later.
Banner calculator port (Phase 3, first category)	P0-PRC-001 through P0-PRC-007 (overhead), P0-PRC-009 through P0-PRC-012 (rounding), P0-PRC-010, P0-PRC-011, P0-PRC-021 (override workflow), P0-PRC-028 (precedence), P0-PRC-033/034 (shipping/install ownership), P0-PRC-036/037 (overtime), P0-PRC-041 (governance rulebook)	Banners is the proof-of-concept for every other category — if its overhead basis, rounding, and override behavior are wrong, every category ported afterward inherits the same error.
Area-based engine port (Rigid Signs, Cut Vinyl, Digital Print)	Same as Banner port, already proven	These categories reuse the Banner-proven engine; no <i>new</i> Phase 0 decisions are required once Banners is done correctly, only continued adherence to the same locked rules.
Vehicle Wrap unification (Phase 4)	P0-PRC-031 (canonical formula direction), P0-PRC-038 (second-installer/travel rules), plus all Phase 3 gating decisions	The merge cannot proceed without a locked direction on which formula wins, and the side-by-side code comparison this decision depends on.
Apparel module (Phase 5)	P0-PRC-005 (setup fees), P0-PRC-007 (whether Apparel applies overhead at all — currently an open verification item)	Apparel's formula behaves differently enough from the area-based family that its overhead treatment must be explicitly confirmed, not assumed to match Banners.
Customer pricing MVP (Phase 7)	P0-PRC-024, P0-PRC-025, P0-PRC-026, P0-PRC-027, P0-PRC-028 (precedence)	The discount mechanism cannot be built without knowing exactly where in the precedence chain it sits and how it interacts with the below-cost guardrail.
Webstore pricing integration (Phase 8)	P0-PRC-022, P0-PRC-023, P0-PRC-029, P0-PRC-030	The entire point of this phase — preserving internal calculation while allowing a public override — is explicitly defined by these decisions; building it without them risks recreating a version of the exact disconnected-pricing defect (PRC-002) the rebuild exists to fix.
Reporting repair (Phase 9)	P0-PRC-001 through P0-PRC-007 (overhead consistency), P0-PRC-039, P0-PRC-040	Cross-category margin reporting is meaningless until every category computes overhead the same way; snapshot-based reporting requires the frozen-snapshot governance rules to be locked.
Historical import improvements (Phase 10)	P0-PRC-012, P0-PRC-032, P0-PRC-039	Audit-logging improvements depend on the change-log routing rules; the consolidation of onboarding flows depends on the Banner Wizard/Quiz decision.
AI Pricing Guidance (Phase 11)	All prior decisions, transitively	AI suggestions are only meaningful once the entire rule set they're suggesting against is stable and correct — this phase has no decisions of its own in this document by design (per the Remediation Plan, it is deliberately last).

17. PHASE 0 APPROVAL CHECKLIST

Decision ID	Decision Title	Recommended Choice	Approve / Revise / Defer	Notes
P0-PRC-002	Design labor in overhead basis?	Exclude (pass-through after markup)		P0-PRC-003 Installation labor in overhead basis? Exclude (diverges from literal default text — see §3)
P0-PRC-004	Subcontractor cost in overhead basis? Exclude		P0-PRC-005 Setup fees in overhead basis? Exclude	
P0-PRC-006	Shipping/delivery in overhead basis? Exclude (pass-through)		P0-PRC-007	Does Apparel/Services/Promotional/Custom apply overhead? Promotional: yes. Apparel/Services: no (verify first). Custom: N/A
Requires code verification for Apparel		P0-PRC-008 Store money as integer cents	Approve	
P0-PRC-009	Rounding method and timing Half-up, round once per finalized cost bucket		P0-PRC-010	Whole-dollar pricing toggle Optional, category/tenant-level opt-in only
P0-PRC-011	Rounding for quantity discount/rush/tax Round once per named bucket, as described in §4		P0-PRC-012	Rounding for manual overrides/imports Direct entry = exact; imports = half-up with confidence flagging
P0-PRC-013	Below-target-margin behavior Warning only		P0-PRC-014	Below-cost behavior Warning + required reason
P0-PRC-015	Below-cost override approval Owner/Admin approval required for Staff-requested overrides		P0-PRC-016	Override reason taxonomy 6-value taxonomy + custom "other"
P0-PRC-017	Warning visibility by surface Per §5 table (internal yes, customer-facing no)		P0-PRC-018	Warnings on discount/wholesale/webstore-driven below-cost Same guardrails as direct override
P0-PRC-019	PRICING_VIEW/EDIT_FOUNDATION/OVERRIDE role mapping Per §6 table		P0-PRC-020	Cost/margin visibility scope Owner/Admin only by default, tenant-configurable for Staff
P0-PRC-021	Override approval workflow steps Per §6's 8-step workflow		P0-PRC-022	Webstore public pricing edit permission Webstore Owner/Manager, override field only
P0-PRC-023	Owner-share/platform-fee visibility Owner/Platform Admin only		P0-PRC-024	Customer discount MVP scope Flat % only, positioned per §7/§8
P0-PRC-025	Customer discount below-cost guardrail Same guardrail as manual override		P0-PRC-026	Customer-specific minimum-margin exceptions Defer to Phase 7 future expansion
P0-PRC-027	Tax-exempt separation reaffirmed No change to existing correct behavior		P0-PRC-028	Canonical 14-layer precedence hierarchy Per §8 table
P0-PRC-029	Webstore override never erases internal snapshot Approve as binding rule		P0-PRC-030	Calculator-backed vs. manually-priced Webstore products Category-mappable = calculator-backed; true one-offs = Custom category
P0-PRC-031	Vehicle Wrap canonical formula direction Merged formula, favoring Foundation structure, pending code diff		P0-PRC-032	Banner Wizard vs. Quiz consolidation Unify into one Quiz with optional guided sub-flow
Requires field-diff verification		P0-PRC-033	Shipping/delivery pricing ownership Pricing Foundation owns inputs, Fulfillment owns execution	
P0-PRC-034	Installation labor ownership split Both (already correct pattern), extend consistently		P0-PRC-035	Service-call/trip-charge ownership Both, generalize Services' existing pattern
P0-PRC-036	Overtime labor rate policy Optional multiplier per role, manually selected		P0-PRC-037	Weekend/after-hours policy Separate multiplier, higher-of-two rule when combined with overtime
P0-PRC-038	Service-call minimum/travel/second-installer Generalize existing Services/Vehicle-Wraps patterns platform-wide		P0-PRC-039	PricingChangeLog vs. Activity Log routing Per §14 table
P0-PRC-040	Snapshots never recalculate Reaffirm existing correct behavior		P0-PRC-041	Adopt Formula Governance Rulebook (§15) Approve as binding

Nothing above is marked approved automatically. Every checkbox is yours to complete.

18. FINAL RECOMMENDATION

Decisions that should be approved now (no further investigation needed — the evidence already fully supports a clear recommendation): P0-PRC-004 through P0-PRC-006 (subcontractor/setup-fee/shipping exclusion from overhead), P0-PRC-008 through P0-PRC-012 (currency/rounding), P0-PRC-013 through P0-PRC-021 (margin/override/approval guardrails), P0-PRC-024 through P0-PRC-030 (customer/webstore pricing scope and precedence, excluding the deferred future items), P0-PRC-033 through P0-PRC-041 (ownership, overtime, change-history, governance rulebook).

Decisions that require additional code investigation before approval:

- **P0-PRC-001 through P0-PRC-003** — the exact overhead-basis mechanism claimed by `PRICING_SPEC.md` §16.7 for Banners/Vehicle Wraps (which specific cost bucket is "included") should be directly re-verified against the live `server.py` code before the recommendation in §3 is finalized as the literal implementation spec, given the confirmed conflict between the two source documents.
- **P0-PRC-007** — whether Apparel's live implementation actually calls the shared overhead function despite not appearing in the simplified `pricing_spec.md` formula needs a direct code check.
- **P0-PRC-031** — the Vehicle Wrap canonical formula cannot be fully finalized (only its *direction* can be approved now) until the side-by-side Foundation-vs-Wrap-Lab code diff and price-discrepancy quantification (PRC-032) is complete.
- **P0-PRC-032** — the Banner Wizard vs. Quiz consolidation cannot be fully finalized until the field-diff verification task is complete.

Decisions that can safely be deferred:

- P0-PRC-026 (customer-specific minimum-margin exceptions) — explicitly deferred to Phase 7 future expansion, not needed for MVP.
- Any AI Pricing Guidance-related governance — deliberately out of scope for Phase 0 entirely, reserved for Phase 11.

Exact conditions required before Sprint 1 begins

Sprint 1 may begin only when:

- Currency and rounding rules are approved (P0-PRC-008 through P0-PRC-012).
- Overhead formula is approved (P0-PRC-001 through P0-PRC-007, with the code-investigation caveat above resolved or explicitly accepted as-is by the Owner).
- Manual override/approval rules are approved (P0-PRC-013 through P0-PRC-021).
- Pricing precedence hierarchy is approved (P0-PRC-028).
- Vehicle Wrap unification direction is approved (P0-PRC-031, direction only — final formula selection may still be pending).
- Pricing change-history rules are approved (P0-PRC-039, P0-PRC-040).
- Core pricing permissions are approved (P0-PRC-019, P0-PRC-020).

Phase 0 is complete only when the required pricing decisions are approved and recorded. No calculator, pricing screen, Wrap Lab pricing flow, Webstore price flow, quote workflow, or pricing migration may begin before those approvals are complete.

End of Phase 0 Pricing Decisions, Formula Governance & Build Rules. This document does not modify any code, screen, or data. It is the approval gate that must be completed before Sprint 1 (per [/app/memory/PRICING_ISSUES_INVENTORY_AND_REMEDIATION_PLAN.md](#)'s Recommended First Implementation Sprint) may begin.

SignGuy AI — Pricing Spec

Source: Reverse-engineered from production backend (`server.py`) and frontend (`PricingCalculator.js`, `CategoryPricingMethodSetup.js`, `BannerSetupWizard.js`) Purpose: Complete behavioral spec for every pricing category — inputs, calculation flow, outputs, and rebuild notes. Date: 2026-06-10

Table of Contents

- 1. [System Architecture Overview](#)
- 2. [The 8 Pricing Methods](#)
- 3. [Global Shared Concepts](#)
- 4. [Category: Banners](#)
- 5. [Category: Rigid Signs](#)
- 6. [Category: Cut Vinyl](#)
- 7. [Category: Digital Print / Printed Vinyl](#)
- 8. [Category: Vehicle Graphics / Wraps](#)
- 9. [Category: Apparel](#)
- 10. [Category: Services](#)
- 11. [Category: Promotional Items](#)
- 12. [Category: Custom / Other](#)
- 13. [Cross-Category Features](#)
- 14. [Data Storage Schema](#)
- 15. [API Endpoints](#)
- 16. [Rebuild Notes & Warnings](#)

1. System Architecture Overview

Where pricing lives

Layer	Location	Purpose
Calculation engine	<code>server.py</code> (lines 696–4356)	One <code>calculate_*</code> async function per category
Route entry point	<code>routes/pricing.py</code>	<code>POST /api/pricing/calculate</code> dispatches to engine
Tenant configuration	<code>pricing_configuration</code> collection	All per-tenant defaults, materials, rates
Historical import	<code>routes/pricing_setup.py</code>	CSV/PDF invoice import → benchmark extraction
Templates	<code>pricing_templates</code> collection	Saved configurations per category
Frontend calculator	<code>PricingCalculator.js</code>	Main UI (4,045 lines)
Category setup	<code>CategoryPricingMethodSetup.js</code>	Per-category method + status selection
Banner wizard	<code>BannerSetupWizard.js</code>	Guided 7-step banner configuration

Canonical category IDs

These are the exact string values used as `category` in API calls and in MongoDB:

Category ID	Display Name	Pricing Engine Function
banners	Banners	calculate_banners
rigid_signs	Rigid Signs	calculate_rigid_signs
cut_vinyl	Cut Vinyl	calculate_cut_vinyl
digital_print	Digital Print / Printed Vinyl	calculate_digital_print
vehicle_graphics	Vehicle Graphics / Wraps	calculate_vehicle_graphics
apparel	Apparel	calculate_apparel
services	Services	calculate_services
promotional	Promotional Items	calculate_promotional
custom	Custom / Other	calculate_custom

Note: The route layer normalizes legacy aliases before dispatching:

- `promo_misc` → `promotional`
- `vehicle_wrap`, `vehicle_wraps` → `vehicle_graphics`

2. The 8 Pricing Methods

These are configurable per category by the shop admin. The method determines how the calculator's "quick entry" UI looks and which calculation path runs.

Method Key	Label	Best For	How It Works
flat_price	Flat Price	Simple items, fixed-priced products	Admin enters a fixed sell price. No calculation.
price_per_sqft	Price Per Square Foot	Banner materials, wide-format print	<code>width × height → area_sqft × rate_per_sqft</code>
quantity_tier	Quantity Tier	Yard signs, apparel, bulk orders	Price-per-unit varies by quantity range (e.g., 1–24 = \$X, 25–\$Y)
detailed_material_labor	Detailed Material + Labor	Vehicle wraps, rigid signs	Full cost-build: material + labor + overhead → cost-plus margin
compare_methods	Compare Methods	Any where you want the best of both	Runs both <code>price_per_sqft</code> AND <code>detailed_material_labor</code> , shows both, recommends the higher (or configured) option
manual_quote	Manual Quote	Outsourced, custom projects	Admin enters price manually. Calculation suppressed.
hourly	Hourly	Design, installation, services	<code>hours × hourly_rate</code> . Minimum billable hours applies.
package	Package Pricing	Vehicle lettering packages, event bundles	Pre-defined packages with fixed prices. User selects package

Suggested Method Per Category (from frontend config)

Category	Suggested Method
Banners	compare_methods
Yard Signs	quantity_tier

Rigid Signs	compare_methods
Printed Vinyl / Digital Print	compare_methods
Cut Vinyl	compare_methods
Vehicle Lettering	package
Vehicle Wraps	compare_methods
Apparel	quantity_tier
Design	hourly
Installation	hourly
Custom / Promotional	manual_quote

3. Global Shared Concepts

These concepts apply across all categories.

3.1 Billable Area Minimum

Every area-based category has a `default_minimum_billable_area`. Even if the physical piece is smaller, the shop is charged for at least this area.

Category	Default Minimum
Cut Vinyl	0.5 sqft
Rigid Signs	1.0 sqft
Digital Print	1.0 sqft
Banners	4.0 sqft

3.2 Waste Percentage

Material is ordered with waste. The waste-adjusted area is what material costs are calculated on.

waste_adjusted_area = total_billable_area × (1 + waste_percent / 100)

Category	Typical Waste
Banners	8%
Rigid Signs	configurable
Cut Vinyl	configurable
Vehicle Wraps	12–15% by coverage type
Digital Print	configurable

3.3 Labor System — Minutes vs. Hours

The pricing engine supports two labor systems:

- Minute-based (new):** `labor_minutes` per order/piece × `shop_labor_rate_per_hour`. Optionally has an `include_labor` toggle (if `false`, labor is tracked internally but not added to the sell price).
- Hours-based (legacy fallback):** `production_labor_hours_per_sqft` × area + `min_hours`. Used if minute-based config is absent.

3.4 Design Charge System

Design charge is controlled by `get_design_charge_config(defaults)` which returns:

- `charge_separately`: "yes" | "no" | "over_included_minutes" - `default_design_rate`: hourly rate for billable design
- `included_minutes`: free design minutes per job (only when `charge_separately = "over_included_minutes"`)

The behavior:

- `artwork_ready = true` → no design charge (customer provided ready file)
- `artwork_needed = false` → no design charge
- `charge_separately = "no"` → design tracked internally, not billed
- `charge_separately = "yes"` → full design hours × design rate billed
- `charge_separately = "over_included_minutes"` → only billable minutes beyond `included_minutes`

3.5 Rush Order Multiplier

Applied as a percentage surcharge to the suggested price when `rush_order: true`.

```
suggested_price = apply_rush_order_multiplier(suggested_price, data.rush_order)
```

Configured in defaults as `rush_fee_percentage`.

3.6 Overhead Cost

```
overhead_cost = (basis_amount × overhead_percentage / 100) + (labor_hours × shop_overhead_per_hour)
```

- `basis_amount` = material_cost + labor_cost (setup fees deliberately excluded from basis)
- Configurable: `overhead_percentage` (%) and `shop_overhead_per_hour` (\$/hr)

3.7 Quantity Discounts

Configured as a list of tier objects per category:

```
[
  { "min_qty": 25, "max_qty": 99, "discount_percent": 5 },
  { "min_qty": 100, "max_qty": null, "discount_percent": 10 }
]
```

Applied to `sell_base` before adding design/install/setup fees.

3.8 Standardized Response Shape (PricingCalculation)

Every calculation returns the same structure:

<code>suggested_price</code>	float	Final suggested sell price
<code>total_cost</code>	float	Sum of all cost buckets
<code>profit_margin_percent</code> <code>suggested_price × 100</code>	float	(<code>suggested_price - total_cost</code>) /
# Cost buckets		
<code>material_cost</code>	float	
<code>labor_cost</code>	float	
<code>design_cost</code>	float	

```

setup_cost                float
finishing_cost            float
hardware_cost             float
install_cost             float
outsourcing_cost         float
overhead_cost            float

# Metadata
estimated_labor_minutes  float
pricing_method           string  "sell_rate" | "markup" | "hourly" | "manual"
markup_multiplier        float | null
target_margin_percent    float | null
minimum_charge           float

# Breakdown arrays (itemized line-level detail)
materials_breakdown      [{name, quantity, unit, unit_cost, total_cost}]
labor_breakdown          [...]
design_breakdown          [...]
setup_breakdown          [...]
finishing_breakdown      [...]
hardware_breakdown       [...]
install_breakdown        [...]

# Overhead explainability
overhead_basis           {formula, basis_amount, basis_components, labor_hours,
overhead_percentage, ...}

# Dimensions
area_sqft               float | null
billable_sqft           float | null
quantity                float
width_inches            float | null
height_inches           float | null
waste_percentage        float | null

warnings                 [string]  Human-readable warnings (missing materials,
fallbacks used)
legacy_breakdown         {}        Full original calculation debug dict
(backward-compatible)

```

4. Category: Banners

Use cases: 13oz vinyl banners, 18oz vinyl banners, mesh banners, fabric banners, step-and-repeat backdrops, pole banners.

4.1 Inputs

Input Field	Type	Default	Notes
width_inches	float	—	Width in inches OR feet depending on unit_of_measure
height_inches	float	—	Height / length in same unit
unit_of_measure	string	"feet"	"inches" or "feet"
quantity	float	1	Number of banners
banner_material_key	string	"banner_13oz"	Material library key
banner_double_sided	string	"no"	"no" / "same" / "different"
banner_hems	string	"standard"	"none" / "standard" / "reinforced"
banner_grommets	string	"corners"	"none" / "corners" / "every_2ft" / "every_3ft" / "custom"
banner_grommet_count	int	—	Only for "custom" grommet mode
banner_pole_pockets	string	"none"	"none" / "top" / "top_and_bottom" / "side_pockets"
banner_reinforced_corners	bool	false	Extra corner reinforcement
banner_wind_slits	bool	false	Wind relief cuts
banner_specialty_sewing	bool	false	Specialty sewn finish
banner_laminate	bool	false	Laminate/coating applied
banner_laminate_type_key	string	"banner_laminate_coating"	Material library key
banner_hardware_keys	[string]	[]	Hardware accessories from hardware_accessories config
banner_use_type	string	"outdoor"	"indoor" / "outdoor" / "backwall_step_repeat" / "event_display"
banner_event_premium	bool	null	Force event premium (auto-detected from banner_use_type)
artwork_ready	bool	false	If true, no design charge
artwork_needed	bool	null	null = assume needed
design_complexity	string	"simple"	"simple" / "medium" / "complex" / "extreme"
install_required	bool	false	Add installation labor
install_complexity	string	"easy"	"easy" / "medium" / "difficult"
rush_order	bool	false	Apply rush surcharge

4.2 Calculation Flow

1. Resolve dimensions → area_per_piece → billable_area (min 4 sqft)
2. Apply waste (default 8%) → waste_adjusted_area
3. MATERIAL COST

$$\text{banner_material_cost} = \text{waste_adjusted_area} \times \text{material_cost_per_sqft} \times \text{sided_mult}$$

$$\text{print_consumable_cost} = \text{waste_adjusted_area} \times \text{consumable_cost_per_sqft} \times \text{sided_mult}$$

$$\text{laminate_cost} = \text{waste_adjusted_area} \times \text{laminate_cost_per_sqft} \times \text{sided_mult} \quad [\text{if laminate}]$$

$$\text{grommet_material_cost} = \text{total_grommets} \times \text{grommet_cost_each}$$

$$\text{hardware_cost} = \text{sum}(\text{hardware item purchase_costs}) \times \text{quantity}$$
4. FINISHING SELL COMPONENTS (added to sell price, not cost basis)

$$\text{hem_cost} = \text{perimeter_feet} \times \text{hem_rate_per_linear_foot} \times \text{quantity}$$


```

grommet_sell      = max(total_grommets × grommet_sell_each,
grommet_min_charge × qty)
pole_pocket_cost  = pole_linear_feet × pole_pocket_rate × quantity
reinforced_corners = flat_charge × quantity [if enabled]
wind_slit_cost    = flat_charge × quantity [if enabled]
specialty_sewing_cost = perimeter_feet × specialty_sewing_rate × quantity
[if enabled]

5. LABOR COST
production_cost = labor_minutes/60 × shop_labor_rate [OR fallback:
hrs_per_sqft × area × qty × rate]
design_cost      = design_hours × design_rate [see §3.4]
install_cost    = max(install_minimum, install_hours × install_rate) [if
install_required]
finishing_labor = grommet_labor_hours × finishing_rate
hardware_labor  = hardware_labor_minutes × production_rate

6. OVERHEAD
overhead_cost = calculate_overhead_cost(material + labor, total_hours,
defaults, cfg)

7. SELL PRICE
sell_base      = material_sell_rate × total_billable_area × sided_mult
sell_base      = max(sell_base, min_sell_per_item × quantity)
suggested_price = sell_base + finishing_sell + design_cost + install_cost +
hardware_sell
suggested_price *= (1 - discount_percent/100) [quantity discount]
suggested_price *= event_premium_multiplier [if event/backwall use type]
suggested_price *= rush_multiplier [if rush_order]

```

4.3 Sidedness Multipliers

Mode	Key	Typical Multiplier
Single-sided	single	1.0
Double same art	double_same	~1.6
Double different art	double_diff	~1.9

4.4 Grommet Logic

- **corners:** default 4 grommets per banner
- **every_2ft:** `max(4, round(perimeter_feet / 2))`
- **every_3ft:** `max(4, round(perimeter_feet / 3))`
- **custom:** use `banner_grommet_count`
- Minimum charge enforced per item: `grommet_sell_subtotal = max(total × grommet_sell_each, min_charge × qty)`

4.5 Configurable Defaults (stored in `category_defaults.banners`)

```
pricing_method          string    compare_methods (default)
```

default_material	string	13oz_banner
default_unit_of_measure	string	feet
default_minimum_billable_area	float	4.0
waste_percentage	float	8.0
standard_hem_rate_per_linear_foot	float	0.75
reinforced_hem_rate_per_linear_foot	float	1.25
grommet_cost_each	float	0.20
grommet_sell_each	float	0.75
grommet_minimum_charge	float	4.0
grommet_default_corner_count	int	4
pole_pocket_rate_per_linear_foot	float	3.50
reinforced_corners_charge	float	6.0
wind_slit_charge	float	2.0
specialty_sewing_rate_per_linear_foot	float	2.0
default_minimum_sell_price	float	35.0
production_labor_hours_per_sqft	float	0.10
min_production_labor_hours_per_item	float	0.20
install_base_hours	float	0.5
install_hours_per_sqft	float	0.04
event_premium_multiplier	float	~1.15
material_retail_rates	[...]	see BannerSetupWizard
addon_defaults	[...]	hems, grommets, brackets, pole_pockets,
design, setup_fee, install		
product_templates	[...]	small_pole_banner, large_pole_banner

4.6 Materials Required in Library

Material Key	Category	Purpose
13oz_banner	banner_material	13 oz vinyl
18oz_banner	banner_material	18 oz vinyl
mesh_banner	banner_material	Mesh material
fabric_banner	banner_material	Fabric / pole banner
banner_print_consumable	banner_material	Ink/consumable cost
banner_laminate_coating	laminate	Optional laminate

Each material needs:

- shop_cost_per_sqft (cost to shop)
- suggested_material_charge_per_sqft (sell rate used in compare_methods)
- waste_percent, is_active, compatible_categories

5. Category: Rigid Signs

Use cases: Coroplast yard signs, ACM/aluminum composite panel signs, PVC board signs, aluminum plate signs.

5.1 Inputs

Input Field	Type	Default	Notes
height_inches	float	24	unit_of_measure string "inches" quantity float 1 substrate_type_key string "coroplast_4mm" Material library key (overrides substrate_type enum) substrate_type enum — Legacy: coroplast, aluminum, pvc, etc. thickness string — "4mm", "10mm", "0.040", "0.063", "0.080" graphic_method string "direct_print" "direct_print" / "mounted_print" / "cut_vinyl_applied" sidedness string "single" "single" / "double" double_sided_art string "same" "same" / "different" shape_type string "rectangle" "rectangle" / "circle" / "custom_shape" finish_quality string "standard" "standard" / "premium" protective_finish bool false Add protective finish/laminate protective_finish_type string "rigid_finish_standard" Material library key hardware_included bool false Add mounting hardware hardware_type string — Hardware key from hardware_accessories config drill_prep_required bool false Pre-drill holes charge artwork_ready bool false artwork_needed bool null design_complexity string "simple" install_required bool false install_complexity string "easy" rush_order bool false

5.2 Calculation Flow

1. Resolve dimensions → area_per_piece → billable_area (min 1.0 sqft)
2. Apply waste → waste_adjusted_area
3. MATERIAL COST
$$\begin{aligned}\text{substrate_cost} &= \text{waste_adjusted_area} \times \text{substrate_cost_per_sqft} \\ \text{graphic_face_cost} &= \text{waste_adjusted_area} \times \text{graphic_cost_per_sqft} \times \text{sided_mult} \\ &\quad [\text{direct_print: uses direct_print_consumable_key material}] \\ &\quad [\text{mounted_print: uses mounted_print_graphic_key material} + \\ &\quad \text{mounting_hours}] \\ &\quad [\text{cut_vinyl_applied: uses oracal_651 (or custom vinyl key) material}] \\ \text{finish_cost} &= \text{waste_adjusted_area} \times \text{finish_cost_per_sqft} \times \text{sided_mult} \\ &\quad [\text{if protective_finish}] \\ \text{hardware_cost} &= \text{get_hardware_cost}(\text{defaults}, \text{hardware_key}) \times \text{quantity} \quad [\text{if hardware_included}]\end{aligned}$$
4. MULTIPLIERS applied to production hours and sell rate:
$$\begin{aligned}\text{sided_mult:} &\quad \text{single}=1.0, \text{ double_same}=?, \text{ double_diff}=? \quad (\text{from config}) \\ \text{shape_mult:} &\quad \text{rectangle}=1.0, \text{ circle}=1.1\text{--}1.3 \quad (\text{from config}) \\ \text{thickness_mult:} &\quad \text{thin_basic}=1.0, \text{ medium}=1.1, \text{ thick_heavy}=1.2 \quad (\text{from config}) \\ \text{finish_quality_mult:} &\quad \text{standard}=1.0, \text{ premium}=1.1\text{--}1.2 \quad (\text{from config})\end{aligned}$$
5. LABOR
$$\begin{aligned}\text{production_hours} &= \text{base_hrs_per_sqft} \times \text{billable_area} \times \text{quantity} \times \\ &\quad \text{thickness_mult} \times \text{shape_mult} \times \text{sided_mult} \\ \text{mounting_cost} &= \text{mounting_hours} \times \text{production_rate} \quad [\text{if mounted_print graphic method}] \\ \text{design_cost} &= \text{see §3.4} \\ \text{install_cost} &= \text{install_hrs} \times \text{install_rate} \quad [\text{if install_required}]\end{aligned}$$

```

    hardware_labor      = hardware_handling_labor_cost × quantity [if
hardware_included]
    drill_prep_fee      = drill_prep_fee × quantity [if drill_prep_required]

6. OVERHEAD

    overhead_cost = calculate_overhead_cost(material + labor, all_hours,
defaults, cfg)

7. SELL PRICE

    sell_base = substrate_sell_rate × total_billable_area × sided_mult ×
thickness_mult × shape_mult × finish_quality_mult
    sell_base = max(sell_base, min_sell)
    sell_base *= (1 - discount_percent/100)
    suggested_price = sell_base + design_cost + install_cost + drill_prep_fee +
hardware_sell
    suggested_price *= rush_multiplier

```

5.3 Substrate Keys → Thickness Tiers

Substrate Key	Thickness Tier
coroplast_4mm	thin_basic
coroplast_10mm	thick_heavy
aluminum_040	thin_basic
aluminum_063	medium
aluminum_080	thick_heavy
pvc_3mm	thin_basic
pvc_6mm	medium

5.4 Configurable Defaults

default_substrate_key	string	coroplast_4mm
default_graphic_method	string	direct_print
default_sidedness	string	single
default_shape_type	string	rectangle
default_minimum_billable_area	float	1.0
default_minimum_sell_price	float	25.0
production_labor_hours_per_sqft	float	0.15
min_production_labor_hours_per_item	float	0.2
install_hours_per_sqft	float	0.08
drill_prep_fee	float	3.0
hardware_handling_labor_cost	float	5.0
sidedness_multipliers	{}	single/double_same/double_diff
shape_multipliers	{}	rectangle/circle/custom_shape
thickness_multipliers	{}	thin_basic/medium/thick_heavy
finish_quality_multipliers	{}	standard/premium

6. Category: Cut Vinyl

Use cases: Plotter-cut decals, window lettering, vehicle door lettering (simple), wall lettering, reflective stickers.

6.1 Inputs

Input Field	Type	Default	Notes
height_inches	float	12	
unit_of_measure	string	"inches"	
quantity	float	1	
vinyl_type_key	string		
oracal_651			Material library key (overrides vinyl_type enum)
vinyl_type	enum	—	Legacy vinyl type
num_colors	int	1	Number of vinyl colors (4+ triggers manual review flag)
weeding_complexity	string	"simple"	
simple			
moderate			
complex			
very_complex			
masking_required	bool	null	null = use category default
use_type	string	"indoor"	
indoor			
outdoor			
vehicle			
artwork_ready	bool	false	
artwork_needed	bool		
design_complexity	string	"simple"	
file_cleanup_needed	bool	false	File cleanup fee
install_required	bool	false	
install_complexity	string	"easy"	
surface_type	string	"flat_smooth"	Installation surface type
rush_order	bool	false	

6.2 Calculation Flow

1. Resolve dimensions → area_per_piece → billable_area (min 0.5 sqft)
2. Apply waste → waste_adjusted_area
3. MATERIAL COST
vinyl_cost = waste_adjusted_area × vinyl_cost_per_sqft
transfer_tape_cost = waste_adjusted_area × transfer_tape_cost_per_sqft [if masking_required]
material_cost = vinyl_cost + transfer_tape_cost
4. MULTIPLIERS applied to production hours:
color_mult: 1 color=1.0, 2=1.5, 3=2.0, 4+=2.5 (+ manual_review flag)
weeding_mult: simple=1.0, moderate=1.3, complex=1.7, very_complex=2.5
use_type_mult: indoor=1.0, outdoor=1.1, vehicle=1.2
5. PRODUCTION LABOR
production_hours = base_hrs_per_sqft × billable_area × quantity × color_mult × weeding_mult
production_cost = production_hours × production_rate
6. DESIGN CHARGE (see §3.4)
7. INSTALLATION CHARGE (if install_required)
install_hours = total_billable_area × install_hours_per_sqft × install_complexity_mult × surface_mult
install_cost = max(minimum_install_charge, install_hours × install_rate)
8. FILE CLEANUP FEE (flat, added after sell calculation)
9. OVERHEAD
overhead_cost = calculate_overhead_cost(material + labor, hours, defaults, cfg)

10. SELL PRICE

```
sell_base      = vinyl_sell_rate × total_billable_area × color_mult ×  
weeding_mult × use_type_mult  
sell_base      = max(sell_base, min_sell_per_item) [if sell_method =  
max_of_rate_or_minimum]  
sell_base      *= (1 - discount_percent/100)  
suggested_price = sell_base + design_cost + install_cost + file_cleanup_fee  
suggested_price *= rush_multiplier
```

6.3 Color Multiplier Notes

- 4+ colors sets a `manual_review: true` flag in the response — a warning to the admin to review manually before quoting.
- Color multipliers are configurable per category: `color_multipliers: {"1": 1.0, "2": 1.5, "3": 2.0, "4_plus": 2.5}`

6.4 Vinyl Types in Library

Key	Material Name	Notes
<code>oracal_651</code>	Oracal 651	Standard indoor/outdoor permanent
<code>oracal_751</code>	Oracal 751	Intermediate-cal, longer outdoor life
<code>oracal_951</code>	Oracal 951	Premium cast vinyl
<code>avery_hp750</code>	Avery HP750	High-performance cast
<code>reflective_vinyl</code>	Reflective Vinyl	Higher cost, outdoor signage
<code>metallic_vinyl</code>	Metallic Vinyl	Specialty
<code>fluorescent_vinyl</code>	Fluorescent Vinyl	Specialty
<code>etched_frost_vinyl</code>	Etched / Frost Vinyl	Window application
<code>wall_vinyl</code>	Wall Vinyl	Repositionable
<code>specialty_custom_vinyl</code>	Specialty / Custom	Catch-all

7. Category: Digital Print / Printed Vinyl

Use cases: Wide-format adhesive vinyl prints, posters, canvas prints, backlit film, perforated window film, mounted prints (with substrate).

7.1 Inputs

Input Field	Type	Default	Notes
<code>height_inches</code>	float	24	<code>unit_of_measure</code> string "inches" <code>quantity</code> float 1 <code>print_media_key</code> string "printable_adhesive_vinyl" Material library key <code>print_material</code> enum — Legacy: <code>vinyl_adhesive</code> , <code>poster_paper</code> , <code>canvas</code> , <code>backlit</code> , <code>perforated</code> <code>ink_coverage_percent</code> float 35 % of area with full ink coverage <code>laminate</code> bool false Add laminate/overlaminate <code>laminate_material_key</code> string "laminate_gloss" Material library key <code>mounted_to_substrate</code> bool false Mount print to rigid substrate <code>substrate_material_key</code> string — Material library key <code>substrate_type</code> enum — Legacy: <code>coroplast_4mm</code> , <code>aluminum_040</code> , etc. <code>print_quality_mode</code> string "standard" "draft" / "standard" / "photo_quality" <code>contour_cut_type</code> string "none" "none" / "standard_contour" / "complex_contour" <code>trim_finish_type</code> string "standard" "standard" / "premium" <code>piece_separation_required</code> bool

| false | For gang-printed jobs | | `separated_piece_count` | int | — | Number of pieces to separate | | `include_setup_fee` | bool | false | Add setup fee to price | | `artwork_ready` | bool | false | | | `artwork_needed` | bool | null | | | `design_complexity` | string | "simple" | | | `file_cleanup_needed` | bool | false | | | `install_required` | bool | false | | | `install_complexity` | string | "easy" | | | `rush_order` | bool | false | |

7.2 Calculation Flow

1. Resolve dimensions → `area_per_piece` → `billable_area` (min 1.0 sqft)
2. Apply waste → `waste_adjusted_area`
3. MATERIAL COST


```
media_cost      = waste_adjusted_area × media_cost_per_sqft
ink_cost        = waste_adjusted_area × ink_cost_per_sqft ×
(ink_coverage_percent / 100)
laminate_cost   = waste_adjusted_area × laminate_cost_per_sqft [if
substrate_cost  = waste_adjusted_area × substrate_cost_per_sqft [if
material_cost   = media + ink + laminate + substrate
```
4. QUALITY & CONTOUR MULTIPLIERS


```
quality_mult  = draft=0.9, standard=1.0, photo_quality=1.25
contour_mult  = none=1.0, standard_contour=1.1, complex_contour=1.4
```
5. PRODUCTION LABOR


```
production_hours = base_hrs_per_sqft × area × quality_mult × contour_mult ×
complexity_mult
mounting_hours   = waste_adjusted_area × mounting_labor_hrs_per_sqft [if
mounted]
separation_hours = separated_piece_count × piece_separation_hours_per_piece
[if piece_separation]
```
6. DESIGN CHARGE, INSTALL CHARGE (same as §3.4)
7. FILE CLEANUP FEE (flat add-on)
8. TRIM PREMIUM ADDON (flat per-item add-on if `trim_finish` = "premium")
9. SETUP FEE (optional flat add-on)
10. OVERHEAD


```
overhead_cost = calculate_overhead_cost(material + labor, hours, defaults,
cfg)
```
11. SELL PRICE


```
sell_base      = media_sell_rate × total_billable_area × quality_mult ×
contour_mult
sell_base      += laminate_sell_addon [laminate sell rate × area]
sell_base      = max(sell_base, min_sell)
sell_base      *= (1 - discount_percent/100)
suggested_price = sell_base + design_cost + install_cost + file_cleanup_fee
+ trim_addon + setup_fee
suggested_price *= rush_multiplier
```

7.3 Print Media Keys

Key	Material Name
printable_adhesive_vinyl	Printable Adhesive Vinyl
poster_paper	Poster Paper
canvas	Canvas
backlit_film	Backlit Film
perforated_window_film	Perforated Window Film

8. Category: Vehicle Graphics / Wraps

Use cases: Full vehicle wraps, partial wraps, color change wraps, door lettering, spot graphics, window perforations.

8.1 Inputs

Input Field	Type	Default	Notes
vehicle_type	string	"van_cargo"	Resolved against vehicle base sqft table
coverage_type	string	"spot"	"spot" / "partial" / "half" / "full" / "custom"
custom_coverage_percent	float	—	Only when coverage_type = "custom" (0–100)
estimated_vehicle_sqft	float	—	Override auto-estimated area (from base_sqft × coverage_factor)
quantity	float	1	Number of vehicles
wrap_material_key	string	"wrap_standard_calendared"	Material library key
wrap_laminate_required	bool	null	null = use config default (true for printed graphics)
wrap_laminate_type_key	string	"wrap_laminate_gloss"	Material library key
window_perf_included	bool	null	null = use config default
window_perf_scope	string	"rear"	"rear" / "side" / "full"
surface_prep_required	bool	false	Cleaning/prep time before wrap
surface_prep_hours	float	—	Hours for surface prep
removal_required	bool	false	Old wrap removal
removal_hours	float	—	Hours for removal
second_installer	bool	false	Second installer needed
second_installer_hours	float	—	Hours for second installer
install_complexity	string	—	Applied as difficulty multiplier
seam_complexity	string	"standard"	"standard" / "complex"

| artwork_needed | bool | true | null = assume needed | | design_complexity | string | "medium" | | | rush_order | bool | false | |

8.2 Coverage Factors

Coverage Type	Factor	Notes
spot	10%	Door lettering, small graphics
partial	25%	One panel / partial sides
half	45%	Half-wrap

full	100%	Full vehicle coverage
custom	user-defined%	Interpolated; maps to nearest key for waste %

8.3 Vehicle Base Area Table (in Materials Library, category = vehicle_type)

Vehicle Key	Base Sqft
van_cargo	160
van_transit	195
pickup_truck	120
sedan	95
suv_large	145
semi_trailer	480
box_truck	260

8.4 Calculation Flow

1. COVERAGE RESOLUTION

coverage_factor = {spot: 0.10, partial: 0.25, half: 0.45, full: 1.0}

estimated_area = vehicle_base_sqft × coverage_factor [or data.estimated_vehicle_sqft]

total_area = estimated_area × quantity

2. WASTE (by coverage key)

waste_by_coverage: {spot: 20%, partial: 15%, half: 12%, full: 12%}

waste_adjusted_area = total_area × (1 + waste_percent/100)

3. MATERIAL COST

vinyl_cost = waste_adjusted_area × vinyl_cost_per_sqft

laminate_cost = waste_adjusted_area × laminate_cost_per_sqft [if laminate]

perf_cost = perf_area × perf_cost_per_sqft × (1 + waste%) [if window_perf]

material_cost = vinyl + laminate + perf

4. PRODUCTION / PREP LABOR

production_hours = max(estimated_area × base_hrs_per_sqft, min_prod_hrs) × quantity

5. DESIGN LABOR (see §3.4)

design_time_by_coverage: {spot: 0.5h, partial: 1.5h, half: 2.0h, full: 3.0h}

design_hours × design_complexity_mult

6. SURFACE PREP LABOR (if surface_prep_required)

prep_cost = surface_prep_hours × production_rate

7. REMOVAL LABOR (if removal_required)

removal_cost = removal_hours × removal_rate

8. INSTALL LABOR

```

install_base_hours = estimated_area × install_hours_per_sqft
install_hours      × install_complexity_mult × seam_complexity_mult
install_cost       = max(install_minimum, install_hours × install_rate)

```

9. SECOND INSTALLER (if second_installer)

```
second_cost = second_installer_hours × helper_rate
```

10. OVERHEAD

```
overhead_cost = calculate_overhead_cost(material + all_labor, all_hours,
defaults, cfg)
```

11. SELL PRICE

```

Package benchmarks: benchmarks[vehicle_type][coverage_key] ×
Cost-plus: (material + labor + overhead) × markup
suggested_price = max(package_benchmark, cost_plus_sell)
+ perf_sell (window perf sell at area × sell_rate_per_sqft)
+ hardware / add-ons
≥ min_sell_per_vehicle × quantity
× rush_multiplier

```

8.5 Wrap Materials in Library

Key	Category	Description
wrap_standard_calendared	wrap_vinyl	Entry-level wrap film (Avery MPI 1005)
wrap_premium_cast	wrap_vinyl	Premium cast wrap film (3M 1080)
wrap_color_change	wrap_vinyl	Solid color change film
wrap_laminate_gloss	wrap_laminate	Gloss overlamine
wrap_laminate_matte	wrap_laminate	Matte overlamine
wrap_laminate_satin	wrap_laminate	Satin finish
wrap_window_perf	window_perf	Perforated window film

9. Category: Apparel

Use cases: T-shirts, hoodies, polo shirts, hats (fitted / snapback / trucker), decorated with HTV, screen print, embroidery, DTF, sublimation.

9.1 Inputs

Input Field	Type	Default	Notes
quantity	float	1	Total piece count (sum of all sizes)
apparel_product_type	string	"short_sleeve_tee"	Key from available_product_types config
apparel_brand_style_key	string	—	Key from available_brand_styles[product_type] config
apparel_placement_set	string	"front"	"front" / "front_back" / "front_back_sleeve" / etc.
apparel_decoration_method	string	"htv"	"htv" / "screen_print" / "embroidery" / "dtf" / "sublimation"
apparel_num_colors	int	1	Number of print colors (for screen print / HTV)

apparel_stitch_count	int	6000	Stitch count (for embroidery)
apparel_plus_size_count	int	0	Number of plus-size pieces in order
apparel_custom_name_number	bool	false	Add names/numbers
apparel_custom_name_number_count	int	0	How many names/numbers
apparel_specialty_finish	bool	false	Puff, foil, or specialty finish
apparel_two_tone_hat_finish	bool	false	Hats only — two-tone finish
apparel_leather_patch	bool	false	Hats only — leather patch
apparel_bag_and_fold	bool	false	Individual bag and fold finishing
apparel_rush_percent	float	17.5	Rush surcharge % (overrides category default)
apparel_manual_quote_override	float	—	Admin manual price override
blank_cost_override	float	—	Override blank garment cost per piece

| artwork_needed | bool | null | | | design_complexity | string | "simple" | | | rush_order | bool | false | |

9.2 Calculation Flow

1. QUANTITY TIER RESOLUTION

```
tiers: [{min_qty: 1, max_qty: 5, key: "1_5"}, {min_qty: 6, max_qty: 11, key: "6_11"}, ...]
tier_key = first tier where min_qty ≤ qty ≤ max_qty
```

2. BLANK COST

```
blank_material = find_material(defaults, brand_style_key)
blank_cost_per_piece = material.cost_per_unit [or blank_cost_override]
total_blank_cost = blank_cost_per_piece × qty
```

3. SUGGESTED SELL (two paths)

PATH A — SHOP TABLE (uses_shop_table = true)

```
per_piece_sell = shop_pricing_table[brand_key][tier_key][placement_key]
decoration_sell = per_piece_sell × qty
[shop table price already includes blank markup]
```

PATH B — COST-PLUS FALLBACK (non-table methods or missing table rows)

```
setup_fee_flat = method_cfg.default_setup_fee
material_per_piece = depends on method:
    htv/screen_print: material_cost_per_color_per_piece × num_colors
    dtf: material_cost_per_piece
    embroidery: cost_per_1k_stitches × (stitch_count / 1000)
    sublimation: material_cost_per_sqin × ~80 sqin
labor_per_piece = apparel_labor_minutes_per_piece / 60 × prod_rate
setup_amortized = setup_fee_flat / qty
cost_per_piece = blank + material + labor + setup_amortized
per_piece_sell = max(cost_per_piece × markup, min_sell_per_piece)
```

4. BASE SELL

```

    retail_base = material.retail_base_no_print [floor price even without
decoration]
    per_piece_sell = max(per_piece_sell, retail_base)
    decoration_sell = per_piece_sell × qty

5. ADD-ONS (all additive to decoration_sell)
    plus_size_cost = plus_size_count × plus_size_rate_per_x [garments only]
    custom_nn_cost = custom_name_number_count × (hat_rate or garment_rate)
    specialty_cost = specialty_finish_rate × qty [if specialty_finish]
    two_tone_cost = two_tone_hat_finish_rate × qty [hats only]
    patch_cost = leather_patch_hat_rate × qty [hats only]
    bag_fold_cost = bag_and_fold_each × qty

6. DESIGN / SETUP FEE
    setup_fee = design_complexity_setup_fees[complexity] [if artwork_needed and
!artwork_ready]
    method_setup = method_cfg.default_setup_fee [if shop table and not
setup_included_in_table]

7. RUSH
    suggested_price *= (1 + rush_percent/100) [if rush_order]

8. MINIMUM
    suggested_price = max(suggested_price, min_sell_per_piece × qty)

9. MANUAL OVERRIDE
    if apparel_manual_quote_override > 0: suggested_price = override

```

9.3 Decoration Methods

Key	Description	Material Cost Driver
htv	Heat Transfer Vinyl	material_cost_per_color_per_piece
screen_print	Screen Print	material_cost_per_color_per_piece
dtf	Direct-to-Film	material_cost_per_piece
embroidery	Embroidery	cost_per_1k_stitches
sublimation	Dye Sublimation	material_cost_per_sqin

9.4 Apparel Config Structure

```

available_product_types:    [{key, label, is_hat, allowed_placement_set}]
available_brand_styles:    {product_type_key: [{key, label, cost_per_unit,
retail_base_no_print}]}
available_decoration_methods: [string]
default_decoration_method: string
methods_using_shop_table:  [string]
method_config:             {method_key: {uses_shop_table, default_setup_fee,
setup_included_in_table, ...}}
quantity_tiers:            [{min_qty, max_qty, key}]

```



```

shop_pricing_table:      {brand_key: {tier_key: {placement_key: price}}}
plus_size_upcharge_per_x: float
custom_name_number_garment: float
custom_name_number_hat: float
specialty_finish_garment: float
specialty_vinyl_hat:    float
two_tone_hat_finish:    float
leather_patch_hat:      float
bag_and_fold_each:      float
default_setup_fee:      float
design_complexity_setup_fees: {simple, medium, complex}
default_rush_percent:    float    17.5
default_markup_multiplier: float    2.15
default_minimum_sell_price: float    10.0
apparel_labor_minutes_per_piece: float
apparel_handling_labor_minutes_per_piece: float

```

10. Category: Services

Use cases: Design hours, installation, removal, site survey, consultation, travel, equipment rental.

10.1 Service Types (configured in `available_service_types`)

Key	Label	Billing Unit	Notes
<code>installation</code>	Installation	<code>hour</code>	requires_travel <code>removal</code> Removal / Demolition <code>hour</code> <code>site_survey</code> Site Survey <code>hour</code> requires_travel <code>consultation</code> Consultation <code>hour</code> <code>travel</code> Travel / Mileage <code>mile</code> billing_unit = "mile" <code>other_labor</code> Other Labor <code>hour</code> <code>general_labor</code> General Labor <code>hour</code> fallback

10.2 Billing Units

Unit	How Quantity Maps	Cost Calculation
<code>hour</code>	quantity = hours	$\text{hours} \times \text{num_workers} \times \text{labor_cost_rate} \times \text{complexity_mult}$
<code>flat</code>	quantity = jobs	flat_fee (ignores hours, uses estimated_hours for cost tracking only)
<code>piece</code>	quantity = pieces	$\text{qty} \times \text{unit_rate} \times \text{complexity_mult}$
<code>sqft</code>	quantity = sqft	$\text{qty} \times \text{unit_rate} \times \text{complexity_mult}$
<code>linear_foot</code>	quantity = linear feet	$\text{qty} \times \text{unit_rate} \times \text{complexity_mult}$
<code>mile</code>	quantity = miles	$\text{miles} \times \text{travel_cost_per_mile}$
<code>trip</code>	quantity = trips	$\text{trips} \times \text{trip_cost_rate} + (\text{hours} \times \text{workers} \times \text{labor_rate})$
<code>day</code>	quantity = days	$\text{days} \times \text{day_hours} \times \text{workers} \times \text{labor_cost_rate} \times \text{complexity_mult}$

10.3 Inputs

Input Field	Type	Default	Notes
<code>quantity</code>	float	1	Depends on billing unit: hours, miles, pieces, etc.

service_type	string	"general_labor"	Key from available_service_types
services_billing_unit	string	—	Overrides service type default
services_labor_role	string	—	Overrides service type default
hourly_rate_override	float	—	Override sell rate per hour
services_flat_fee	float	—	Override flat fee
services_unit_rate_override	float	—	Override rate per unit
estimated_hours	float	0	Required for hour/flat/piece billing
num_workers	int	1	Rounds to nearest whole integer
services_complexity	string	"medium"	"simple" / "medium" / "complex" / "extreme"
services_minimum_applies	bool	true	Apply minimum billable hours
services_travel_required	bool	—	Auto from service type requires_travel
services_travel_miles	float	0	One-way miles
services_trip_charge_applies	bool	false	Add trip charge
services_trip_count	int	1	Number of trips
services_equipment_required	bool	false	Add equipment rental
services_equipment_type	string	"custom"	Key from equipment_library
services_equipment_days	float	0	Equipment rental days
services_equipment_hours	float	0	Equipment rental hours
services_subcontracted	bool	—	Auto from service type
services_subcontract_cost	float	—	Subcontractor cost

| services_permit_external_fee | float | 0 | Permit or external pass-through | | rush_order | bool | false | |

10.4 Complexity Multipliers (applied to labor cost AND sell)

```
complexity_multipliers:
  simple: 0.85
  medium: 1.25 (default)
  complex: 1.50
  extreme: 2.00
```

10.5 Additional Cost Components

- **Travel:** $\text{travel_miles} \times \text{travel_cost_per_mile}$ (\$0.65/mi) → $\text{travel_sell: travel_miles} \times \text{travel_sell_rate}$ (\$1.25/mi)
- **Trip charge:** $\text{trips} \times \text{trip_cost_rate}$ (not applied when billing_unit is already mile or trip — avoids double-billing)
- **Equipment:** Per-day + per-hour rates from equipment_library config
- **Subcontract:** $\text{subcontract_cost} \times (1 + \text{subcontract_markup_percent}/100)$ (default 20% markup)
- **Permit/pass-through:** Added to both cost and sell at 1:1 (no markup by default)

10.6 Sell Method

```
services_sell_method: "cost_plus" | "pass_through_plus_markup" | "max_of_both"

cost_plus: (total_cost + overhead) × markup_multiplier
pass_through_plus_markup: labor_sell_baseline + travel_sell + equipment_sell + subcontract_sell + permit_sell
```

max_of_both: max(cost_plus, pass_through_plus_markup)

Each service type can define a `default_suggested_sell_per_hour` that is used as the labor sell rate baseline.

11. Category: Promotional Items

Use cases: Custom magnets, branded merchandise, keychains, pens, branded apparel outsourced to a vendor, promotional kits.

11.1 Inputs

Input Field	Type	Default	Notes
<code>unit_cost</code>	float	0	Vendor cost per unit <code>promo_product_type</code> enum — Product type label for breakdown display
<code>markup_percent</code>	float	null	null = use default markup <code>double_sided_art</code> string null <code>null</code> /"same"/"different" — affects material cost multiplier <code>include_setup_fee</code> bool false Add setup fee to price <code>setup_fee</code> float — Override setup fee amount <code>rush_order</code> bool false

11.2 Calculation Flow

- `base_cost` = `unit_cost` [or `defaults.misc_material_cost`]
double_sided multiplier:
 "different": × 1.5 (different art both sides)
 "same": × 1.2 (same art both sides)
 null/other: × 1.0
- `material_cost` = `base_cost` × `quantity` × `double_sided_multiplier`
- `labor_cost` = (`default_labor_hours_per_unit` × `quantity`) × `production_rate`
- `setup_fee` = `data.setup_fee` OR `defaults.promo_setup_fee` [only if `include_setup_fee`]
 `setup_fee` is added FLAT (not marked up, not in overhead basis)
- `overhead_cost` = `calculate_overhead_cost(material + labor, hours, defaults, cfg)`
- `markup_multiplier` = `data.markup_percent` ? (1 + `pct/100`) : category defaults
- `suggested_price` = (`material` + `labor` + `overhead`) × `markup_multiplier` + `setup_fee`
 × `quantity_discount` [if applicable]
 × `rush_multiplier` [if `rush_order`]

11.3 Configurable Defaults

<code>default_labor_hours_per_unit</code>	float	0.25
<code>default_markup_multiplier</code>	float	2.5
<code>promo_setup_fee</code>	float	15.0

12. Category: Custom / Other

Use cases: Anything not covered by other categories. Full manual entry.

12.1 Inputs

- `unit_cost` (vendor/material cost per unit)
- `quantity`
- `markup_percent` (or uses global default)
- `include_setup_fee` / `setup_fee`
- `rush_order`

12.2 Behavior

Identical to Promotional Items (shares the same structure). Uses `category_defaults.custom` config instead of `category_defaults.promotional`. Suggested for outsourced items where you just need a cost + markup.

13. Cross-Category Features

13.1 Pricing Templates

- Saved per-tenant, per-category configurations
- Stored in `pricing_templates` collection
- Fields: `id`, `tenant_id`, `name`, `description`, `category`, `pricing_data` (full `JobItemPricingData`), `quantity`, `is_favorite`
- Used to pre-fill the calculator for common repeat jobs (e.g., "Standard 3×8 Banner")
- CRUD: `GET/POST/PUT/DELETE /api/pricing/templates`
- Toggle favorite: `PUT /api/pricing/templates/{id}/favorite`

13.2 Historical Invoice Import (Benchmark Setup)

System to upload past invoices (PDF, CSV, XLSX) and extract pricing benchmarks.

Flow:

1. `POST /api/pricing-setup/imports` (upload files)
2. `PUT /api/pricing-setup/imports/{id}/mapping` (map CSV columns)
3. `POST /api/pricing-setup/imports/{id}/analyze` (AI extraction + aggregate)
4. `POST /api/pricing-setup/imports/{id}/review` (accept/reject suggestions) → Accepted suggestions write to `pricing_configuration.selling_price_benchmarks`

Benchmark fields extracted per category:

Category	Benchmark Field
vehicle_wraps, banners, rigid_signs, cut_vinyl	<code>average_sell_price_per_sqft</code>
apparel, custom	<code>average_sell_price_per_unit</code>
services	<code>average_sell_price_per_hour</code>

13.3 Materials Library

All per-tenant materials stored as an array on the `pricing_configuration` document.

Material schema:

id	string	mat-{key}
key	string	canonical key (e.g., "oracal_651")
name	string	display name
category wrap_vinyl etc.	string	vinyl_type banner_material substrate
purchase_type	string	roll sheet piece sqft
shop_cost_per_sqft	float	what shop pays per sqft
waste_percent	float	recommended waste %
markup_percent	float	suggested markup %
suggested_material_charge_per_sqft	float	sell rate used in "compare_methods"
manual_material_charge_per_sqft	float	manual override sell rate
cost_per_unit rolls, blanks)	float	used for piece-based materials (vinyl
retail_base_no_print decoration	float	(apparel only) floor price without
is_active	bool	
compatible_categories	[string]	
notes	string	

Starter banner materials are auto-injected when banner settings are first saved (13oz, 18oz, mesh, fabric).

14. Data Storage Schema

Collection: `pricing_configuration` (one per tenant)

```
{
  "tenant_id": "uuid",
  "updated_at": "ISO string",

  // Global rates
  "hourly_rate": 75,
  "production_hourly_rate": 75,
  "design_hourly_rate": 85,
  "install_hourly_rate": 95,
  "removal_hourly_rate": 65,
  "overhead_percentage": 0,
  "shop_overhead_per_hour": 0,
  "default_markup_multiplier": 2.5,
  "target_profit_margin_percent": 40.0,
  "rush_fee_percentage": 25,
  "minimum_install_charge": 0,
  "file_cleanup_fee_default": 0,
```

```

// Structured labor rates (new system)
"labor_rates": {
  "production": { "hourly_rate": 75, "minimum_charge": 0 },
  "design": { "hourly_rate": 85, "minimum_charge": 0 },
  "installation": { "hourly_rate": 95, "minimum_charge": 125 },
  "finishing": { "hourly_rate": 75, "minimum_charge": 0 },
  "removal": { "hourly_rate": 65, "minimum_charge": 0 }
},

// Design charge settings
"design_charge_separately": "yes",    // "yes" | "no" | "over_included_minutes"
"design_included_minutes": 0,
"design_default_rate": 85,

// Per-category setup
"category_pricing_methods": {
  "banners": "compare_methods",
  "rigid_signs": "compare_methods",
  "cut_vinyl": "compare_methods",
  "apparel": "quantity_tier",
  ...
},
"category_setup_status": {
  "banners": "compare_ready",
  "rigid_signs": "not_started",
  ...
},
"category_defaults": {
  "banners": { ... BannerSetupWizard output ... },
  "rigid_signs": { ... },
  "cut_vinyl": { ... },
  "vehicle_wraps": { ... },
  "apparel": { ... },
  "digital_print": { ... },
  "services": { ... },
  "custom": { ... },
  "promotional": { ... }
},

// Materials library
"materials": [ ... MaterialConfig objects ... ],

// Hardware accessories
"hardware_accessories": [

```



```

    { "id": "hw-...", "key": "...", "name": "...",
      "purchase_cost": 0, "default_sell_price": 0,
      "default_labor_addon_minutes": 0 }
  ],

  // Historical benchmarks (from invoice import)
  "selling_price_benchmarks": {
    "banners": { "average_sell_price_per_sqft": 8.50, "average_order_total":
250, "label": "Banners" },
    "vehicle_wraps": { "average_sell_price_per_sqft": 18.00,
"average_order_total": 2400, "label": "Vehicle Wraps" }
  },

  // AI estimation overrides
  "ai_estimation_rules": { ... },
  "benchmark_rules": { ... },
  "global_calc_rules": { ... },
  "quantity_breaks": { ... }
}

```

Collection: pricing_templates

```

{
  "id": "uuid",
  "tenant_id": "uuid",
  "name": "Standard 3x8 Banner",
  "description": "Default outdoor banner with grommets",
  "category": "banners",
  "pricing_data": { ... JobItemPricingData ... },
  "quantity": 1,
  "is_favorite": false,
  "created_at": "ISO string",
  "updated_at": "ISO string"
}

```

Collection: pricing_imports

```

{
  "id": "uuid",
  "tenant_id": "uuid",
  "created_by": "uuid",
  "status": "mapping_required | ready_for_analysis | analyzed | reviewed",
  "files": [ { "id", "filename", "extension", "storage_path", "size_bytes",
"preview" } ],
  "mapping": { "description_field", "quantity_field", "total_field",
"dimension_field", "category_field", "category_overrides" },
  "normalized_rows": [ ... ],
}

```

```

    "analysis_summary": { "invoice_count", "line_item_count", "top_categories",
"category_metrics", "suggestions", "outlier_rows" },
    "suggestions": [ { "id", "category_key", "benchmark_field", "suggested_value",
"confidence", "status", "final_value" } ],
    "created_at": "ISO string",
    "updated_at": "ISO string"
}

```

15. API Endpoints

Pricing Calculation

POST /api/pricing/calculate → Run calculation for any category

Request:

```

{
  "category": "banners",
  "quantity": 5,
  "pricing_data": {
    "width_inches": 36,
    "height_inches": 72,
    "unit_of_measure": "inches",
    "banner_material_key": "18oz_banner",
    "banner_hems": "standard",
    "banner_grommets": "corners",
    "banner_pole_pockets": "top",
    "artwork_ready": true,
    "rush_order": false
  }
}

```

Pricing Configuration

GET	/api/pricing/defaults	→ Get tenant pricing config
PUT	/api/pricing/defaults	→ Update (deep-merges nested objects)
GET	/api/pricing/settings	→ Alias for /defaults
PUT	/api/pricing/settings	→ Alias for /defaults

Materials

GET	/api/pricing/materials category=banner_material)	→ All materials (or ?
-----	---	-----------------------

Templates

GET	/api/pricing/templates	→ All templates (or ?category=banners)
POST	/api/pricing/templates	→ Create template

```
PUT    /api/pricing/templates/{id}      → Update template
DELETE /api/pricing/templates/{id}      → Delete template
PUT    /api/pricing/templates/{id}/favorite → Toggle favorite
```

Historical Invoice Import

```
GET    /api/pricing-setup/imports      → List all imports
POST   /api/pricing-setup/imports      → Upload files (multipart/form-data)
GET    /api/pricing-setup/imports/{id} → Get import
PUT    /api/pricing-setup/imports/{id}/mapping → Set column mapping
POST   /api/pricing-setup/imports/{id}/analyze → Run AI analysis (uses credits)
POST   /api/pricing-setup/imports/{id}/review → Accept/reject suggestions
POST   /api/pricing-setup/imports/migrate-storage → Migrate local files to object storage
```

16. Rebuild Notes & Warnings

16.1 ALL Money Fields Must Be Integer Cents in the Rebuild

Current state: All prices are `float` (`suggested_price`, `material_cost`, etc.) **Rebuild rule:** Store every monetary value as integer cents. Never use `float` for money.

```
# RIGHT in rebuild
@dataclass
class Money:
    cents: int
    def dollars(self) -> Decimal: return Decimal(self.cents) / 100

# WRONG (current)
suggested_price: float = 8.50
```

16.2 The `_normalize_pricing_payload` Shim Is Technical Debt

The current route normalizes 8+ legacy field aliases before calling the engine:

- `width` → `width_inches`
- `height` → `height_inches`
- `length_inches` → `height_inches`
- `square_footage` → `area_sqft`
- `substrate_type + thickness` → normalized `substrate_type_key`

Rebuild rule: Start with canonical field names only. Kill all aliases from Day 1.

16.3 Calculate Function Dispatch Is a Giant If/Switch in `Server.py`

Current state: `calculate_pricing()` is a 4,500-line function in `server.py`. All 9 category functions live in the same file.

Rebuild rule:

- Each category → its own module: `pricing/categories/banners.py`, `pricing/categories/rigid_signs.py`, etc.
 - `calculate_pricing()` → a registry dict: `CALCULATORS = {"banners": calculate_banners, ...}`
 - Category functions accept `(input: CategoryPricingInput, config: TenantPricingConfig) -> PricingResult` — no raw dicts
-

16.4 The Materials Library Is a Single Array in One Document

Current state: All materials are a flat list embedded in the `pricing_configuration` document. As the library grows (50+ materials), reading and writing this document becomes expensive.

Rebuild rule:

- Move to a dedicated `pricing_materials` collection with `{tenant_id, key, category, ...}` per document.
 - Index on `(tenant_id, key)` — uniqueness enforced at DB level.
 - `find_material()` becomes `await db.pricing_materials.find_one({"tenant_id": t, "key": k})`.
-

16.5 The Labor System Has Two Modes — Unify Them

Current state: Two parallel labor systems: minute-based (new) and hours-based (legacy fallback).

`get_labor_minutes_and_rate()` runs both conditionally with `if labor_minutes > 0`.

Rebuild rule: Commit to one system. Minutes is superior (more granular). Remove the hours fallback. Migration: convert old `production_labor_hours_per_sqft` configs to `setup_minutes + minutes_per_sqft` at startup.

16.6 The Apparel Shop Table Is the Right Idea — Generalize It

The apparel category has a brilliant data structure: `shop_pricing_table[brand_key][tier_key][placement_key] = sell_price`.

The shop owner literally fills in the prices they charge, and the calculator just looks them up.

Rebuild rule: Apply this table-first pattern to yard signs and rigid signs too:

- `rigid_signs_table[substrate_key][size_tier] = sell_price`
 - `yard_signs_table[quantity_tier] = sell_price_per_sign`
 - Only fall back to cost-plus when the table entry is missing.
-

16.7 The Overhead Basis Is Inconsistent Across Categories

In Promotional and Cut Vinyl, the setup fee is explicitly excluded from the overhead basis. In Banners and Vehicle Wraps, it's included. This inconsistency makes margin reporting unreliable.

Rebuild rule: Settle on one definition: overhead is calculated on `(material_cost + direct_labor_cost)` only. Setup fees, design fees, and install fees are pass-through charges — never subject to overhead.

16.8 Pricing Defaults PUT /defaults Deep-Merges Some Fields, Not Others

The current `update_pricing_defaults` route deep-merges `category_defaults`, `labor_rates`, `selling_price_benchmarks`, and a

few others — but any field NOT in that list is overwritten at the top level. This creates subtle bugs when a frontend update only sends a partial object.

Rebuild rule: Use a proper JSON Merge Patch (RFC 7396) for all updates, or design atomic `PATCH /pricing/materials/{key}`, `PATCH /pricing/labor-rates`, `PATCH /pricing/category/{category_id}` endpoints instead of one giant PUT.

16.9 There Is No Pricing Change History

If a shop owner changes their material costs or labor rates, there is no audit trail of what the rates were when old quotes/jobs were created.

Rebuild rule: Snapshot the active pricing config on every quote creation: `quote.pricing_snapshot = { ...pricing_configuration }`. This snapshot is used to reproduce the original calculation, even if rates change later.

16.10 Category Setup Status Is Only a Display Badge

`category_setup_status` (`not_started` / `basic_setup` / `detailed_setup` / `compare_ready` / `needs_review`) is set manually by the admin. The calculator fires regardless of whether a category is "set up" or not — it just uses defaults.

Rebuild rule: Add an optional `calculate_requires_setup_complete: bool` guard at the route level. When enabled, reject calculation requests for categories that are still `not_started`.

Generated: 2026-06-10 | Reverse-engineered from server.py (4,843 lines) + PricingCalculator.js (4,045 lines)

PRICING REBUILD VERIFICATION AND PARITY TEST PLAN

Status: Documentation only. No tests implemented beyond Sprint 1's already-passing 58 unit tests (Money, permissions, change log, golden-formula framework). This plan defines what every future phase must test before being considered done.

How to read this document

For each rebuild area: what must be preserved exactly from the legacy behavior (parity), what is intentionally changed (and why), the required test cases, and the acceptance criteria that close that area out. "Preserve From Legacy" does not mean "preserve the legacy code" — it means the *business-visible outcome* (a specific price for a specific input) must match, even though the implementation underneath is completely rewritten.

Rebuild Area	Preserve From Legacy	Intentionally Change	Required
Live recalculation behavior	The existing two-step UX pattern already confirmed in <code>routes/job_tickets.py (/calculate-pricing live preview, then explicit /save-pricing commit)</code> — this pattern is correct and is being generalized, not replaced	Every category and every surface (Quote Item, Order Item, Wrap Lab, Webstore product setup) gets this pattern; today only the general Job Ticket flow has it	Change size/material the live calculation explicit "V"
Banner examples	Legacy <code>calculate_banners()</code> output for a fixed set of representative inputs (a specific size, material, quantity, finishing combination)	Internal math moves from float to <code>Money</code> /integer cents; warnings become typed	A document as a baseline /cost breakdown
Rigid Sign examples	Same principle as Banners	Same as Banners	Same pattern (drilling)
Cut Vinyl examples	Same principle as Banners	Same as Banners	Same pattern
Digital Print examples	Same principle as Banners	Same as Banners	Same pattern mounting
Vehicle Wrap examples	Explicitly NOT full parity with either legacy engine — Phase 4's own build order requires a documented formula-comparison FIRST, because the Foundation engine and Wrap Lab engine are confirmed to already disagree (M-01/M-25). Parity here means: the new unified engine's output is reviewed and Owner-approved against BOTH legacy outputs, with disagreements explained, not silently resolved one way.	The two disconnected engines become one; Wrap Lab loses its own materials/labor/markup/snapshot	A document output vs. Phase 4 c
Apparel examples	Legacy <code>calculate_apparel()</code> output for representative brand/style/quantity-tier/placement/decoration-method combinations	Float → cents; standardized snapshot output	Golden-f
Service examples	Legacy <code>calculate_services()</code> output for representative hourly/flat/per-unit/travel combinations	Float → cents; standardized snapshot output	Golden-f
Promotional examples	Legacy <code>calculate_promotional()</code> output for representative vendor-cost/markup combinations	Float → cents; standardized snapshot output	Golden-f
Custom examples	Legacy <code>calculate_custom()</code> manual-entry behavior	Gains a real <code>PricingResult/PricingSnapshot</code> /permission gate it may not have had before (Needs More Investigation on the legacy version's exact current behavior)	A manual identicall

Manual override tests	The concept of a manual override (users have always been able to type a different price)	Adds <code>PricingOverride</code> provenance (reason, approver, permission) that did not exist before (M-15)	Attempt a per <code>Permi</code> test <code>prici</code>
Margin warning tests	The concept that a thin-margin sale should be flagged	Warnings become typed with severity, replacing free-text strings (M-13/M-24)	Golden-f exactly on test <code>gold</code>
Below-cost approval tests	The concept that below-cost sales are unusual and worth flagging	Below-cost commits require <code>PRICING_OVERRIDE</code> + a reason, where today they require nothing (M-14)	Golden-f critical w test <code>gold</code>
Snapshot freeze tests	Nothing to preserve from legacy here — this is a NEW guarantee (today's snapshots CAN be silently invalidated by later config changes, since they aren't consistently isolated from live config in every surface, e.g., Wrap Lab's compute-and-save-together pattern)	A committed snapshot is provably immutable against later Foundation/config changes	Commit a rejected/g re-fetch d
Material-cost change tests	N/A (new guarantee, same as above)	A material cost change affects only FUTURE calculations, never past snapshots	Change a (should re (should N
Quote/Order integration tests	The general shape of "a Quote can become an Order, an Order can become an Invoice" (existing conversion functions <code>generate_quote_from_order/</code>	The underlying snapshot each conversion step reads comes from ONE standardized <code>PricingSnapshot</code>	End-to-en same pric
Wrap Lab parity tests	See "Vehicle Wrap examples" above — same documented-comparison-then-Owner-approval principle, not blind parity	instead of <code>Wrap Lab's compute-and-save-together pattern (routes/wrap/core.py L1254-1289)</code> becomes the same <code>jobs/job_items_sync_path</code> (confirmed every node in pattern has <code>routes/invoices.py</code>)	A wrap ti recalculation
Webstore pricing tests	The concept of a public retail price and a fundraiser profit split (both confirmed to work today, <code>compute_event_profit_allocation()</code>)	The internal cost price behind that public price now comes from a real calculator instead of a manually-typed <code>base_cost</code> (M-02)	Create a V non-null a
Webstore override tests	The concept of a store owner being able to set a different public price than the internally suggested one (Assumption to Verify whether this exists today at all — Needs More Investigation #7)	The override, if it exists today with no provenance, gains the same <code>PricingOverride</code> treatment every other category's override gets, and — critically — per the final rebuild rules, it must never erase the internal calculated cost, margin, price, proven	A public below-int
Fundraiser/event pricing tests	<code>compute_event_profit_allocation()</code> 's percentage/fixed-per-item modes and running-total tracking (confirmed sound, <code>routes/webstores.py</code> L1034-1073)	its INPUT (total profit) becomes more accurate once product pricing is fixed — the allocation math itself is unchanged	Re-run th against bo itself proc
Profitability report tests	The general concept of a margin/profit dashboard (exists today, <code>routes/profit_analytics.py</code>)	Its data source moves from the confirmed-legacy <code>jobs/job_items</code> collections to <code>PricingSnapshot</code> records exclusively (M-18/M-19)	Price and same test (reading l for the sa
Inventory/vendor cost tests	<code>services/inventory_service.py</code> 's reservation/allocation logic (confirmed sound, no changes recommended)	Only the potential new link from vendor cost data into <code>pricing_materials.cost_per_sqft</code> (Need <code>PRICING_VIEW</code> gation)	If a link is silent)
Permission tests	Nothing (no dedicated pricing permission existed before Sprint 1)	<code>PRICING_EDIT_FOUNDATION/PRICING_OVERRIDE</code> permissions, already unit-tested (Sprint 1, <code>test_pricing_permissions_sprint1.py</code>	Live-endp route actu
Change-log tests	Nothing (no audit trail existed before Sprint 1)	New <code>PricingChangeLog</code> , including explicit regression checks <code>test_pricing_change_log.py</code> , using an in-memory fake collection)	Live-endp route proc

Test artifacts already in place (Sprint 1, verified passing before this handoff package was written)

- `backend/tests/test_money_utility.py` — 36 tests
- `backend/tests/test_pricing_permissions_sprint1.py` — includes explicit no-regression assertions against pre-existing Admin/Staff/Owner/Platform Admin/Webstore Owner permission lists
- `backend/tests/test_pricing_change_log.py` — 4 tests against an in-memory fake collection
- `backend/tests/test_golden_formula_framework.py` — 7 tests proving the harness itself works (exact-cents, warnings, minimum-charge, rounding, override, snapshot generation)

These are the TEMPLATE every future phase's test suite must replicate at the category/integration level — not a starting point to be discarded.